



VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA STROJNÉHO INŽINIERSTVA

FACULTY OF MECHANICAL ENGINEERING

ÚSTAV AUTOMATIZÁCIE A INFORMATIKY

INSTITUTE OF AUTOMATION AND COMPUTER SCIENCE

**OPTIMALIZÁCIA S POUŽITÍM NÁHRADNÝCH
MODELOV V APLIKÁCIÁCH ZAMERANÝCH NA
KLIMATICKÚ ZMENU**

OPTIMIZATION WITH SURROGATE MODELS IN CLIMATE CHANGE-RELATED APPLICATIONS

DIPLOMOVÁ PRÁCA

BACHELOR'S/MASTER'S THESIS

AUTOR PRÁCE

AUTHOR

Bc. Daniel Dlugoš

VEDÚCI PRÁCE

SUPERVISOR

doc. Ing. Jakub Kúdela, Ph.D.

BRNO 2026

Zadání diplomové práce

Ústav: Ústav automatizace a informatiky
Student: **Bc. Daniel Dlugoš**
Studijní program: Aplikovaná informatika a řízení
Studijní obor: bez specializace
Vedoucí práce: **doc. Ing. Jakub Kúdela, Ph.D.**
Akademický rok: 2025/26

Ředitel ústavu Vám v souladu se zákonem č.111/1998 o vysokých školách a se Studijním a zkušebním řádem VUT v Brně určuje následující téma diplomové práce:

Optimalizace s použitím náhradních modelů v aplikacích zaměřených na klimatickou změnu

Stručná charakteristika problematiky úkolu:

Optimalizace s použitím náhradních modelů ("surrogate models", či "meta models") se používá pro aproximaci kritériálních nebo omezujících funkcí v situacích, kdy je vyhodnocení těchto funkcí prohibitivně nákladné (potřeba fyzických experimentů nebo výpočetně náročných simulací). Jednou z oblastí, kde se používají výpočetně náročné simulace, jsou aplikace zaměřené na klimatickou změnu – ať už jde o optimalizaci rozložení sensorů pro monitorování životního prostředí, nebo o optimální lokaci větrných turbín pro maximalizaci jejich efektivity.

Cílem práce bude řešerše technik používaných v náhradních modelech a jejich aplikace na vybrané aplikační úlohy zaměřené na klimatickou změnu.

Cíle diplomové práce:

Analýza vybraných úloh zaměřených na klimatickou změnu.
Řešerše používaných náhradních modelů a optimalizačních algoritmů.
Implementace vybraných algoritmů pro zvolené úlohy.
Zhodnocení vhodnosti vybraných algoritmů pro dané úlohy.

Seznam doporučené literatury:

JIN, Y. Data-Driven Evolutionary Optimization: An Overview and Case Studies. IEEE Trans. Evol. Comp., 2019, vol. 23, no. 5163, pp. 442–458.

KUDELA, J, MATOUSEK, R. Recent advances and applications of surrogate models for finite element method computations: a review. Soft Computing, 2022, vol. 26, pp. 13709–13733.

HELLAN, SP, et al. Bayesian Optimisation Against Climate Change: Applications and Benchmarks. arXiv preprint, 2023. arXiv:2306.04343v1.

BLIEK, L, et al. Benchmarking surrogate-based optimisation algorithms on expensive black-box functions. Applied Soft Computing 2023, vol. 147, article number 110744.

HELLAN, SP, et al. Bayesian optimisation for active monitoring of air pollution. Proceedings of the AAAI Conference on Artificial Intelligence 2022, vol. 36, issue 11, pp. 11908–11916.

Termín odevzdání diplomové práce je stanoven časovým plánem akademického roku 2025/26

V Brně, dne

L. S.

prof. Ing. Zdeněk Hadaš, Ph.D.
ředitel ústavu

doc. Ing. Jiří Hlinka, Ph.D.
děkan fakulty

ABSTRAKT

Táto diplomová práca sa zaoberá optimalizáciou s využitím náhradných modelov v aplikáciách súvisiacich s klimatickou zmenou. Cieľom práce je preskúmať správanie vybraných optimalizačných algoritmov pri riešení dvoch rozdielnych aplikačných úloh: rozmiestnenia environmentálnych senzorov a rozmiestnenia veterných turbín. Teoretická časť práce opisuje nákladné black-box optimalizačné problémy, princíp náhradného modelovania, vybrané typy náhradných modelov, akvizičné funkcie a algoritmy PyBADs, TuRBO a PySOT. Praktická časť je založená na vytvorení experimentálneho rámca, ktorý umožňuje jednotné spúšťanie, vyhodnocovanie a porovnávanie algoritmov na oboch úlohách. Ako referenčná metóda bol použitý Random Search. Výsledky ukázali, že prínos náhradných modelov závisí od charakteru riešeného problému.

ABSTRACT

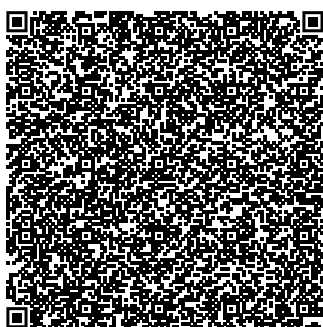
This master's thesis deals with optimization using surrogate models in climate change-related applications. The aim of the thesis is to examine the behaviour of selected optimization algorithms on two different application tasks: environmental sensor placement and wind turbine layout optimization. The theoretical part describes expensive black-box optimization problems, the principle of surrogate modelling, selected types of surrogate models, acquisition functions, and the PyBADs, TuRBO, and PySOT algorithms. The practical part is based on the development of an experimental framework that enables unified execution, evaluation, and comparison of the algorithms on both tasks. Random Search was used as a reference method. The results showed that the benefit of surrogate models depends on the character of the solved problem.

KLÚČOVÉ SLOVÁ

Optimalizácia s náhradnými modelmi, náhradné modely, black-box optimalizácia, Bayesovská optimalizácia, bezderivačná optimalizácia, Gaussovské procesy, radiálne bázové funkcie, polynomiálne modely, akvizičné funkcie, rozmiestnenie veterných turbín, optimalizácia rozmiestnenia, wind wake efekt, rozmiestnenie senzorov, environmentálne monitorovanie, dátovo riadená optimalizácia, strojové učenie, výpočtové modelovanie.

KEYWORDS

Surrogate-based optimization, surrogate models, black-box optimization, Bayesian optimization, derivative-free optimization, Gaussian processes, radial basis functions, polynomial models, acquisition functions, wind turbine layout, layout optimization, wind wake effect, sensor placement, environmental monitoring, data-driven optimization, machine learning, computational modeling.



2026

BIBLIOGRAFICKÁ CITÁCIA

DLUGOŠ, Daniel. *Optimalizace s použitím náhradních modelů v aplikacích zaměřených na klimatickou změnu*. Online, Brno: Vysoké učení technické v Brně, Fakulta strojního inženýrství, Ústav automatizace a informatiky, 2026, 43 s. Diplomová práce. Vedoucí práce: prof. Ing. Docent Jakub KÚDELA, Ph.D. Dostupné z: <https://www.vut.cz/studenti/zav-prace/detail/170570>. [cit. 2026-05-22].

ČESTNÉ PREHLÁSENIE

Prohlašuji, že, že tato práce je mým původním dílem, vypracoval jsem ji samostatně pod vedením vedoucího práce a s použitím odborné literatury a dalších informačních zdrojů, které jsou všechny citovány v práci a uvedeny v seznamu literatury.

Jako autor uvedené práce dále prohlašuji, že v souvislosti s vytvořením této práce jsem neporušil autorská práva třetích osob, zejména jsem nezasáhl nedovoleným způsobem do cizích autorských práv osobnostních a jsem si plně vědom následku porušení ustanovení § 11 a následujících autorského zákona č. 121/2000 Sb., včetně možných trestně právních důsledků.

V Brně dne 22. 5. 2026

.....

Bc. Daniel Dluhoš

OBSAH

1	ÚVOD	9
2	PREHLAD SÚČASNÉHO STAVU POZNANIA.....	10
2.1	Nákladné optimalizačné problémy	10
2.2	Náhradné modeli	11
2.2.1	Tréning náhradného modelu	12
2.3	Typy náhradných modelov.....	12
2.3.1	Polynomiálne modely	13
2.3.2	Modely radiálnych bázových funkcií.....	13
2.3.3	Gaussovské procesy	13
2.4	Optimalizácia s využitím náhradných modelov	14
2.4.1	Sekvenčný optimalizačný proces	14
2.4.2	Výber a vzorkovanie kandidátnych bodov.....	15
2.4.3	Preskúvanie a zlepšovanie riešení	16
2.4.4	Akvizičné funkcie a výberové stratégie	16
2.4.5	Bayesovská optimalizácia	19
2.5	Vybrané algoritmy optimalizácie s náhradným modelom	21
2.5.1	PyBADS.....	21
2.5.2	TuRBO	22
2.5.3	PySOT	24
2.5.4	Porovnanie vybraných algoritmov	25
2.6	Klimaticky orientované aplikačné úlohy	26
2.6.1	Optimalizácia rozmiestnenia veterných turbín.....	26
2.6.2	Optimalizácia rozmiestnenia senzorov	27
3	VLASTNÉ RIEŠENIE	29
3.1	Kritériá výberu aplikačných úloh.....	29
3.2	Formulácia riešených optimalizačných úloh.....	30
3.2.1	Rozmiestnenie environmentálnych senzorov.....	30
3.2.2	Rozmiestnenie veterných turbín.....	30
3.3	Testovacie prostredia klimaticky orientovaných úloh	31
3.3.1	LAQN.....	31
3.3.2	ExpensiveOptimBenchmark	32
3.4	Porovnávané algoritmy a ich použitie v experimentoch	33
3.4.1	Architektúra implementačného rámca	33
3.4.2	Optimalizačné skripty pre rozmiestnenie environmentálnych senzorov	34
3.4.3	Optimalizačné skripty pre rozmiestnenie veterných turbín	35
3.4.4	Hromadné spúšťanie experimentov	37
3.4.5	Výpočtové prostredie	37
3.5	Hodnotiace metriky	38
3.5.1	Kvalita dosiahnutého riešenia	38

3.5.2	Konvergenčný priebeh	39
3.5.3	Efektivita využitia evaluácií.....	39
3.5.4	Časová náročnosť.....	39
3.5.5	Špecifické metriky environmentálnych senzorov	40
3.5.6	Význam metrík pre porovnanie výsledkov	40
4	VÝSLEDKY EXPERIMENTOV.....	42
4.1	Výsledky úlohy rozmiestnenia environmentálnych senzorov.....	42
4.1.1	Kvalita dosiahnutých riešení	42
4.1.2	Konvergenčný priebeh algoritmov.....	43
4.1.3	Efektivita využitia evaluácií.....	51
4.1.4	Časová náročnosť	52
4.2	Výsledky úlohy rozmiestnenia veterných turbín.....	53
4.2.1	Kvalita dosiahnutých riešení	53
4.2.2	Konvergenčný priebeh algoritmov.....	56
4.2.3	Efektivita využitia evaluácií.....	79
4.2.4	Časová náročnosť	82
5	ZHODNOTENIE A DISKUSIA	86
5.1	Súhrnné porovnanie diskkrétnej a spojitý úlohy	86
5.2	Interpretácia výsledkov úlohy rozmiestnenia veterných turbín	87
5.3	Interpretácia výsledkov úlohy rozmiestnenia environmentálnych senzorov	87
5.4	Porovnanie algoritmov z pohľadu kvality, konvergenzie a času	88
5.5	Limity implementácie a experimentálneho nastavenia	88
6	ZÁVER.....	90
	ZOZNAM POUŽITEJ LITERATÚRY	92
	ZOZNAM SKRATIEK, SYMBOLOV, OBRÁZKOV A TABULIEK	94
	ZOZNAM PRÍLOH	98

1 ÚVOD

Optimalizačné úlohy sa v technickej praxi objavujú všade tam, kde je potrebné zvoliť najvhodnejšie riešenie z veľkého množstva možných variantov. V mnohých inžinierskych a vedeckých aplikáciách však nie je vyhodnotenie kvality kandidátneho riešenia jednoduché. Hodnota cieľovej funkcie môže byť získavaná prostredníctvom numerickej simulácie, výpočtovo náročného modelu alebo experimentálneho merania. V takýchto prípadoch nie je možné vykonať veľké množstvo vyhodnotení a klasické optimalizačné prístupy sa stávajú časovo alebo výpočtovo neefektívnymi.

Jedným z prístupov k riešeniu takýchto problémov je optimalizácia s využitím náhradných modelov. Náhradný model slúži ako výpočtovo lacnejšia aproximácia pôvodnej cieľovej funkcie. Optimalizačný algoritmus ho využíva na aproximáciu tvaru funkcie v nevyhodnotených častiach priestoru riešení a na výber ďalších kandidátnych bodov, ktoré budú následne vyhodnotené pôvodnou cieľovou funkciou. Cieľom takéhoto postupu je dosiahnuť kvalitné riešenie pri obmedzenom počte evaluácií.

Význam týchto metód rastie najmä v oblastiach, kde sú optimalizačné problémy spojené s modelovaním fyzikálnych, environmentálnych alebo energetických procesov. Medzi takéto oblasti patria aj aplikácie súvisiace s klimatickou zmenou. Pri návrhu technických riešení zameraných na efektívnejšie využívanie obnoviteľných zdrojov energie alebo na monitorovanie životného prostredia je často potrebné rozhodovať o priestorovom rozmiestnení technických prvkov. Typickými príkladmi sú rozmiestnenie veterných turbín vo veternom parku alebo výber vhodných lokalít pre environmentálne senzory.

Táto diplomová práca sa zameriava na využitie optimalizácie s náhradnými modelmi v dvoch klimaticky orientovaných aplikačných úlohách. Prvou úlohou je rozmiestnenie environmentálnych senzorov na základe dát historických meraní zo siete London Air Quality Network. Táto úloha predstavuje diskretný optimalizačný problém, v ktorom sa riešenie vyberá z konečnej množiny kandidátnych lokalít. Druhou úlohou je rozmiestnenie veterných turbín, ktoré je formulované ako spojitý optimalizačný problém s cieľom maximalizovať výkon veternej farmy prostredníctvom simulátoru veterného poľa. Tieto dve úlohy boli zvolené zámerne, pretože umožňujú porovnať správanie algoritmov v rozdielnych typoch optimalizačného priestoru.

Cieľom práce je preskúmať, ako sa vybrané algoritmy založené na náhradných modeloch správajú pri riešení týchto dvoch úloh a za akých podmienok môže byť ich použitie prínosné. V práci sú porovnané algoritmy PyBADs, TuRBO a viaceré konfigurácie knižnice PySOT, ktoré využívajú rôzne typy náhradných modelov a stratégie výberu kandidátnych bodov. Ako referenčná metóda je použitý Random Search, ktorý nevyužíva náhradný model a slúži ako základné porovnanie voči modelovo riadeným prístupom.

Praktická časť práce je založená na vytvorení experimentálneho rámca, ktorý umožňuje jednotné spúšťanie, zaznamenávanie a vyhodnocovanie výsledkov jednotlivých algoritmov. Výsledky sú hodnotené podľa kvality dosiahnutého riešenia, konvergenčného priebehu, efektivity využitia evaluácií a časovej náročnosti.

2 PREHĽAD SÚČASNÉHO STAVU POZNANIA

2.1 Nákladné optimalizačné problémy

V mnohých vedeckých a inžinierskych aplikáciách sa optimalizačné úlohy vyznačujú tým, že vyhodnotenie kritériálnej funkcie je výpočtovo alebo časovo náročné. Takéto úlohy sa označujú ako optimalizačné problémy s nákladnou účelovou funkciou typu black-box. V týchto prípadoch nie je analytický tvar funkcie známy a jej hodnotu je možné získať iba prostredníctvom simulácií, experimentov alebo komplexných výpočtových modelov. Každé vyhodnotenie kritériálnej funkcie preto môže vyžadovať značné výpočtové zdroje alebo čas. [1], [2], [3]

Nákladné optimalizačné úlohy sa objavujú v mnohých oblastiach inžinierstva, vedy a výskumu, napríklad pri aerodynamickom návrhu, optimalizácii hyperparametrov v strojovom učení alebo pri modelovaní environmentálnych systémov. V takýchto aplikáciách je kvalita riešenia často hodnotená pomocou numerických simulácií, napríklad CFD¹, ktoré môžu trvať niekoľko minút až hodín pre jedno vyhodnotenie kandidátneho riešenia. Všeobecný optimalizačný problém možno matematicky zapísať v tvare

$$\min_{x \in \mathcal{X}} f(x)$$

kde x predstavuje vektor rozhodovacích premenných a $f(x)$ je kritériálna funkcia hodnotiaca kvalitu riešenia. V prípade nákladných optimalizačných úloh je funkcia $f(x)$ dostupná iba prostredníctvom tzv. black-box vyhodnotenia, pri ktorej nie sú známe gradienty ani analytický tvar cieľovej funkcie. V praxi sú navyše pozorované hodnoty funkcie často zaťažené šumom spôsobeným meracími chybami alebo numerickými aproximáciami. Pozorovanie kritériálnej funkcie preto možno zapísať ako

$$y_i = f(x_i) + \varepsilon_i$$

kde x_i je konkrétne testované riešenie, y_i je pozorovaná hodnota kritériálnej funkcie a ε_i reprezentuje náhodnú chybu merania alebo simulácie. [1], [3], [4]

Kľúčovou charakteristikou nákladných optimalizačných problémov je obmedzený počet možných vyhodnotení cieľovej funkcie. Tento limit sa formalizuje ako rozpočet evaluácií B

$$N_{eval} \leq B$$

pričom optimalizačný výkon algoritmu sa hodnotí podľa kvality najlepšieho doteraz nájdeného riešenia pri danom počte evaluácií N_{eval} . Tento prístup je štandardný pri porovnávaní algoritmov pre nákladné black-box optimalizačné úlohy. [1], [2], [6]

Tradičné optimalizačné metódy, napríklad evolučné algoritmy alebo heuristické globálne vyhľadávanie, často vyžadujú tisíce až desaťtisíce vyhodnotení cieľovej funkcie. Ak však jedno vyhodnotenie trvá niekoľko minút alebo hodín, takýto prístup sa stáva prakticky nepoužiteľným. Z tohto dôvodu sa vyvinuli špecializované metódy optimalizácie, ktorých

¹ Computational Fluid Dynamics - numerické modelovanie prúdenia tekutín a plynov v technických systémoch.

cieľom je minimalizovať počet potrebných vyhodnotení nákladnej kritériálnej funkcie. [1], [7], [8]

Jedným z najvýznamnejších prístupov je využitie náhradných modelov. Tieto modely aproximujú správanie nákladnej kritériálnej funkcie pomocou výpočtovo nenákladnej predikčnej funkcie. Optimalizačný algoritmus potom využíva túto aproximáciu na identifikáciu sľubných oblastí priestoru riešení a iba vybrané kandidátne riešenia vyhodnocuje pomocou pôvodnej nákladnej kritériálnej funkcie. [3], [4], [9]

Kombinácia náhradného modelovania a optimalizačných algoritmov vedie k metodike nazývanej optimalizácia s využitím náhradných modelov. Tieto metódy iteratívne vytvárajú a aktualizujú model aproximujúci kritériálnu funkciu a využívajú ho na efektívne vyhľadávanie optimálnych riešení. Vďaka tomu je možné dosiahnuť kvalitné riešenia s výrazne menším počtom nákladných vyhodnotení funkcie v porovnaní s klasickými optimalizačnými metódami. [6], [10], [11]

Táto metodika je v aplikáciách súvisiacich s klimatickou zmenou významná. Simulácie environmentálnych procesov alebo výroby energie môžu byť výpočtovo náročné. Medzi typické príklady patria optimalizácia rozmiestnenia veterných turbín alebo umiestňovanie senzorov na monitorovanie životného prostredia. Kritickou požiadavkou je zníženie počtu nákladných simulácií pri súčasnom dosiahnutí spoľahlivých výsledkov optimalizácie. [5], [12], [13]

2.2 Náhradné modely

Jedným z najrozšírenejších prístupov k riešeniu nákladných optimalizačných problémov je využitie náhradných modelov. Náhradný model predstavuje výpočtovo nenákladnú aproximáciu inak nákladnej kritériálnej funkcie. Namiesto opakovaného vyhodnocovania pôvodnej funkcie sa počas optimalizačného procesu používa jej aproximácia, ktorá umožňuje rýchlo odhadovať hodnoty funkcie v rôznych bodoch priestoru riešení. Tým sa výrazne znižuje počet nákladných simulácií alebo experimentov potrebných na nájdenie optimálneho riešenia. [14]

Náhradný model sa vytvára na základe obmedzeného množstva pozorovaných dát získaných vyhodnotením pôvodnej kritériálnej funkcie vo vybraných bodoch priestoru riešení a následne sa používa na predikciu jej hodnôt v ďalších bodoch bez potreby vykonania ďalšej nákladnej simulácie alebo experimentu. Matematicky možno aproximáciu cieľovej funkcie zapísať ako

$$\hat{f}(x) \approx f(x)$$

kde $f(x)$ je pôvodná nákladná účelová funkcia a $\hat{f}(x)$ predstavuje náhradný model aproximujúci účelovú funkciu.

Náhradný model je vytvorený na základe množiny tréningových dát

$$\mathcal{D} = (x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$$

kde x_i predstavujú vstupné parametre systému a $y_i = f(x_i) + \varepsilon_i$ sú hodnoty kriteriálnej funkcie získané simuláciou alebo experimentom. Cieľom je získať model, ktorého výpočet bude nenákladný, no zároveň dostatočne presný na to, aby mohol usmerňovať optimalizačný proces. Pri pravdepodobnostných modeloch je navyše dôležité, aby vedel poskytovať aj informáciu o neistote predikcie. [8], [14]

2.2.1 Tréning náhradného modelu

Konštrukcia náhradného modelu prebieha v troch krokoch.

Začína výberom bodov, v ktorých sa vykoná skutočné vyhodnotenie nákladnej kriteriálnej funkcie. Tento krok sa nazýva vzorkovanie a má zásadný vplyv na kvalitu výsledného modelu. Počet vzoriek určuje, koľko bodov priestoru riešení bude vyhodnotených pomocou pôvodnej nákladnej funkcie. Rozmiestnenie vzoriek určuje, v ktorých častiach priestoru riešení sa tieto body nachádzajú. Ak je vzoriek príliš málo alebo sú sústredené len v obmedzenej časti priestoru, náhradný model nemusí dostatočne zachytiť správanie funkcie mimo týchto oblastí. Naopak, vhodne zvolený počet aj rozloženie vzoriek zvyšujú pravdepodobnosť, že model bude vedieť presnejšie aproximovať priebeh cieľovej funkcie v relevantných častiach priestoru riešení. [14]

Rozlišujú sa dve základné stratégie vzorkovania: statické a adaptívne. Statické vzorkovanie vyberá body vopred podľa určitého návrhu experimentu, zatiaľ čo adaptívne vzorkovanie pridáva nové body postupne podľa toho, kde môže nový výpočet najviac zlepšiť kvalitu modelu. Pri optimalizácii pomocou náhradných modelov je dôležitá rovnováha medzi preskúmaním málo známych oblastí v priestore riešení, a spresňovaním modelu v oblastiach, ktoré sa javia ako sľubné. [14]

Po získaní počiatkových dát sa vytvorí prediktívny model, ktorý aproximuje účelovú funkciu na základe dostupných vstupno-výstupných dvojíc, kde x_i predstavuje konkrétny bod priestoru riešení a teda zvolenú kombináciu hodnôt vstupných premenných, a y_i predstavuje hodnotu cieľovej funkcie získanú jej reálnym vyhodnotením v danom bode. Ide o dvojice typu zadané parametre a zodpovedajúci výsledok, z ktorých sa model učí odhadovať správanie funkcie aj v nových bodoch. Následne sa tento model používa na predikciu hodnôt funkcie v nových kandidátnych bodoch. Ak sa počas optimalizácie vyhodnotia ďalšie body pomocou pôvodnej nákladnej funkcie, súbor dát sa rozšíri a náhradný model sa aktualizuje. Ide o iteratívny proces, v ktorom sa model priebežne spresňuje správanie pôvodného systému. [14]

Posledným bodom konštrukcie náhradného modelu je predikcia. Natrénovaný model sa používa na odhad hodnôt cieľovej funkcie v nových bodoch priestoru riešení. [14]

2.3 Typy náhradných modelov

Úspešnosť náhradného modelovania výrazne závisí od voľby konkrétneho modelu. Líšia sa v tom, ako presne dokážu zachytiť nelineárne správanie cieľovej funkcie, či poskytujú odhad neistoty predikcie a aká je výpočtová náročnosť ich tréningu a aktualizácie. Medzi najčastejšie používané patria polynomiálne modeli, modeli radiálnych bázových funkcií a Gaussovské procesy. [12], [14]

2.3.1 Polynomiálne modely

Polynomiálne modeli aproximujú kritériálnu funkciu pomocou polynómu

$$\hat{f}(x) = \sum_{i=0}^p a_i x^i$$

kde a_i predstavujú parametre modelu získané regresiou. Model sa snaží nájsť hladkú matematickú krivku alebo plochu, ktorá čo najlepšie vystihuje správanie pôvodnej funkcie v okolí pozorovaných bodov.

Výhodou polynomiálnych modelov je ich nízka výpočtová náročnosť. Nevýhodou je obmedzená schopnosť reprezentovať zložitejšie priebehy cieľovej funkcie. Ak funkcia obsahuje viacero lokálnych extrémov alebo nelineárny priebeh v rôznych oblastiach priestoru riešení. Polynomiálna aproximácia sa tak stáva príliš robustnou a nedokáže zachytiť lokálne rozdiely s dostatočnou presnosťou. [14]

2.3.2 Modely radiálnych bazových funkcií

Modeli RBF aproximujú funkciu pomocou súčtu lokálnych funkcií a výsledná predikcia v novom bode je určená podľa jeho vzdialenosti od známych tréningových bodov.

$$\hat{f}(x) = \sum_{i=1}^n \lambda_i \phi(\|x - x_i\|)$$

kde w_i sú váhy modelu, $\phi(\cdot)$ je radiálna funkcia, x_i sú tréningové body. Body, ktoré sú bližšie, majú na predikciu väčší vplyv, zatiaľ čo vzdialenejšie body menší. Výsledná predikcia vzniká kombináciou všetkých lokálnych vplyvov. RBF modeli dokážu, na rozdiel od polynomiálnych modelov, lepšie zachytiť nelineárny priebeh funkcie. Štandardne však neposkytujú prirodzený odhad neistoty predikcie, čo obmedzuje ich priame využitie v Bayesovskej optimalizácii. [14]

2.3.3 Gaussovské procesy

Gaussovské procesy patria medzi najvýznamnejšie pravdepodobnostné náhradné modeli používané pri aproximácii nákladných účelových funkcií. Hlavnou výhodou je, že neposkytujú iba odhad hodnoty funkcie v novom bode, ale aj odhad neistoty tejto predikcie. Táto vlastnosť je dôležitá najmä v úlohách, kde je potrebné rozhodovať, ktoré body priestoru riešení sa majú ďalej vyhodnotiť pomocou pôvodnej nákladnej funkcie. [2], [14], [15]

Formálne sa Gaussovský proces zapisuje v tvare

$$f(x) \sim \mathcal{GP}(m(x), k(x, x'))$$

kde $m(x)$ predstavuje strednú funkciu procesu a $k(x, x')$ kovariančnú funkciu. Stredná funkcia určuje očakávaný priebeh modelovanej funkcie, zatiaľ čo kovariančná funkcia určuje, ako intenzívne súvisí hodnota funkcie v jednom bode s hodnotou v inom bode. [6], [14], [15]

Po zohľadnení pozorovaných dát zostáva predikcia v novom bode normálne rozdelená. Pri šumových pozorovaniach $\varepsilon \sim \mathcal{N}(0, \sigma_n^2)$, kde ε predstavuje náhodnú chybu merania modelovanú normálnym rozdelením so strednou hodnotou 0 a rozptylom σ_n^2 , možno aktualizovaný odhad strednej hodnoty a rozptyl zapísať ako

$$\mu(x_*) = m(x_*) + k(x_*, X) [K(X, X) + \sigma_n^2 I]^{-1} (y - m(X))$$

$$\sigma^2(x_*) = k(x_*, x_*) - k(x_*, X) [K(X, X) + \sigma_n^2 I]^{-1} k(X, x_*)$$

Prvý vzťah určuje očakávanú hodnotu funkcie v novom bode x_* , teda najlepší odhad modelu. Druhý vzťah vyjadruje neistotu tejto predikcie. [2], [15]

V okolí aktuálne vyhodnotených bodov býva neistota modelu menšia, pretože v týchto oblastiach má model dostatok informácií. Naopak, v slabo preskúmaných častiach priestoru riešení je neistota vyššia. Vďaka tejto vlastnosti sú Gaussovské procesy vhodné nielen na aproximáciu cieľovej funkcie, ale aj na riadenie ďalšieho výberu bodov v optimalizačnom procese. [6], [15]

Nevýhodou Gaussovských procesov je narastajúca výpočtová náročnosť pri väčšom počte dátových bodov. S rastúcim počtom vyhodnotených vzoriek rastie aj veľkosť kovariančnej matice, čo zvyšuje náročnosť tréningu a aktualizácie modelu. Pri vyššej dimenzionalite priestoru riešení alebo pri veľkom rozpočte evaluácií sa preto ich použitie stáva menej praktickým. Napriek tomu patria Gaussovské procesy medzi najdôležitejšie modely v úlohách, kde je počet drahých vyhodnotení obmedzený a kde je odhad neistoty zásadný pre ďalšie rozhodovanie. [8], [14], [15]

2.4 Optimalizácia s využitím náhradných modelov

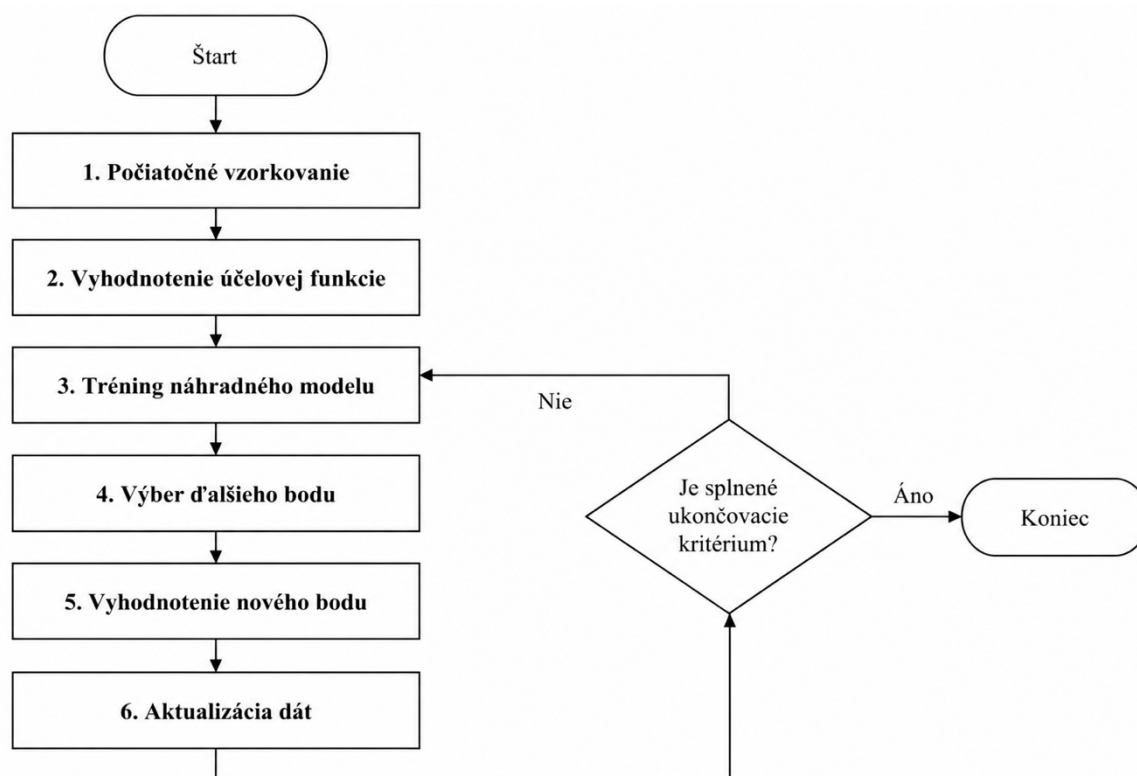
Optimalizácia s využitím náhradných modelov predstavuje metodický rámec, v ktorom sa náhradný model využíva na aproximáciu nákladnej cieľovej funkcie a riadenie výberu nových kandidátnych bodov v priestore riešení, ktoré budú následne vyhodnotené pôvodnou cieľovou funkciou. Pri nákladných black-box úlohách je preto dôležité nájsť kvalitné riešenie a dosiahnuť ho s čo najmenším počtom vyhodnotení pôvodnej cieľovej funkcie. [3], [4]

2.4.1 Sekvenčný optimalizačný proces

Optimalizačný proces prebieha iteratívne ako cyklus rozhodovania, ktorý možno zhrnúť do nasledujúcich krokov:

1. Výber počiatočnej množiny bodov,
2. Vyhodnotenie cieľovej funkcie,
3. Vytvorenie alebo aktualizácia náhradného modelu,
4. Výber nového bodu pomocou rozhodovacej (akvizičnej) funkcie,
5. Vyhodnotenie tohto bodu a jeho pridanie do datasetu.

Náhradný model pri tom aproximuje správanie cieľovej funkcie v priestore riešení na základe aktuálne vyhodnotených kandidátnych bodov, ku ktorým sú známe príslušné hodnoty cieľovej funkcie. Rozhodovanie o ďalšom kroku je následne riadené optimalizačným algoritmom prostredníctvom akvizičnej funkcie, ktorá zohľadňuje kvalitu modelu aj neistotu predikcie. Tento postup sa opakuje až do splnenia zastavovacej podmienky, ktorou môže byť napríklad maximálny počet vyhodnotení cieľovej funkcie alebo stabilizácia najlepšej doteraz nájdenej hodnoty cieľovej funkcie, t.j. situácia, keď sa jej hodnota v priebehu niekoľkých iterácií už významne nemení. [3], [15]



Obr. 1: Postup optimalizácie s využitím náhradného modelu.

2.4.2 Výber a vzorkovanie kandidátnych bodov

V optimalizácii s využitím náhradných modelov zohráva dôležitú úlohu výber bodov, v ktorých bude cieľová funkcia skutočne vyhodnotená. Keďže pri nákladných black-box úlohách nie je možné vyhodnotiť veľké množstvo kandidátnych riešení, musí byť každý nový bod zvolený tak, aby čo najviac prispel k hľadaniu optimálneho riešenia. Nové kandidátne body sa preto nevyberajú všetky naraz pred začiatkom optimalizácie, ale postupne počas jej priebehu. Každý ďalší bod je zvolený na základe doteraz vyhodnotených riešení a aktuálneho náhradného modelu, ktorý odhaduje správanie cieľovej funkcie v nevyhodnotených častiach priestoru riešení. Takýto prístup je typický pre efektívnu globálnu optimalizáciu drahých black-box funkcií, kde je počet evaluácií obmedzený časom alebo výpočtovými nákladmi. [1], [4]

Pojem vzorkovanie môže byť v tomto kontexte chápaný ako proces výberu alebo generovania kandidátnych bodov z priestoru riešení. Môže ísť o počiatočné vzorkovanie, ktorým sú získané prvé potrebné pozorovania na zostavenie náhradného modelu, alebo o priebežné vzorkovanie počas optimalizácie, keď sú na základe modelu vyberané ďalšie body na vyhodnotenie. Vzorkovanie teda nemusí znamenať iba náhodný výber bodov, ale všeobecne označuje spôsob, akým sú z priestoru riešení vyberané kandidátne návrhy pre ďalšiu evaluáciu. [4]

Pri počiatočnom vzorkovaní je cieľom získať informácie o správaní cieľovej funkcie v rôznych častiach priestoru riešení. V tejto fáze ešte náhradný model nemusí byť dostatočne presný. Preto bývajú počiatočné body volené tak, aby aspoň čiastočne pokryli riešenú oblasť. V neskorších iteráciách už výber bodov závisí od aktuálneho náhradného modelu a od stratégie,

ktorá rozhoduje, či má byť uprednostnené zlepšovanie v sľubných oblastiach alebo preskúvanie oblastí s vyššou neistotou. [4]

Vzorkovanie je preto úzko spojené s kompromisom medzi preskúvaním a zlepšovaním riešení. Ak sú nové body vyberané najmä v okolí doteraz najlepšieho riešenia, optimalizácia sa sústreďuje na zlepšovanie známej oblasti. Ak sú naopak vyberané body v menej preskúmaných častiach priestoru, zvyšuje sa šanca objaviť nové sľubné oblasti. Náhradný model a výberová stratégia spolu určujú, akým spôsobom bude tento kompromis riadený. [1], [4]

Špecifickým prípadom modelovo riadeného vzorkovania je Thompsonovo vzorkovanie. Pri tomto prístupe je z pravdepodobnostného náhradného modelu vytvorená jedna možná realizácia cieľovej funkcie a nový kandidátny bod je vybraný podľa tejto realizácie. Takýto postup prirodzene zohľadňuje neistotu modelu, pretože body s vysokou neistotou môžu byť niekedy vybrané ako perspektívne aj vtedy, keď ich stredná predikovaná hodnota nie je najlepšia. [16]

2.4.3 Preskúvanie a zlepšovanie riešení

Veľký kompromis pri optimalizácii s využitím náhradných modelov predstavuje voľba medzi dvoma protichodnými stratégiami. Prvou je preskúvanie priestoru riešení, teda vyhľadávanie nových alebo málo preskúmaných oblastí, ktoré môžu obsahovať kvalitné riešenia. Druhou je zlepšovanie už známych sľubných oblastí, teda sústreďovanie výpočtového rozpočtu do okolia aktuálne najlepších bodov. Efektívna optimalizácia vyžaduje vyváženie oboch prístupov. Uprednostnenie iba jednej stratégie vedie k neefektívnemu využitiu dostupných evaluácií. [1], [2], [6]

Preskúvanie je dôležité, pretože náhradný model nemusí byť spoľahlivý v celom priestore riešení. Naopak, zlepšovanie známych riešení umožňuje spresňovať hľadanie v okolí bodov, ktoré už dosiahli priaznivú hodnotu cieľovej funkcie. Ak by algoritmus preskúmaval iba nové oblasti, mohol by zbytočne plytvať rozpočet evaluácií. Ak by sa naopak sústredil len na zlepšovanie, mohol by uviaznuť v lokálnom optime a prehliadnuť lepšie riešenia v iných častiach priestoru. [2], [6]

V kontexte optimalizácie s využitím náhradných modelov sa tento kompromis posudzuje na základe predikovanej hodnoty cieľovej funkcie a neistoty predikcie. Body s nízkou predikovanou hodnotou sú zaujímavé z hľadiska zlepšovania riešenia, zatiaľ čo body s vysokou neistotou podporujú preskúvanie nových oblastí. [2], [3], [6]

Tento princíp tvorí základ sekvenčného rozhodovania pri optimalizácii s využitím náhradných modelov a priamo vedie k zavedeniu akvizičných funkcií. [2], [6]

2.4.4 Akvizičné funkcie a výberové stratégie

Akvizičné funkcie predstavujú rozhodovacie kritérium, podľa ktorého sa v každej iterácii optimalizačného procesu vyberá ďalší bod na vyhodnotenie. Ich úlohou je previesť informáciu o predikovanej hodnote cieľovej funkcie a neistote modelu do jednej skalárnej veličiny, ktorá umožní porovnať kandidátne body medzi sebou. [2], [6]

Jednou z najčastejšie používaných akvizičných funkcií je očakávané zlepšenie (EI), ktoré uprednostňuje body s vysokou pravdepodobnosťou zlepšenia aktuálne najlepšieho riešenia. Pri

minimalizačnej úlohe sa najprv zavedie zlepšenie oproti aktuálnej najlepšej známej hodnote f_{best} vzťahom

$$I(x) = \max(0, f_{\text{best}} - f(x))$$

pričom samotné očakávané zlepšenie je definované ako

$$EI(x) = \mathbb{E}[I(x)]$$

Ak má predikcia v bode x normálne rozdelenie s parametrami $\mu(x)$ a $\sigma^2(x)$, potom možno očakávané zlepšenie zapísať v tvare

$$EI(x) = (f_{\text{best}} - \mu(x))\Phi(z) + \sigma(x)\phi(z), \quad z = \frac{f_{\text{best}} - \mu(x)}{\sigma(x)}$$

kde Φ označuje distribučnú funkciu a ϕ hustotu štandardného normálneho rozdelenia. Na rozdiel od funkcií $m(x)$ a $k(x, x')$, ktoré definujú strednú hodnotu a mieru neistoty v Gaussovskom procese pred zohľadnením dát, veličiny $\mu(x)$ a $\sigma^2(x)$ vyjadrujú aktualizovanú predikciu modelu v bode x . [1], [2]

Ďalšou možnosťou je pravdepodobnosť zlepšenia (PI), ktorá hodnotí pravdepodobnosť, že nový bod prinesie zlepšenie aspoň o zvolenú toleranciu ξ

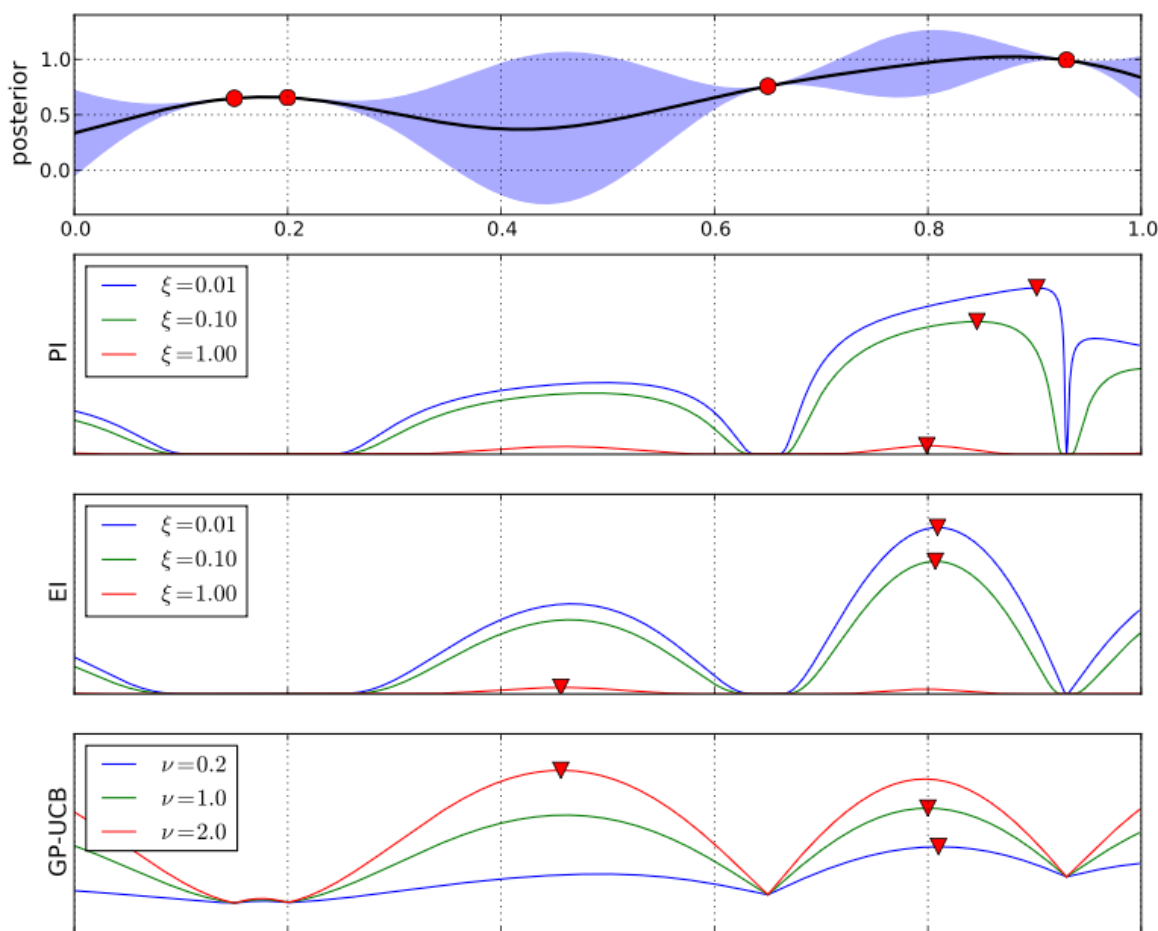
$$PI(x) = P(f(x) \leq f_{\text{best}} - \xi)$$

Výhodou tejto funkcie je jej interpretačná jednoduchosť. Na druhej strane však môže byť pri nevhodne zvolenom parametri ξ príliš krátkozraká a sústreďovať sa predovšetkým na lokálne zlepšenia namiesto systematickejšieho preskúmavania priestoru riešení. [2], [6]

Alternatívou je aj dolná hranica spoľahlivosti (LCB), ktorá explicitne vyvažuje medzi hodnotou predikcie a neistotou modelu. Zapisuje sa v tvare

$$LCB(x) = \mu(x) - \kappa\sigma(x)$$

kde parameter $\kappa > 0$ určuje mieru preskúmavania priestoru riešení. Vyššia hodnota tohto parametra zvýhodňuje body s väčšou neistotou, zatiaľ čo nižšia hodnota kladie väčší dôraz na nízku predikovanú hodnotu cieľovej funkcie. [1], [2]



Obr. 2: Porovnanie akvizičných funkcií PI, EI a UCB nad rovnakým posteriorným modelom. Prevzaté z [11].

Okrem klasických akvizičných funkcií, ktoré vyjadrujú výber kandidátneho bodu jedným explicitným matematickým kritériom, sa pri optimalizácii s využitím náhradných modelov používajú aj širšie výberové stratégie. Tie sa od akvizičných funkcií líšia tým, že výber ďalšieho bodu nemusí byť určený jednou funkčnou hodnotou, ale pozostávajú z viacerých krokov. Tieto stratégie kombinujú viacero rozhodovacích pravidiel, napríklad predikovanú hodnotu cieľovej funkcie náhradným modelom, vzdialenosť od aktuálne vyhodnotených bodov alebo perturbáciu² aktuálneho riešenia. [17], [18], [19], [20], [21]

Stratégia SRBF vychádza z optimalizácie riadenej náhradným modelom radiálnych bázových funkcií. Pri výbere ďalšieho bodu sa najprv vytvorí množina kandidátnych riešení, ktoré sú následne hodnotené podľa kritérií predikovanej hodnoty cieľovej funkcie pomocou náhradného modelu a podľa vzdialenosti od aktuálne vyhodnotených bodov. Kandidáti s priaznivou predikovanou hodnotou podporujú zlepšovanie riešenia, zatiaľ čo kandidáti vzdialenejší od doteraz vyhodnotených bodov podporujú preskúvanie priestoru riešení. Výber bodu je preto založený na kombinácii modelového odhadu a priestorovej rozmanitosti kandidátov. [17], [18], [21]

² Perturbácia predstavuje malú zmenu existujúceho riešenia, napríklad miernym posunutím vybraných súradníc aktuálneho riešenia, čím vznikne nový kandidátny bod.

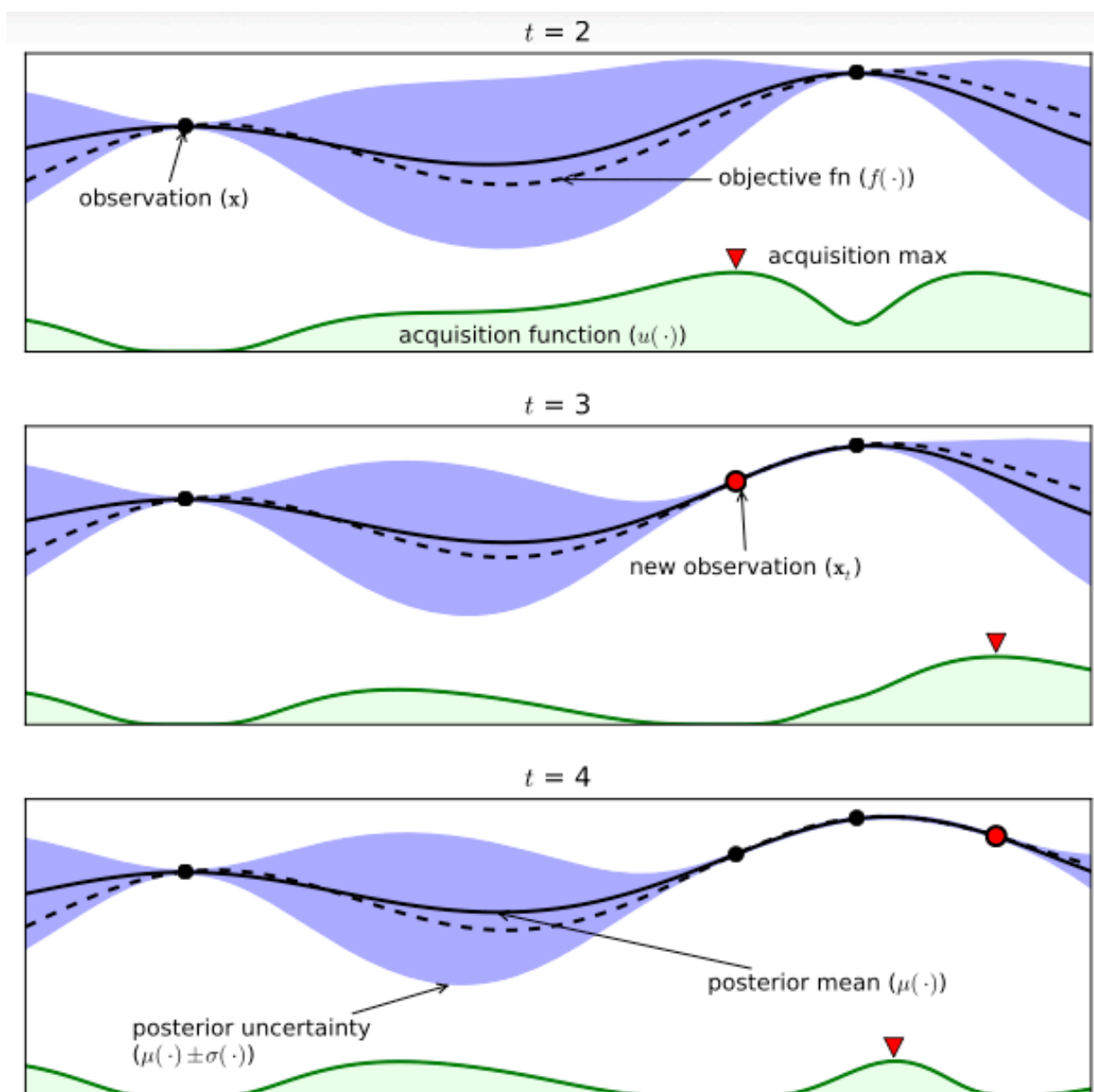
Stratégia DYCORDS rozširuje prístup založený na RBF modeli o dynamické perturbovanie súradníc aktuálneho najlepšieho riešenia. Kandidátne body sú vytvárané tak, že sa náhodne mení iba vybraná podmnožina súradníc aktuálneho bodu. Pravdepodobnosť perturbácie jednotlivých súradníc sa počas optimalizácie postupne znižuje. V skorších fázach sa tým podporuje širšie preskúmavanie priestoru riešení, zatiaľ čo v neskorších fázach sa vyhľadávajú viac sústredujúce na lokálne zlepšovanie v okolí najlepšieho nájdeného riešenia. Táto stratégia bola navrhnutá najmä pre vysokodimenzionálne nákladné black-box optimalizačné problémy. [19]

Stratégia SOP je určená na paralelné vyhodnocovanie kandidátnych bodov pri optimalizácii s využitím náhradných modelov. Jej cieľom je vybrať viacero bodov tak, aby boli zároveň perspektívne podľa náhradného modelu a neboli sústredené iba v jednej malej oblasti priestoru riešení. Vďaka tomu možno efektívnejšie využiť paralelné výpočtové zdroje a zároveň zachovať rozmanitosť vybraných kandidátov. [20], [21]

2.4.5 Bayesovská optimalizácia

Bayesovská optimalizácia predstavuje špecifický prípad optimalizácie s využitím náhradných modelov, určený najmä pre úlohy s nákladnou účelovou funkciou a obmedzeným rozpočtom evaluácií. Na rozdiel od všeobecného prístupu je založená na pravdepodobnostnom modeli cieľovej funkcie, ktorý poskytuje nielen predikciu jej hodnoty, ale aj odhad neistoty v jednotlivých bodoch priestoru riešení. Vďaka tomu umožňuje systematicky rozhodovať o tom, kde bude účelová funkcia vyhodnotená v ďalšom kroku. [1], [2], [6]

Základná myšlienka Bayesovskej optimalizácie spočíva v tom, že sa z aktuálne dostupných dát najprv vytvorí pravdepodobnostný model cieľovej funkcie. Na základe jeho predikovanej hodnoty a neistoty sa potom pomocou akvizičnej funkcie vyberie nový bod, ktorého vyhodnotenie má potenciál priniesť najväčší prínos pre optimalizačný proces. Po vyhodnotení tohto bodu sa dátový súbor rozšíri, model sa aktualizuje a celý postup sa opakuje. [1], [2], [6]



Obr. 3: Iteračný princíp Bayesovskej optimalizácie. Náhradný model sa po každom pozorovaní aktualizuje a akvizičná funkcia určuje ďalší bod na vyhodnotenie. Prevzaté z [11].

Zatiaľ čo všeobecná optimalizácia s využitím náhradných modelov môže pracovať s rôznymi typmi aproximácií a rôznymi pravidlami výberu nových bodov, pre Bayesovskú optimalizáciu je charakteristické práve použitie pravdepodobnostného modelu a explicitná práca s neistotou predikcie. Táto neistota umožňuje prirodzene vyvažovať preskúvanie nových oblastí priestoru riešení a zlepšovanie už nájdených sľubných riešení bez potreby zavádzať toto rozhodovanie osobitne. [1], [2], [6]

Pri implementácii Bayesovskej optimalizácie je potrebné explicitne určiť použitý pravdepodobnostný model, jeho kovariančnú funkciu, hyperparametre, zvolenú akvizičnú funkciu, počiatočný návrh experimentu a celkový rozpočet nákladných vyhodnotení. Hyperparametre predstavujú vnútorné nastavenia náhradného modelu, ktoré ovplyvňujú jeho

predikčné správanie, napríklad mieru hladkosti modelovanej funkcie alebo posudzovanie podobnosti medzi jednotlivými bodmi priestoru riešení. [1], [2], [6]

2.5 Vybrané algoritmy optimalizácie s náhradným modelom

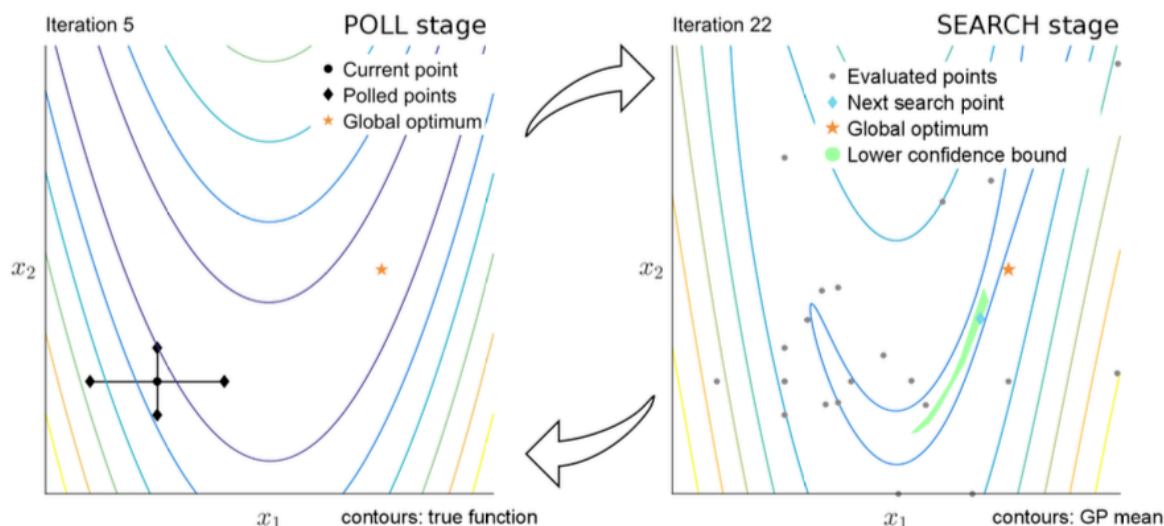
2.5.1 PyBADS

PyBADS je softvérová implementácia algoritmu Bayesian Adaptive Direct Search, určená na riešenie nákladných black-box optimalizačných úloh. Algoritmus BADS bol navrhnutý pre situácie, v ktorých nie je dostupný analytický tvar cieľovej funkcie ani jej derivácie a hodnota funkcie je získavaná prostredníctvom simulácie, numerického výpočtu alebo iného nákladného vyhodnotenia. [22]

Základnou vlastnosťou algoritmu BADS je kombinácia priameho lokálneho vyhľadávania a modelovo riadeného výberu kandidátnych bodov. Nejedná sa teda o klasickú Bayesovskú optimalizáciu ani o náhodné alebo deterministické lokálne vyhľadávanie. Algoritmus využíva doteraz vyhodnotené kandidátne body a im priradené hodnoty cieľovej funkcie. Na základe týchto údajov aktualizuje lokálny model optimalizovanej funkcie, no zároveň si zachováva štruktúru priameho vyhľadávania, v ktorej sa riešenie postupne zlepšuje v okolí aktuálneho kandidátneho bodu. Tento hybridný charakter umožňuje algoritmu pracovať s obmedzeným počtom evaluácií a zároveň obmedziť výpočtovú činnosť spojenú s budovaním modelu. [22], [23], [24]

BADS pracuje so striedaním dvoch základných krokov označovaných ako poll a search. V kroku poll sú kandidátne body generované na lokálnej mriežke v okolí aktuálneho riešenia. Body sú skúšané postupne v jednotlivých smeroch a cieľom je nájsť zlepšenie oproti aktuálnej hodnote. Ak sa zlepšenie nájde, veľkosť kroku sa môže zväčšiť, čím sa podporí rýchlejší postup v sľubnom smere. Ak sa zlepšenie nenájde, veľkosť kroku sa zmenší, čím sa hľadanie sústreďí na jemnejšie lokálne doladovanie riešenia. [23], [24]

Krok search predstavuje modelovo riadenú časť algoritmu. V tejto fáze je na lokálnej podmnožine aktuálne vyhodnotených bodov vytvorený Gaussovský proces, ktorý slúži ako náhradný model cieľovej funkcie. Tento model poskytuje odhad očakávanej hodnoty a neistoty predikcie v kandidátnych bodoch. Na základe týchto informácií sú vybrané ďalšie body na vyhodnotenie pomocou stratégie dolnej konfidenčnej hranice LCB, ktorá umožňuje kombinovať zlepšovanie v oblastiach s priaznivou predikovanou hodnotou a preskúmavanie oblastí s vyššou neistotou. [23], [24]



Obr. 4: Dve fázy algoritmu PyBADS. V poll fáze algoritmus skúša nové body na lokálnej mriežke v okolí aktuálneho riešenia a vyhodnocuje ich priamo pôvodnou cieľovou funkciou. V search fáze sa na základe doteraz vyhodnotených bodov zostaví lokálny Gaussovský proces a ďalší kandidátny bod sa vyberie pomocou dolnej konfidencie hranice. Obrázok ilustruje hybridný charakter PyBADS, ktorý kombinuje priame vyhľadávanie s náhradným modelovaním. Prevzaté z [24].

Dôležité je, že Gaussovský proces v BADS nie je používaný ako globálny model celej cieľovej funkcie. Náhradný model je budovaný lokálne z relevantnej časti aktuálne získaných pozorovaní a slúži na podporu výberu ďalšieho bodu v okolí aktuálneho riešenia. Tým sa znižujú výpočtové nároky na vytvorenie a aktualizáciu náhradného modelu a zároveň sa algoritmus viac sústreďuje na zlepšovanie riešenia v sľubnej oblasti. [22], [23], [24]

Výhodou PyBADS je jeho praktická použiteľnosť pri náročných optimalizačných úlohách bez potreby rozsiahleho ručného nastavovania algoritmu. Z tohto dôvodu je PyBADS vhodný najmä v situáciách, kde je potrebné opakovane riešiť nákladné black-box úlohy a zároveň nie je účelné detailne ladiť veľké množstvo interných parametrov algoritmu. [23], [24]

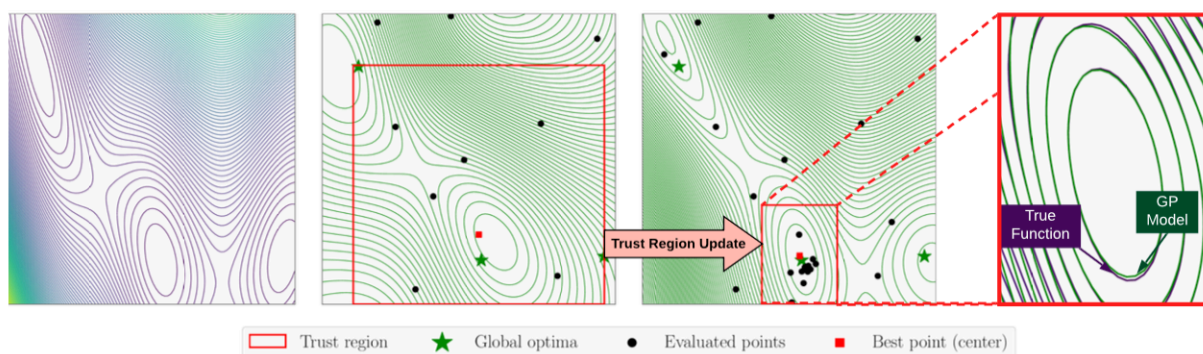
2.5.2 TuRBO

TuRBO, teda Trust Region Bayesian Optimization, je algoritmus lokálnej Bayesovskej optimalizácie určený na riešenie nákladných black-box optimalizačných úloh. Bol navrhnutý ako reakcia na obmedzenia klasickej Bayesovskej optimalizácie, ktorá pri väčších alebo vyššie-dimenzionálnych úlohách môže narážať na problém neefektívneho globálneho modelovania priestoru riešení. TuRBO preto nepoužíva jeden globálny model pre celý optimalizačný priestor, ale koncentruje modelovanie a výber ďalších bodov do lokálnych oblastí dôvery. [25]

Klasická Bayesovská optimalizácia často pracuje s jedným pravdepodobnostným náhradným modelom, ktorý má opisovať správanie cieľovej funkcie v celom priestore riešení. Pri vyššej dimenzii alebo pri veľkom priestore však môže byť takýto model menej presný a výber ďalšieho bodu môže viesť k nadmernému preskúmvaniu oblastí, ktoré nie sú z hľadiska zlepšenia riešenia dostatočne perspektívne. TuRBO tento problém rieši tak, že optimalizácia je

obmedzená na lokálne oblasti, v ktorých je možné cieľovú funkciu modelovať presnejšie a efektívnejšie. [25]

Podstatným prvkom algoritmu je oblasť dôvery. Tá predstavuje ohraničenú lokálnu časť optimalizačného priestoru, v ktorej sa predpokladá, že náhradný model dokáže poskytovať dostatočne užitočné predikcie. Namiesto toho, aby bol ďalší bod vyberaný z celého priestoru riešení, sú kandidátne body generované iba v rámci tejto oblasti. Tým sa vyhľadávanie sústreďuje na okolie sľubného riešenia a zároveň sa znižuje riziko, že algoritmus bude neefektívne preskúmať príliš vzdialené časti priestoru. [25]



Obr. 5: Princíp algoritmu TuRBO. Oblasť dôvery sa počas optimalizácie posúva smerom k perspektívnej oblasti a lokálny GP model aproximuje cieľovú funkciu iba v jej okolí. Prevzaté z [16].

V rámci oblasti dôvery je používaný Gaussovský proces ako lokálny náhradný model. Tento model je zostavovaný na základe vyhodnotených bodov relevantných pre danú oblasť a slúži na odhad hodnoty cieľovej funkcie aj neistoty predikcie. Dôležité je, že Gaussovský proces v TuRBO nie je chápaný ako jediný globálny model celej funkcie, ale ako lokálny model, ktorého platnosť a praktická využiteľnosť sú viazané na konkrétnu oblasť dôvery. [16]

Veľkosť oblasti dôvery sa počas optimalizácie adaptívne mení podľa úspechu alebo neúspechu predchádzajúcich evaluácií. Ak sa v danej oblasti opakovane darí nachádzať zlepšenia, oblasť môže byť zväčšená, aby bolo umožnené širšie preskúmanie sľubného okolia. Ak sa naopak zlepšenie nedarí dosiahnuť, oblasť sa zmenšuje a výber ďalších bodov sa sústreďuje na užšiu časť priestoru riešení. [25]

Výber ďalších kandidátnych bodov je v počiatočnom návrhu TuRBO riadený Thompsonovým vzorkovaním. Na základe lokálneho pravdepodobnostného modelu sú v oblasti dôvery generované kandidátne body. Z nich sa následne vyberie bod alebo skupina bodov, ktoré budú vyhodnotené pôvodnou cieľovou funkciou. Keďže Thompsonovo vzorkovanie pracuje so vzorkami z pravdepodobnostného modelu, môže vybrať nielen body s priaznivou predikovanou hodnotou, ale aj body z oblastí, v ktorých je model neistý. Tým sa pri výbere kandidátov kombinuje zlepšovanie s preskúmaním priestoru riešení. [25]

Počiatočný algoritmus TuRBO je možné chápať aj ako systém viacerých lokálnych modelov. Pri variante s viacerými oblasťami dôvery sú jednotlivé oblasti spravované samostatne a výpočtový rozpočet je pridelený medzi ne na základe ich úspešnosti. Tým sa kombinuje lokálne modelovanie s globálnejším preskúmaním priestoru, pretože algoritmus

nemusí byť viazaný iba na jednu oblasť. Ak sa niektorá oblasť ukáže ako neperspektívna, jej význam sa môže znížiť alebo môže byť nahradená inou oblasťou. [25]

TuRBO možno zaradiť medzi prístupy lokálnej Bayesovskej optimalizácie, ktoré sa nespoliehajú na jeden globálny náhradný model pre celý priestor riešení. Jeho hlavnou charakteristikou je spojenie lokálneho Gaussovského procesu, adaptívne riadenej oblasti dôvery a modelovo riadeného výberu kandidátnych bodov. Tento prístup umožňuje sústrediť vyhľadávanie do sľubných oblastí priestoru riešení a zároveň priebežne upravovať rozsah lokálneho hľadania podľa úspešnosti predchádzajúcich evaluácií. [25]

2.5.3 PySOT

PySOT, teda Python Surrogate Optimization Toolbox, je programovacia nástrojová sada určená na globálnu optimalizáciu výpočtovo náročných black-box funkcií. Na rozdiel od algoritmov PyBADS a TuRBO nepredstavuje jeden pevne definovaný optimalizačný algoritmus, ale rámec, v ktorom je možné kombinovať rôzne náhradné modeli, počiatočné návrhy bodov a stratégie výberu ďalších evaluácií. [26]

Základ optimalizácie založenej na náhradnom modeli v PySOT je tvorený štyrmi komponentmi: optimalizačným problémom, počiatočným návrhom bodov, náhradným modelom a stratégiou výberu nových bodov. Optimalizačný problém obsahuje informácie o dimenzii, type premenných, prípustných hraniciach a cieľovej funkcii. Počiatočný návrh bodov slúži na získanie prvých evaluácií potrebných na zostavenie náhradného modelu. Náhradný model následne aproximuje cieľovú funkciu na základe aktuálne vyhodnotených bodov a stratégia určuje, ktoré ďalšie body majú byť vyhodnotené. [26]

Proces optimalizácie v PySOT zodpovedá sekvenčnému rámcu optimalizácie s náhradným modelom. Na počiatku sú vygenerované a vyhodnotené počiatočné body, následne je z dostupných dát zostavený náhradný model a potom sa v cykle opakuje výber nových bodov, ich vyhodnotenie a aktualizácia modelu. Ukončenie optimalizácie je zvyčajne viazané na stanovený rozpočet evaluácií, ale môže byť viazané aj na iné zastavovacie kritérium.

Z pohľadu náhradných modelov PySOT podporuje viacero typov aproximačných prístupov. Konkrétne sa jedná o náhradné modeli založené na radiálnych bázičných funkciách, Gaussovské procesy a polynomiálne modeli. [26]

Dôležitou súčasťou PySOT sú stratégie výberu ďalších bodov. Nástroj podporuje viacero stratégií, medzi ktoré patria napríklad SRBF, DYCORDS, SOP, EI a LCB. Tieto stratégie sa líšia tým, ako využívajú náhradný model a akým spôsobom navrhujú ďalšie kandidátne body pre vyhodnotenie cieľovou funkciou. Stratégie ako SRBF, DYCORDS a SOP pracujú s množinou vygenerovaných kandidátnych bodov. Následne sa vyberajú ďalšie body na vyhodnotenie podľa predikovanej hodnoty cieľovej funkcie pomocou náhradného modelu, vzdialenosti od aktuálne vyhodnotených bodov alebo spôsobu generovania kandidátov. Naopak stratégie EI a LCB vychádzajú z akvizičných funkcií používaných v Bayesovskej optimalizácii. [26]

Pri použití PySOT je dôležité, že nie všetky stratégie sú vhodné pre každý náhradný model. Stratégia Expected Improvement môže byť použitá iba v kombinácii s modelom GPRegressor, pretože pre svoj výpočet potrebuje predikciu neistoty. Tento princíp je dôležitý aj z hľadiska experimentálneho návrhu, pretože PySOT síce umožňuje kombinovať modeli a

stratégie, ale zmysluplné sú iba také kombinácie, pri ktorých stratégia dokáže využiť informáciu poskytovanú daným typom modelu. [26]

PySOT podporuje rôzne režimy vykonávania optimalizácie. Stratégie môžu byť spúšťané sériovo, synchrónne dávkovo alebo asynchrónne paralelne. Táto vlastnosť je pri nákladných cieľových funkciách významná, najmä pri dostupných výpočtových zdrojoch, kedy môže byť užitočné vyhodnocovať viac kandidátnych bodov súčasne. [26]

2.5.4 Porovnanie vybraných algoritmov

PyBADs, TuRBO a PySOT patria medzi prístupy optimalizácie s využitím náhradných modelov, no líšia sa tým, ako je náhradný model zapojený do výberu ďalších kandidátnych bodov. Všetky tri prístupy využívajú informáciu z predchádzajúcich evaluácií, ale odlišujú sa mierou lokálnosti, spôsobom riadenia vyhľadávania a flexibilitou konfigurácie. [22], [25], [26]

PyBADs predstavuje lokálne orientovaný optimalizátor, ktorý kombinuje priame vyhľadávanie s podporou lokálneho pravdepodobnostného modelu. Jeho základom je striedanie krokov poll a search. Krok poll prehľadáva okolie aktuálneho riešenia pomocou lokálnej mriežky, zatiaľ čo krok search využíva lokálny Gaussovský proces na návrh sľubných kandidátnych bodov. Náhradný model tu neslúži na opis celého priestoru riešení, ale na podporu rozhodovania v okolí aktuálneho riešenia. [22], [23], [24]

TuRBO je tiež lokálne orientovaný prístup, jeho lokálnosť je však riadená explicitne pomocou oblasti dôvery. Namiesto priameho lokálneho vyhľadávania používa lokálnu Bayesovskú optimalizáciu v ohraničenej časti priestoru riešení. Veľkosť tejto oblasti sa počas optimalizácie mení podľa toho, či sa v nej darí nachádzať zlepšenia. Oproti PyBADs teda TuRBO neriadi lokálne hľadanie pomocou mriežkového kroku poll, ale pomocou adaptívne upravovanej oblasti dôvery. [25]

PySOT sa od PyBADs a TuRBO odlišuje tým, že nepredstavuje jeden pevne definovaný algoritmus. Ide o rámec, v ktorom sa samostatne volí optimalizačný problém, počiatočný návrh bodov, náhradný model a stratégia výberu ďalších bodov. Vďaka tomu umožňuje vytvárať viacero kombinácií modelov a stratégií, napríklad spojenie Gaussovského procesu s EI alebo LCB, prípadne použitie RBF modelov so stratégiami SRBF, DYCORS alebo SOP. [26]

Z pohľadu výberu ďalšieho bodu možno rozdiel medzi prístupmi zhrnúť nasledovne. PyBADs vyberá nové body pomocou kombinácie lokálneho priameho vyhľadávania a modelovo riadenej search fázy. TuRBO vyberá nové body v rámci adaptívnej oblasti dôvery pomocou lokálneho pravdepodobnostného modelu. PySOT ponecháva mechanizmus výberu na zvolenej stratégii, a preto jeho správanie závisí od konkrétnej kombinácie náhradného modelu a výberového pravidla. [22], [25], [26]

Z hľadiska teoretického členenia teda PyBADs reprezentuje lokálne modelovo podporené vyhľadávanie, TuRBO lokálnu Bayesovskú optimalizáciu s oblasťou dôvery a PySOT konfigurovateľný rámec pre kombinovanie náhradných modelov so stratégiami výberu kandidátnych bodov. Toto rozlíšenie je dôležité preto, že jednotlivé prístupy nevyužívajú náhradný model rovnakým spôsobom, hoci všetky patria do širšej skupiny optimalizácie s využitím náhradných modelov. [22], [23], [24], [25]

2.6 Klimaticky orientované aplikačné úlohy

Optimalizácia s využitím náhradných modelov nachádza uplatnenie všade tam, kde je hodnotenie cieľovej funkcie výpočtovo, časovo alebo experimentálne náročné. V problémoch súvisiacich s klimatickou zmenou ide o problémy, pri ktorých je potrebné opakovane vyhodnocovať simulácie fyzikálnych procesov alebo rozhodovať o priestorovom usporiadaní technických prvkov v danom prostredí. Práve v takýchto situáciách umožňujú náhradné modeli aproximovať správanie kritériálnej funkcie na základe obmedzeného počtu vzoriek a následne efektívnejšie riadiť samotný optimalizačný proces. [5]

Konkrétne prípady klimatických úloh, na ktoré sa táto práca zameriava sú optimalizácia rozmiestnenia veterných turbín, kde sa hľadá priestorové usporiadanie turbín, ktoré maximalizuje energetický výnos veterného parku pri rešpektovaní fyzikálnych a priestorových obmedzení a optimalizácia rozmiestnenia senzorov za účelom enviromentálneho monitorovania, kde je cieľom zvoliť také umiestnenie meracích zariadení, aby pri obmedzenom rozpočte prinášali čo najväčšiu informačnú hodnotu o sledovanom jave. Obe úlohy zodpovedajú charakteru nákladných black-box optimalizácií. [5], [8]

2.6.1 Optimalizácia rozmiestnenia veterných turbín

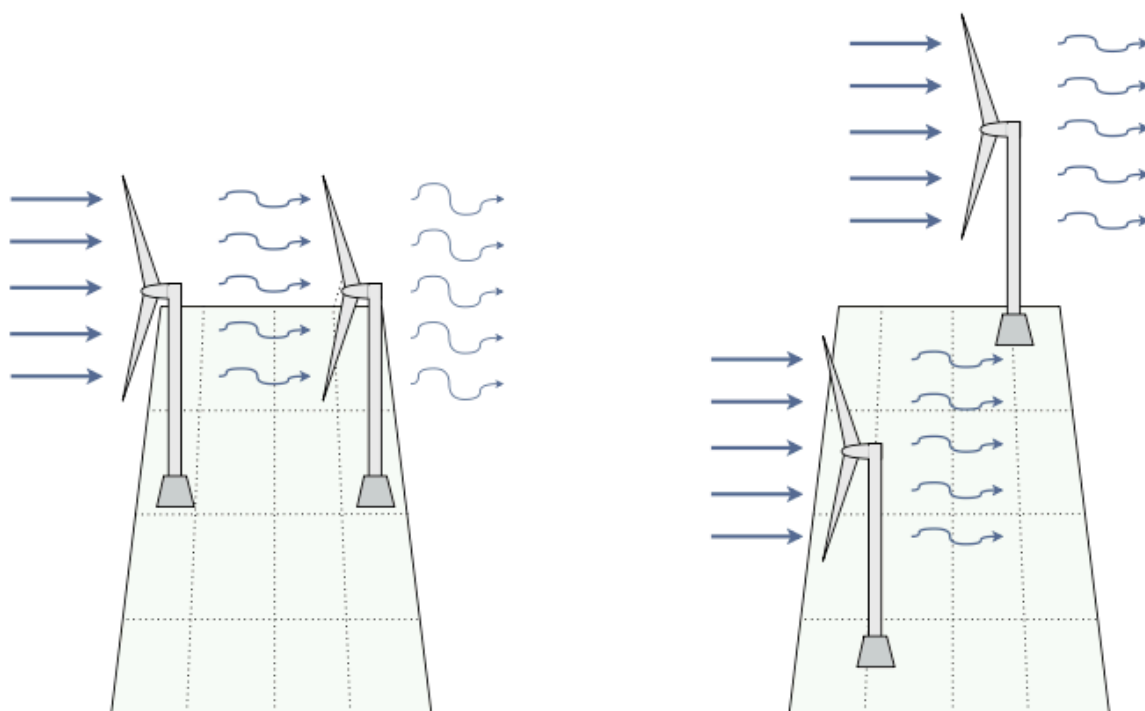
Prvý problém aplikácie optimalizačných metód v oblasti klimatických zmien je návrh rozmiestnenia veterných turbín vo veternom parku. Pred samotnou výstavbou je potrebné určiť pozície jednotlivých turbín tak, aby bola čo najlepšie využitá dostupná energia vetra t.j. aby sa maximalizovala celková produkcia elektrickej energie alebo minimalizovala energetické straty spôsobené vzájomným ovplyvňovaním turbín. Vhodnejším rozložením turbín možno zvýšiť výrobu elektrickej energie bez potreby budovania ďalších zariadení, a tým podporiť efektívnejšie využívanie obnoviteľných zdrojov energie. [5]

Hlavnou komplikáciou tohto problému je tzv. wind wake efekt. Pri prechode vzduchu cez rotor turbíny dochádza k odberu kinetickej energie z prúdenia, čím sa za turbínou vytvára oblasť spomaleného a turbulentnejšieho prúdenia. Ak sa ďalšia turbína nachádza v tejto oblasti, jej výkon môže klesať a zároveň môže byť vystavená vyššiemu dynamickému zaťaženiu. Výsledná kvalita rozmiestnenia preto nezávisí iba od polohy jednotlivých turbín, ale najmä od ich vzájomnej konfigurácie, smeru a rýchlosti vetra, vzdialeností medzi turbínami a charakteru lokality. [5]

Matematicky možno problém formulovať ako maximalizáciu výkonu veterného parku a optimalizáciu kritériálnej funkcie závislej od pozícií všetkých turbín.

$$\max_{x_1, x_2, \dots, x_N} P(x_1, x_2, \dots, x_N)$$

kde x_i predstavuje polohu i -tej turbíny a P označuje celkový výkon veterného parku. Do formulácie zároveň vstupujú obmedzenia určujúce prípustnú oblasť rozmiestnenia a minimálne vzdialenosti medzi turbínami. Nejde teda iba o maximalizáciu výkonu, ale aj o hľadanie kompromisu medzi energetickým ziskom a rešpektovaním fyzikálnych a prevádzkových obmedzení [5], [8]



Obr. 6: Ilustrácia vplyvu wind wake efektu na rozmiestnenie veterných turbín. Turbína umiestnená po vetre za inou turbínou pracuje so slabším a turbulentnejším prúdením. Prevzaté z [5].

Vyhodnotenie jednej konfigurácie turbín sa spravidla odvíja od numerického modelu alebo simulátoru wake efektov. Takéto hodnotenie je výpočtovo náročné, pretože pre každé kandidátne rozmiestnenie je potrebné prepočítať kolektívne správanie celého veterného parku. Práve preto je táto úloha vhodná pre optimalizáciu s použitím náhradných modelov. Namiesto priameho vyhodnocovania veľkého počtu návrhov sa vytvorí aproximácia závislosti medzi rozmiestnením turbín a výsledným výkonom, ktorá následne slúži na výber sľubných kandidátnych riešení. Tým sa významne znižuje počet potrebných simulácií pri zachovaní schopnosti nájsť kvalitné rozloženie. [5]

2.6.2 Optimalizácia rozmiestnenia senzorov

Druhou významnou oblasťou aplikácie náhradných modelov v klimaticky orientovaných úlohách je optimalizácia rozmiestnenia senzorov pre environmentálne monitorovanie. V tejto ide primárne o výber takých meracích lokalít, ktoré umožnia čo najlepšie pozorovať priestorové rozloženie sledovaných veličín. Typickými príkladmi sú monitorovanie kvality ovzdušia, sledovanie environmentálnych parametrov v mestskom prostredí alebo identifikácia lokalít s najvyššími hodnotami znečistenia. [5]

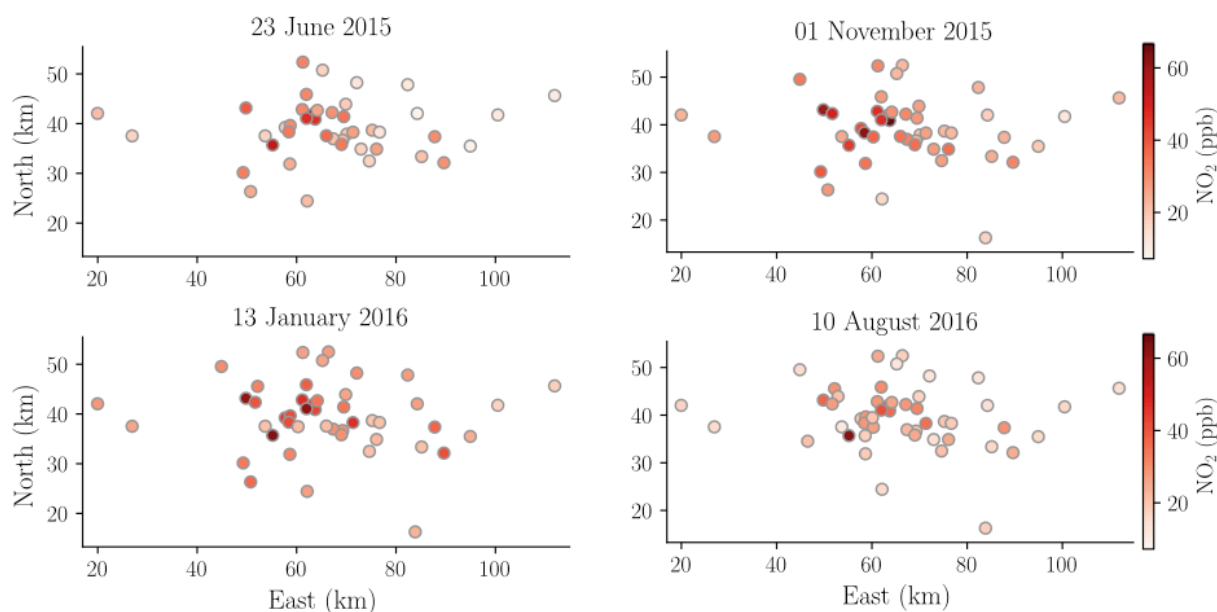
Podstata problému spočíva v tom, že počet dostupných senzorov býva obmedzený rozpočtom, technickými možnosťami alebo prevádzkovými nákladmi. Úlohou preto nie je vykonávať merania v celej oblasti, ale zvoliť také lokality, z ktorých bude možné čo najlepšie odhadnúť správanie sledovaného javu aj v nepozorovaných miestach, alebo čo najefektívnejšie lokalizovať extrémne hodnoty sledovaného javu. Nejedná sa len o technický problém

rozmiestnenia zariadení, ale o rozhodovanie pod neistotou, kde je potrebné priebežne vyvažovať preskúvanie nových lokalít a využívanie už známych sľubných oblastí. [5]

Matematicky možno túto úlohu formulovať ako výber množiny S obsahujúcu k počet senzorových pozícií z kandidátnej oblasti Ω .

$$\max_{S \subseteq \Omega, |S|=k} J(S)$$

kde $J(S)$ predstavuje kritérium kvality zvoleného rozmiestnenia. Tým môže byť napríklad informačný zisk, presnosť rekonštrukcie priestorovej oblasti alebo schopnosť identifikovať miesta s vysokou koncentráciou sledovanej veličiny. [5]



Obr. 7: Príklady problémových inštancií LAQN-BO pre rôzne dni. Body predstavujú dostupné meracie lokality siete London Air Quality Network a farebná škála vyjadruje koncentráciu NO_2 v jednotkách ppb. Obrázok zobrazuje nerovnomerné priestorové rozmiestnenie staníc, zhľukovanie senzorov v centrálnej časti Londýna, rozdielnu dostupnosť meracích bodov medzi dňami a zmenu polohy oblastí s najvyššou koncentráciou znečistenia. Prevzaté z [5].

V tejto oblasti sú náhradné modeli vhodnou voľbou, pretože enviromentálne javy majú priestorový charakter a sú zaťažené neistotou. Pravdepodobnostný náhradný model dokáže odhadnúť očakávanú hodnotu sledovanej veličiny v nepozorovaných bodoch a kvantifikovať mieru neistoty tohto odhadu. Práve miera neistota rozhoduje o tom, kam umiestniť ďalší senzor alebo ktoré lokality sú z hľadiska zberu dát najinformatívnejšie. [5]

3 VLASTNÉ RIEŠENIE

V predchádzajúcej kapitole boli predstavené teoretické východiská optimalizácie s využitím náhradných modelov, vrátane vybraných typov náhradných modelov, akvizičných stratégií a algoritmov použitých v tejto práci. V tejto sekcii je pozornosť sústredená na ich konkrétne nasadenie v experimentálnej časti práce. Cieľom sekcie je oddôvodniť výber aplikačných úloh, formuláciu riešených optimalizačných problémov, použité testovacie prostredia, implementačné nástroje, experimentálny protokol a hodnotiace metriky.

V úvode sekcie budú, ako prvé, zdôvodnené kritériá výberu aplikačných úloh a budú sformulované konkrétne riešené problémy rozmiestnenia environmentálnych senzorov a rozmiestnenia veterných turbín. Následne budú opísané testovacie prostredia, použité programovacie knižnice a výpočtové prostredie, v ktorom boli experimenty realizované. Ďalej bude uvedené, akým spôsobom boli jednotlivé algoritmy zaradené do experimentálneho porovnania, aké parametre boli v experimentoch sledované a aké metriky boli použité na ich vyhodnotenie pri porovnávaní.

Záverečná časť kapitoly je venovaná implementačnému rámcu a spôsobu zaznamenávania výsledkov.

3.1 Kritériá výberu aplikačných úloh

Pri výbere aplikačných úloh bolo žiadané, aby boli zvolené problémy, ktoré sú tematicky previazané s oblasťou klimatickej zmeny a zároveň sú vhodné na skúmanie charakteristík optimalizačných algoritmov založených na náhradných modelov. Tým pádom bolo požadované, aby išlo o problémy, pri ktorých je vyhodnotenie cieľovej funkcie dostatočne nákladné a použitie optimalizačných algoritmov s náhradným modelom tak bolo odôvodnené.

Ďalším kritériom bolo, aby sa zvolené problémy líšili charakterom optimalizačného priestoru. Z tohto dôvodu bol uprednostnený výber diskkrétnej a spojitej úlohy. Tým bolo umožnené sledovanie správania vybraných algoritmov v dvoch rozdielnych podmienkach a zároveň malo byť overené, do akej miery závisí prínos náhradného modelu od povahy riešeného problému.

Za vhodnú diskkrétne úlohu bolo zvolené rozmiestnenie environmentálnych senzorov. V tomto prípade sa experimentovalo s konečnou množinou kandidátnych lokalít, medzi ktorými bolo vyhľadávané riešenie s najvyššou hodnotou sledovaného kritéria. Táto úloha bola zvolená preto, že reprezentuje praktický problém environmentálneho monitorovania a zároveň umožňuje analyzovať správanie algoritmov s náhradným modelom v priestore s obmedzeným počtom prípustných riešení.

Pre reprezentáciu spojitej úlohy bolo zvolené rozmiestnenie veterných turbín. V tejto úlohe bolo optimalizované priestorové usporiadanie turbín s cieľom maximalizácie výkonu celej farmy. Zvolená úloha bola považovaná za vhodnú najmä preto, že ide o simulačne náročný problém, pri ktorom narastá zložitosť s počtom turbín a pri ktorom môže byť využitie rozpočtu evaluácií rozhodujúce.

Výber zvolených úloh bol preto uskutočnený so zámerom pokryť dve rozdielne triedy klimaticky orientovaných optimalizačných problémov. Súčasne malo byť umožnené nielen porovnanie finálnej kvality dosiahnutých riešení, ale aj sledovanie rozdielov v konvergencii, časovej náročnosti a celkovej vhodnosti vybraných algoritmov pre diskkrétne a spojité úlohy.

3.2 Formulácia riešených optimalizačných úloh

3.2.1 Rozmiestnenie environmentálnych senzorov

Prvá riešená aplikačná úloha, rozmiestnenie environmentálnych senzorov, nadväzuje na dáta zo siete London Air Quality Network [5]. Využitá bola pripravená formulácia úlohy z testovacieho prostredia, v ktorej vstupné dáta tvorí množina kandidátnych lokalít s priradenými hodnotami cieľovej funkcie. Problém bol zvolený ako reprezentant diskrétného priestoru, pri ktorom sa optimálne riešenie vyhľadáva v konečnej množine prípustných bodov.

Vstupné dáta pre túto úlohu tvorí množina kandidátnych lokalít s prislúchajúcimi hodnotami, reprezentujúce kvalitu danej lokality z hľadiska sledovaného kritéria. V riešenej formulácii testovacieho prostredia nie je priestor riešení chápaný ako spojitá oblasť, ale ako vopred určená diskrétna množina bodov. Rozhodovanie je preto vykonávané nad konečným súborom lokalít, pričom v každom kroku optimalizácie je vyberaná ďalšia lokalita určená na vyhodnotenie.

Rozhodovacou premennou je výber konkrétnej lokality z kandidátnej množiny. Cieľová funkcia je definovaná kritériálnou hodnotou, priradenou danej lokalite. Cieľom optimalizácie je nájdenie bodu s maximálnou hodnotou, ktorý predstavuje najrelevantnejšiu lokalitu pre enviromentálne pozorovanie.

Diskrétny charakter úlohy umožňuje presne určiť množinu prípustných kandidátnych lokalít aj najlepšiu známu hodnotu cieľovej funkcie v danej problémovej inštancii. Vďaka tomu možno pri experimentoch hodnotiť nielen najlepšiu dosiahnutú hodnotu, ale aj odchýlku od známeho optima a úspešnosť jeho nájdenia. Táto vlastnosť robí úlohu vhodnou na porovnanie algoritmov pri obmedzenom rozpočte evaluácií.

Zvolená úloha tak poskytuje vhodný priestor na porovnanie základného prístupu a algoritmov založených na náhradnom modeli v diskrétnom prostredí. Súčasne umožňuje sledovať, ako sa pri takomto type problému prejavuje vzťah medzi veľkosťou rozpočtu evaluácií, charakterom priestoru riešení a prínosom optimalizácie s náhradným modelom.

3.2.2 Rozmiestnenie veterných turbín

Ako druhá aplikačná úloha bolo zvolené rozmiestnenie veterných turbín. Táto úloha bola prevzatá z benchmarkového prostredia [5] pre nákladnú optimalizáciu a bola riešená ako spojitý optimalizačný problém. Na rozdiel od diskrétnej úlohy rozmiestnenia enviromentálnych senzorov nebolo riešenie vyhľadávané v konečnej množine kandidátnych bodov, ale v spojitom priestore, v ktorom boli optimalizované súradnice jednotlivých turbín.

Vstupné dáta pre úlohu rozmiestnenia veterných turbín predstavovali počet turbín, prípustnú oblasť ich rozmiestnenia, veterné podmienky a simulačné parametre potrebné na

výpočet výkonu. Rozhodovacie premenné tvorili polohy turbín v riešenej oblasti. Pre každú turbínu boli optimalizované dve priestorové súradnice, preto dimenzia úlohy rástla s počtom turbín. V experimentoch boli z tohto dôvodu použité viaceré konfigurácie s rôznym počtom turbín, aby mohlo byť sledované správanie algoritmov pri rastúcej zložitosti problému.

Cieľová funkcia bola viazaná na simulovaný výkon veterného parku. Cieľom optimalizácie bolo nájsť také rozmiestnenie veterných turbín, pri ktorom by bol dosiahnutý maximálny kumulatívny výnos elektrickej energie celej veternej farmy. Úloha bola z aplikačného hľadiska chápaná ako maximalizačná, pretože cieľom bolo maximalizovať výkon veterného parku. Algoritmy PyBADs a TuRBO však pracujú s minimalizačnou optimalizačnou formuláciou. V týchto prípadoch bola preto cieľová funkcia prevedená na minimalizáciu zápornej hodnoty výkonu. Pri interpretácii výsledkov bol výkon veterného parku opäť hodnotený ako maximalizačné kritérium.

Zvolená úloha zároveň umožnila sledovať vplyv rastúcej dimenzie problému na správanie algoritmov. Vďaka použitiu viacerých konfigurácií s rôznym počtom turbín mohlo byť posúdené, ako sa pri jednotlivých algoritmoch mení konvergencia, časová náročnosť a celková vhodnosť pre simulačne náročný spojitý optimalizačný problém.

3.3 Testovacie prostredia klimaticky orientovaných úloh

Pri realizácii experimentov nebolo postupované formou úplnej implementácie všetkých úloh a algoritmov od začiatku. Vychádzalo sa z existujúcich testovacích prostredí a dostupných softvérových knižníc. Tie boli použité ako základ, nad ktorým bol následne budovaný vlastný experimentálny rámec práce. Tento prístup umožnil pracovať s úlohami v štandardizovanej podobe a nadviazať na existujúce vedecké a výskumné nástroje. Vlastná časť práce sa preto mohla sústrediť najmä na prepojenie týchto prostredí s vybranými optimalizačnými algoritmami, ich experimentálne nasadenie a následné porovnanie výsledkov.

3.3.1 LAQN

Pri úlohe rozmiestnenia environmentálnych senzorov bolo využité testovacie prostredie LAQN-BO, ktoré bolo vytvorené na generovanie Bayesovských optimalizačných úloh z dát siete London Air Quality Network. Toto prostredie poskytuje reálne environmentálne merania a štandardizované problémové inštancie vhodné na experimentálne porovnanie optimalizačných algoritmov riadených náhradným modelom. Vygenerované úlohy sú formulované ako diskkrétne optimalizačné problémy, v ktorých sú jednotlivým kandidátnym lokalitám priradené hodnoty cieľovej funkcie a cieľom je nájdenie bodu s maximálnou hodnotou pri čo najmenšom počte evaluácií. Testovacie prostredie LAQN-BO bolo využité ako zdroj pripravených problémov, nad ktorými mohli byť následne spúšťané porovnávané algoritmy.

Súčasťou repozitára je séria skriptov, prostredníctvom ktorým sú z pôvodných dát LAQN postupne vytvárané problémové inštancie. Počiatočný krok spočíva v stiahnutí dostupných meraní. Následne je vykonávané odfiltrovanie nepoužívaných lokalít. Ďalšími krokmi je doplnenie informácií o lokalitách a ich geografickej polohe, klasifikácia lokalít podľa konkrétneho umiestnenia environmentálnych senzorov a zoradenie dát. Po vykonaní týchto

krokov sú generované samotné optimalizačné úlohy pre zvolený rok. Z pôvodných meraní siete LAQN je vytvorená štandardizovaná sada problémov pripravených na ďalšie experimentálne spracovanie.

Z metodického hľadiska je dôležité, že LAQN-BO neposkytuje iba surové dáta, ale aj jednotnú reprezentáciu problémovej inštancie vo forme Python objektu. Tento objekt obsahuje počiatkové pozorované body, ich hodnoty, celú kandidátnu doménu, pravé hodnoty cieľovej funkcie v jednotlivých kandidátskych bodoch, informáciu o globálnom maxime a jeho polohe, ako aj identifikátor konkrétnej úlohy. Konkrétne sú v ňom uchovávané položky ako `xx` a `yy` pre počiatkové pozorované body a ich hodnoty, `domain` a `labels` pre celú množinu kandidátnych bodov a ich hodnoty, `maximum` a `maximiser` pre najlepšiu známu hodnotu a zodpovedajúci bod, ako aj ďalšie pomocné údaje potrebné na prácu s danou úlohou. Vďaka tejto štruktúre bolo možné optimalizačné algoritmy testovať na pripravených problémových inštanciách jednotným spôsobom bez potreby vlastného budovania alebo získavania vstupných dát od začiatku.

Za dôležitú vlastnosť testovacieho prostredia bolo považované aj to, že cieľ riešenia je v dokumentácii explicitne definovaný ako nájdenie optimálnej lokality s čo najmenším počtom vzoriek. Súčasne je uvedené, že každá problémová inštancia začína s piatimi počiatkové pozorovanými bodmi. Takáto formulácia je z pohľadu tejto práce vhodná, pretože zodpovedá princípu optimalizácie založenej na náhradnom modeli, pri ktorej sa z obmedzeného počtu počiatkových pozorovaní postupne buduje model a následne sa volia ďalšie evaluácie tak, aby bolo optimum dosiahnuté čo najefektívnejšie. Testovacie prostredie preto nebolo využité len ako zdroj dát, ale aj ako už pripravená experimentálna formulácia problému, ktorá prirodzene podporuje porovnávanie algoritmov z pohľadu kvality riešenia aj efektívnosti využitia rozpočtu evaluácií.

V rámci tejto práce bolo testovacie prostredie LAQN-BO využité ako zdroj spracovaných vstupných dát pre optimalizáciu výberu lokality v diskretnom prostredí s maximálnou hodnotou merania sledovaného kritéria NO_2 . Z testovacieho prostredia boli prevzaté problémové inštancie, kandidátne body, im prislúchajúce hodnoty a referenčné informácie o maximálnych kritériálnych hodnotách, ktoré enviromentálne senzory snímajú. Nad touto vrstvou boli následne nasadené porovnávané optimalizačné algoritmy a bol vybudovaný vlastný experimentálny rámec určený na ich spúšťanie, vyhodnocovanie a porovnanie.

3.3.2 ExpensiveOptimBenchmark

Pri úlohe rozmiestnenia veterných turbín bolo využité testovacie prostredie ExpensiveOptimBenchmark, ktoré slúži ako rámec pre formuláciu a testovanie nákladných optimalizačných úloh. Úlohu Veterných turbín v tomto testovacom prostredí predstavuje optimalizáciu rozmiestnenia veterných turbín s cieľom maximalizácie výkonu veterného parku.

Samotná úloha Veterných turbín je v testovacom prostredí implementovaná ako optimalizačný problém, v ktorom rozhodovacie premenné predstavujú priestorové súradnice veterných turbín. Z pohľadu optimalizácie ide o spojitý problém, pretože polohy turbín nie sú vybrané z konečnej množiny kandidátnych bodov, ale sú hľadané v spojitom priestore. Dimenzia úlohy pritom rastie s počtom turbín, keďže každej turbíne zodpovedajú dve

rozhodovacie premenné. Táto úloha je reprezentovaná triedou, ktorá zapuzdruje rozmery problému, hranice prípustného priestoru a samotné vyhodnotenie cieľovej funkcie.

Na samotnú evaluáciu kvality kandidátneho riešenia bol použitý nástroj FLORIS, ktorý v tejto práci nevystupuje ako vrstva testovacieho prostredia, ale ako simulačný model veterného poľa a interakcií medzi veternými turbínami. FLORIS vykonával výpočet výkonu veterného parku pre konkrétne rozmiestnenie turbín. Vstupné parametre simulácie sú definované v konfiguračných súboroch, v ktorých sú obsiahnuté informácie o rozložení turbín, veterných podmienkach a vlastnostiach modelovaného prostredia. Vstupné dáta, použité pri simulácii úlohy rozmiestnenia veterných turbín, pracujú s parametrami ako súradnice rozmiestnenie turbín, rýchlosť a smer vetra, a hustota vzduchu. Tieto parametre určujú fyzikálny a simulačný kontext, v ktorom je kandidátne riešenie vyhodnocované.

V rámci tejto práce tak ExpensiveOptimBenchmark poskytol normalizovanú definíciu úlohy rozmiestnenia veterných turbín, zatiaľ čo FLORIS určoval výsledný výkon veterného parku. Nad týmito vrstvami boli následne nasadené porovnávané optimalizačné algoritmy a bol vytvorený vlastný experimentálny rámec, prostredníctvom ktorého mohli byť jednotlivé metódy spúšťané, vyhodnocované a vzájomne porovnávané.

3.4 Porovnávané algoritmy a ich použitie v experimentoch

3.4.1 Architektúra implementačného rámca

Pri praktickom nasadení optimalizačných algoritmov bolo potrebné vytvoriť jednotnú implementačnú vrstvu, ktorá umožnila spúšťať všetky vybrané algoritmy založené na náhradnom modeli porovnateľným spôsobom na oboch aplikačných úlohách orientovaných na klimatickú zmenu. Použité algoritmy pochádzali z rôznych knižníc a repozitárov, a preto neposkytovali rovnaké rozhranie, rovnaký formát výstupov ani rovnaký spôsob evidencie priebehu optimalizácie. Z tohto dôvodu boli vytvorené vlastné skripty, ktoré zabezpečili prepojenie medzi testovacím prostredím, optimalizačným algoritmom a následným vyhodnotením výsledkov.

Prvú vrstvu implementácie tvorili optimalizačné skripty. Ich úlohou bolo vykonať jedno spustenie konkrétneho algoritmu na jednej problémovej inštancii alebo jednej konfigurácii úlohy. V rámci týchto skriptov bolo zabezpečené načítanie alebo vytvorenie problému, a načítanie vstupných dát pre optimalizáciu. Výstupy skriptov boli zjednotené do spoločnej štruktúry tak, aby bolo možné výsledky rôznych algoritmov ďalej spracovať rovnakým spôsobom. Pri jednom behu boli ukladané navštívené a preskúmané body priestoru riešenia, hodnoty cieľovej funkcie, priebeh najlepšej doteraz nájdennej hodnoty, finálne najlepšie riešenie, počet volaní cieľovej funkcie, počet evaluácií potrebných na dosiahnutie najlepšieho riešenia a čas behu. Pripravené výstupy tvorili spoločný základ pre skripty hromadného spustenia a pre následný výpočet hodnotiacich metrík.

Druhú vrstvu implementácie tvorili skripty hromadného spustenia. Tieto skripty nastavovali experimentálne parametre, ako sú rozpočet evaluácií, počet behov a pri úlohe veterných turbín aj počet turbín. Ich úlohou bolo opakovane spúšťať príslušný optimizer skript,

zhromažďovať výsledky jednotlivých behov a ukladať detailné výstupy pre finálne spracovanie.

Na úrovni skriptov hromadného spustenia boli zároveň počítané súhrnné štatistiky. Z jednotlivých behov boli odvodené priemerné a mediánové hodnoty, a smerodajné odchylky vybraných metrík, ako aj priemerné konvergenčné krivky. Pri úlohe rozmiestnenia environmentálnych senzorov boli navyše agregované metriky vychádzajúce zo známeho optima, najmä úspešnosť a odchýlka od optima. Pri úlohe rozmiestnenia veterných turbín boli výsledky agregované samostatne pre jednotlivé konfigurácie počtu turbín.

Výsledky batch vrstvy boli ukladané vo forme súhrnných výstupov vo formátoch JSON a CSV. JSON súbory slúžili na uloženie detailných aj agregovaných metrík, zatiaľ čo CSV výstupy boli využívané najmä na ďalšie spracovanie, tvorbu grafov a prípadný export do nástrojov určených na analýzu optimalizačných priebehov. Takto bolo možné oddeliť samotné spustenie optimalizácie od následného vyhodnotenia experimentov.

3.4.2 Optimalizačné skripty pre rozmiestnenie environmentálnych senzorov

Pri úlohe rozmiestnenia environmentálnych senzorov bola optimalizačná vrstva prispôbená diskretnému charakteru testovacieho prostredia LAQN-BO. Problémová inštancia obsahovala konečnú množinu kandidátnych lokalít a každej lokalite bola priradená hodnota cieľovej funkcie. Z tohto dôvodu nebolo možné pri všetkých algoritmoch pracovať priamo s ľubovoľným bodom spojitého priestoru, ale iba s bodmi patriacimi do kandidátnej domény testovacieho prostredia. KD-strom je dátová štruktúra určená na vyhľadávanie najbližších bodov vo viacrozmernom priestore. V tejto práci bol použitý na premietnutie bodov navrhnutých v spojitom priestore na najbližšiu prípustnú lokalitu z diskretné množiny kandidátskych bodov LAQN. Princíp spočíva v tom, že množina bodov je hierarchicky delená podľa jednotlivých súradnicových osí, čím sa pri hľadaní najbližšieho suseda nemusí porovnávať navrhnutý bod so všetkými kandidátskymi lokalitami. Vďaka tomu bolo možné rýchlo určiť, ktorá diskretná lokalita najlepšie zodpovedá spojitému návrhu optimalizačného algoritmu.

Ako referenčný algoritmus bol pre túto úlohu použitý Random Search. Vzhľadom na diskretný charakter úlohy pracovala jeho implementácia priamo s kandidátnou doménou LAQN. V každom kroku bol náhodne vybraný index jednej prípustnej lokality a následne bola načítaná príslušná hodnota cieľovej funkcie. Algoritmus pri výbere ďalších bodov nepoužíval náhradný model ani informáciu z predchádzajúcich evaluácií, a preto slúžil ako jednoduchý referenčný základ pre porovnanie s modelovo riadenými metódami.

Pri algoritme PyBADs bola vytvorená funkcia sprostredkujúca komunikáciu medzi spojitým charakterom algoritmu a diskretnou doménou úlohy rozmiestnenia environmentálnych senzorov. Bod navrhnutý algoritmom bol premietnutý na najbližšiu kandidátsku lokalitu pomocou KD-stromu. Riešená úloha bola formulovaná ako maximalizačná, zatiaľ čo PyBADs pracuje s minimalizačnou formuláciou úloh. Do optimalizačného skriptu bola odovzdávaná záporná hodnota cieľovej funkcie. Rozpočet evaluácií bol kontrolovaný špecificky pre implementáciu PyBADs. Algoritmus si počas svojho vnútorného behu môže vyžiadať ďalšie

volania cieľovej funkcie. Tým bolo zabezpečené, že počet reálnych vyhodnotení cieľovej funkcie pre danú problémovú inštanciu neprekročil stanovený rozpočet.

Podobný výzvu bolo potrebné prekonať pri algoritme TuRBO. Aj v tomto prípade bol algoritmus sám o sebe schopný navrhovať iba body v spojitom priestore, zatiaľ čo skutočná úloha bola definovaná na konečnej množine kandidátnych lokalít. Rovnako, ako pri algoritme PyBADs, je TuRBO algoritmus určený pre minimalizačnú optimalizáciu a vracal zápornú hodnotu cieľovej funkcie. Pri spustení TuRBO boli z kandidátnej domény odvodené hranice spojitého priestoru pomocou minima a maxima súradníc.

Pri algoritme PySOT bola implementácia prispôbená diskretnému charakteru úlohy podobným spôsobom ako pri algoritmoch PyBADs a TuRBO. Keďže PySOT pracuje so spojitým optimalizačným priestorom, bola nad problémovou inštanciou vytvorená adaptačná vrstva medzi spojitým návrhom algoritmu a diskretnou kandidátnou doménou enviromentálnych senzorov. Bod navrhnutý algoritmom bol pomocou KD-stromu premietnutý na najbližšiu prípustnú lokalitu z kandidátnej množiny a následne bola z dostupných hodnôt problémovej inštancie načítaná príslušná hodnota cieľovej funkcie.

Riešená úloha bola z pohľadu optimalizácie formulovaná ako maximalizačná, zatiaľ čo PySOT pracuje s minimalizačnou formuláciou problému. Preto bola optimalizačnému algoritmu odovzdávaná záporná hodnota cieľovej funkcie. V implementácii boli pripravené tri typy náhradných modelov: model radiálnych bázových funkcií, Gaussovský proces a polynomiálny model. Tieto modeli bolo možné kombinovať so stratégiami DYCORs, SRBF, SOP, EI a LCB. Nie všetky kombinácie však boli prípustné. Stratégie EI a LCB boli obmedzené na použitie s Gaussovským procesom, pretože pri výbere kandidátov pracujú s neistotou predikcie.

Pre PySOT bola vytvorená samostatná reprezentácia LAQN úlohy kompatibilná s rozhraním tejto knižnice. Tá obsahovala dimenziu problému, dolné a horné hranice spojitého priestoru odvodené z kandidátnej domény a evaluačnú funkciu zabezpečujúcu prevod spojitého návrhu na diskretnú lokalitu. Počas jedného behu boli zaznamenávané navštívené lokality, hodnoty cieľovej funkcie, priebeh najlepšej aktuálne dosiahnutej hodnoty, počet volaní cieľovej funkcie, počet unikátne vyhodnotených lokalít a najlepšie nájdené riešenie.

3.4.3 Optimalizačné skripty pre rozmiestnenie veterných turbín

Pri úlohe rozmiestnenia veterných turbín boli optimalizačné skripty prispôbené spojitému charakteru riešeného problému. Problémová inštancia predstavovala konkrétnu konfiguráciu veterného parku, v ktorej bol určený počet veterných turbín, rozmer optimalizačného priestoru, prípustné hranice súradníc jednotlivých turbín a funkcia na vyhodnotenie výkonu celého parku. Kandidátne riešenie bolo reprezentované vektorom hodnôt, ktorý určoval súradnice jednotlivých turbín v riešenej oblasti. Spoločným základom všetkých optimalizačných skriptov bolo vytvorenie tejto problémovej inštancie, získanie prípustných hraníc súradníc a odovzdanie cieľovej funkcie príslušnému optimalizačnému algoritmu. Hodnota cieľovej funkcie bola získavaná volaním evaluačnej funkcie, ktorá pre navrhnuté rozmiestnenie vypočítala hodnotu odvodenú zo simulačného modelu veterného parku.

Ako referenčný algoritmus bol aj pri tejto úlohe použitý Random Search. V jeho implementácii boli kandidátne riešenia vyberané náhodne v rámci dolných a horných hraníc optimalizačného priestoru. Každé navrhnuté riešenie bolo následne vyhodnotené pomocou funkcie pre vyhodnotenie cieľovej funkcie. Priebežne bola ukladaná história navštívených bodov, získaných hodnôt cieľovej funkcie a najlepšej aktuálne dosiahnutej hodnoty cieľovej funkcie. Na konci behu bolo uložené najlepšie nájdené riešenie, jeho hodnota, počet evaluácií potrebných na jeho dosiahnutie a čas behu. Táto implementácia slúžila ako referenčná metóda, voči ktorej mohli byť porovnávané algoritmy riadené náhradným modelom.

Pri algoritme PyBADS bolo možné využiť spojitý charakter úlohy bez dodatočného premietania kandidátnych riešení na diskretnú množinu bodov. Optimalizované premenné predstavovali priame súradnice turbín v riešenej oblasti. Počiatočný bod bol zvolený v rámci prípustných hraníc rozmiestnenia a samotná optimalizácia bola obmedzená stanoveným rozpočtom vyhodnotení cieľovej funkcie.

Algoritmus TuRBO bol pri úlohe rozmiestnenia veterných turbín použitý rovnakým spôsobom napojenia na cieľovú funkciu a simulačný model veterného parku. Kandidátne riešenia generované algoritmom predstavovali návrhy rozmiestnenia turbín, ktoré boli následne vyhodnotené pomocou cieľovej funkcie naviazanej na simulačný model veterného parku. Optimalizačný skript zabezpečil odovzdanie hraníc prípustného priestoru, spustenie algoritmu a zaznamenanie priebehu optimalizácie. Na rozdiel od úlohy rozmiestnenia environmentálnych senzorov tu nebolo potrebné používať KD-strom ani vyhľadávať najbližší prípustný bod, pretože kandidátne riešenia zostávali v spojitom priestore súradníc turbín.

Najrozsiahljšia implementačná časť bola vytvorená pre knižnicu PySOT, pretože v tomto prípade nebola testovaná iba jedna pevná podoba algoritmu. PySOT bol použitý ako rámec, ktorý umožnil kombinovať viaceré typy náhradných modelov so stratégiami výberu ďalšieho bodu. Do experimentov boli zahrnuté konfigurácie založené na Gaussovskom procese, RBF modeli a polynomiálnom modeli, ako aj stratégie EI, LCB, SRBF, DYCORS a SOP. Zaradenie viacerých konfigurácií malo umožniť širšie porovnanie prístupov, ktoré sa líšia tým, ako aproximujú cieľovú funkciu a ako z tejto aproximácie určujú ďalší bod na vyhodnotenie.

Neboli však testované všetky možné kombinácie modelov a stratégií. V implementácii boli použité validačné pravidlá, ktoré obmedzovali výber len na prípustné kombinácie. Stratégie EI a LCB boli používané iba v spojení s Gaussovským procesom, pretože pracujú s predikovanou neistotou modelu. RBF a polynomiálne modeli boli kombinované so stratégiami vhodnými pre kandidátny výber ďalšieho bodu, ako SRBF, DYCORS a SOP.

Pri použití PySOT bolo potrebné prispôsobiť úlohu rozmiestnenia veterných turbín rozhraniu tejto knižnice. Knižnici museli byť poskytnuté informácie o počte rozhodovacích premenných, prípustné hranice súradníc a funkcia, ktorou sa vyhodnocuje navrhnuté rozmiestnenie turbín. Na základe zvolenej konfigurácie bol následne vytvorený príslušný náhradný model a stratégia, ktorá určovala výber ďalšieho kandidátneho riešenia. Pri každom behu boli okrem výsledkov optimalizácie ukladané aj informácie o tom, aký model a aká stratégia boli použité.

Spoločným znakom všetkých optimalizačných skriptov pre úlohu rozmiestnenia veterných turbín bolo to, že pracovali s rovnakou formuláciou úlohy a rovnakým spôsobom

vyhodnocovania kandidátnych riešení. Rozdiel medzi algoritmami spočíval v tom, akým spôsobom navrhovali ďalšie rozmiestnenie turbín na vyhodnotenie. Vďaka tomu mohli byť rozdiely vo výsledkoch interpretované ako dôsledok odlišného prístupu optimalizačných algoritmov.

3.4.4 Hromadné spúšťanie experimentov

Skripty hromadného spúšťania nadväzovali na optimalizačné skripty. Ich úlohou bolo spúšťať jednotlivé algoritmy opakovane pri rovnakých experimentálnych pravidlách a pripraviť výstupy vhodné na následné porovnanie.

Pri úlohe rozmiestnenia environmentálnych senzorov boli skripty hromadného spúšťania navrhnuté tak, aby prechádzali cez všetky pripravené problémové inštancie z testovacieho prostredia LAQN-BO. Pre každú problémovú inštanciu bolo vykonaných 20 nezávislých behov daného algoritmu pri rozpočte 500 evaluácií cieľovej funkcie. Jeden beh bol obmedzený stanoveným rozpočtom evaluácií cieľovej funkcie. Po dokončení behov boli výsledky najprv agregované pre danú problémovú inštanciu a následne aj naprieč všetkými spracovanými inštanciami.

Pri úlohe rozmiestnenia veterných turbín bolo hromadné spúšťanie organizované podľa konfigurácie počtu turbín. Experimenty boli pripravované pre konfigurácie s 5, 10 a 20 veternými turbínami. Pre každú konfiguráciu a každý algoritmus bolo vykonaných 20 nezávislých behov a každý beh bol obmedzený rozpočtom 500 evaluácií cieľovej funkcie. Tým bolo umožnené porovnať správanie algoritmov nielen pri jednej úlohe, ale aj pri rastúcej dimenzii optimalizačného priestoru, pretože počet rozhodovacích premenných rástol spolu s počtom turbín.

Pri oboch úlohách boli zo získaných behov vytvárané súhrnné štatistiky. Pri rozmiestnení environmentálnych senzorov boli sledované najmä najlepšia dosiahnutá hodnota cieľovej funkcie, úspešnosť nájdenia známeho optima, odchýlka od optima, počet evaluácií potrebných na dosiahnutie najlepšej hodnoty a čas behu. Pri rozmiestnení veterných turbín boli sledované najmä najlepšia dosiahnutá hodnota cieľovej funkcie, ktorá predstavovala kumulatívny výkon veterného parku, počet evaluácií do dosiahnutia najlepšej hodnoty a časová náročnosť experimentu.

Výstupy boli ukladané vo forme súborov JSON a CSV. Detailné JSON súbory obsahovali výsledky jednotlivých behov, zatiaľ čo súhrnné JSON súbory obsahovali agregované hodnoty, konfiguráciu experimentu a priemerné konvergenčné krivky. CSV výstupy boli pripravené v jednotnej štruktúre obsahujúcej identifikátor algoritmu, identifikátor problému, číslo behu, číslo evaluácie, hodnotu cieľovej funkcie a najlepšiu doteraz dosiahnutú hodnotu. Takto pripravené výstupy boli použité pri tvorbe výsledkových tabuliek a grafov v nasledujúcej kapitole.

3.4.5 Výpočtové prostredie

Experimenty boli vykonávané na osobnom počítači Mac mini 2024 s procesorom Apple M4 a operačnou pamäťou 16 GB na operačnom systéme macOS Tahoe 26.3.1. Uvedenie výpočtového prostredia je v kontexte porovnávania algoritmov založených na náhradnom

modeli považované za dôležité najmä z dôvodu interpretácie časovej náročnosti experimentov. Namerané časy behov závisia nielen od samotného algoritmu a riešenej úlohy, ale aj od dostupného hardvérového výkonu.

Implementácia a spúšťanie experimentov boli realizované v programovacom jazyku Python. Pre jednotlivé časti práce boli využívané samostatné virtuálne prostredia, aby mohli byť oddelene závislosti a programovacie knižnice potrebné pre použité benchmarky, simulačné nástroje a optimalizačné knižnice. Tento prístup bol zvolený najmä z dôvodu rozdielnych požiadaviek na verzie balíkov pri knižniciach FLORIS, pyBADs, TuRBO a PySOT.

Oddelenie virtuálnych prostredí bolo dôležité najmä pri úlohe rozmiestnenia veterných turbín, kde bolo potrebné zosúladiť simulačný nástroj FLORIS s použitými optimalizačnými algoritmami. Rovnako bolo potrebné prihliadať na kompatibilitu jednotlivých knižníc využívaných pri implementovaní algoritmov. Vďaka samostatným prostrediam bolo možné minimalizovať konflikty medzi závislosťami a zachovať stabilitu experimentálneho rámca.

Časy behov testovaných algoritmov musia byť interpretované v kontexte uvedeného výpočtového prostredia. Pri porovnávaní algoritmov je možné tieto časy využiť na relatívne porovnanie metód v rámci rovnakého experimentálneho nastavenia. Nemožno ich však chápať ako univerzálne hodnoty nezávislé od použitého hardvéru a softvérového prostredia.

3.5 Hodnotiace metriky

Na vyhodnotenie výsledkov optimalizačných algoritmov boli použité metriky, ktoré zachytávajú kvalitu dosiahnutého riešenia, priebeh zlepšovania počas optimalizácie, efektivitu využitia dostupného rozpočtu evaluácií a časovú náročnosť výpočtu. Cieľom bolo hodnotiť algoritmy s náhradným modelom porovnať z pohľadu viacerých kritérií. Algoritmus môže napríklad dosiahnuť kvalitné riešenie, ale za cenu dlhej doby výpočtu, alebo môže svoje najlepšie riešenie nájsť veľmi skoro, no toto riešenie nemusí byť v absolútnom porovnaní najlepšie.

Pri úlohe rozmiestnenia environmentálnych senzorov boli navyše použité aj metriky založené na známom optime. Keďže v tejto diskkrétnej úlohe bola známa najlepšia hodnota v kandidátnej doméne, bolo možné priamo vyhodnotiť, či algoritmus optimum našiel a ako veľmi sa od neho prípadne odchyľil. Pri úlohe rozmiestnenia veterných turbín sa hodnotenie sústredilo najmä na dosiahnutú hodnotu cieľovej funkcie, konvergenčný priebeh, efektivitu využitia evaluácií a čas výpočtu.

Metriky boli sledované jednak na úrovni jednotlivých behov algoritmov, jednak vo forme agregovaných hodnôt vypočítaných z viacerých opakovaní. Takýto spôsob vyhodnotenia umožnil posúdiť nielen najlepší dosiahnutý výsledok, ale aj stabilitu algoritmov pri opakovanom spúšťaní.

3.5.1 Kvalita dosiahnutého riešenia

Za kvalitné dosiahnuté riešenie sa v tejto práci považuje také riešenie, ktoré dosahuje čo najvyššiu hodnotu cieľovej funkcie. Obe aplikačné úlohy boli z hľadiska optimalizácie chápané ako maximalizačné problémy. Pri rozmiestnení environmentálnych senzorov bolo cieľom nájsť

kandidátnu s čo najvyššou hodnotou sledovaného kritéria. Pri rozmiestnení veterných turbín bolo cieľom dosiahnuť čo najvyšší energetický výnos veternej farmy.

V jednom behu algoritmu bola ako metrika kvality použitá najlepšia hodnota cieľovej funkcie dosiahnutá počas daného behu. Táto hodnota vyjadruje, aké najlepšie riešenie algoritmus našiel v rámci dostupného rozpočtu evaluácií. Vyššia hodnota znamená kvalitnejšie dosiahnuté riešenie.

Pri opakovaných behoch algoritmu boli z hodnôt jednotlivých behov vypočítané súhrnné štatistiky v podobe najlepšej priemernej hodnoty, medián najlepších hodnôt a smerodajná odchýlka najlepších hodnôt. Priemerná hodnota umožnila porovnať celkovú výkonnosť algoritmov, medián poskytol doplnkový pohľad menej citlivý na extrémne behy a smerodajná odchýlka vyjadrovala varibialitu výsledkov algoritmu pri opakovanom spúšťaní.

3.5.2 Konvergenčný priebeh

Na sledovanie priebehu optimalizácie bola použitá najlepšia doteraz dosiahnutá hodnota po každej evaluácii cieľovej funkcie. Táto metrika ukazuje, ako sa algoritmus počas behu postupne zlepšoval a v ktorej fáze výpočtu nachádzal kvalitnejšie riešenia.

Z týchto hodnôt boli vytvárané konvergenčné krivky. Konvergenčná krivka zobrazuje vývoj najlepšej doteraz dosiahnutej hodnoty v závislosti od počtu evaluácií. Umožňuje preto porovnať finálny výsledok algoritmov, a rýchlosť ich zlepšovania počas optimalizácie.

Pri opakovaných behoch bol použitý priemerný konvergenčný priebeh. Ten bol vypočítaný ako priemer konvergenčných kriviek viacerých behov.

3.5.3 Efektivita využitia evaluácií

Efektivita využitia rozpočtu evaluácií bola hodnotená podľa počtu evaluácií potrebných na dosiahnutie najlepšieho riešenia v danom behu. Táto metrika vyjadruje, v ktorej fáze optimalizácie algoritmus našiel svoje najlepšie riešenie.

Nižší počet evaluácií do najlepšieho riešenia znamená, že algoritmus dosiahol svoje maximum v rámci daného behu skôr. Túto metriku však nemožno interpretovať samostatne bez ohľadu na kvalitu dosiahnutého riešenia. Algoritmus môže svoje najlepšie riešenie nájsť veľmi skoro, ale ak je toto riešenie nekvalitné, nejde o lepší výsledok než pri algoritme, ktorý sa zlepšoval dlhšie a dosiahol vyššiu finálnu hodnotu.

Pri opakovaných behoch boli hodnotené priemerný počet evaluácií do najlepšieho riešenia, medián počtu evaluácií a smerodajná odchýlka. Tieto hodnoty umožnili posúdiť, či algoritmus nachádzal svoje najlepšie riešenia stabilne v podobnej fáze optimalizácie, alebo či sa tento okamih medzi jednotlivými behmi výrazne líšil.

3.5.4 Časová náročnosť

Časová náročnosť bola hodnotená podľa času potrebného na vykonanie všetkých experimentálnych behov daného algoritmu. Táto metrika zahŕňala čas potrebný na generovanie kandidátnych riešení a vyhodnocovanie cieľovej.

Pri opakovaných behoch bol sledovaný priemerný čas jedného behu a jeho smerodajná odchýlka. Priemerný čas umožnil porovnať výpočtovú náročnosť algoritmov, zatiaľ čo smerodajná odchýlka ukázala variabilitu v časovej náročnosti medzi jednotlivými behmi.

Okrem času jedného behu bol, pri experimentoch hromadného spustenia optimalizácie s náhradným modelom, sledovaný aj celkový čas potrebný na vykonanie celej série experimentov. Pri interpretácii časových výsledkov bolo preto potrebné rozlišovať medzi časom jedného behu, priemerným časom cez viac behov a celkovým časom celej dávky experimentov.

3.5.5 Špecifické metriky environmentálnych senzorov

Pri úlohe rozmiestnenia environmentálnych senzorov boli použité aj metriky vychádzajúce zo známeho optima. Keďže išlo o diskretnú úlohu s konečnou kandidátnou doménou, bolo možné určiť, či algoritmus v danom behu našiel optimálne riešenie.

Úspešnosť nájdenia optima vyjadrovala, či algoritmus v konkrétnom behu dosiahol známu optimálnu hodnotu danej problémovej inštancie. Pri viacerých behoch bola následne hodnotená percentuálna úspešnosť, teda podiel behov, v ktorých algoritmus optimum našiel. Vyššia úspešnosť znamenala spoľahlivejšie dosahovanie optimálneho riešenia.

Ďalšou metrikou bola odchýlka od optima. Tá vyjadrovala rozdiel medzi najlepšou hodnotou dosiahnutou algoritmom a známou optimálnou hodnotou danej problémovej inštancie. Nižšia odchýlka znamenala, že algoritmus sa dostal bližšie k optimu. Pri opakovaných behoch boli sledované priemerná odchýlka, medián odchýlky a smerodajná odchýlka, ktoré umožnili posúdiť presnosť a stabilitu algoritmu vzhľadom na známe optimum.

Pri tejto úlohe bol sledovaný aj počet unikátne navštívených kandidátnych lokalít. Táto metrika bola dôležitá preto, že niektoré algoritmy navrhovali riešenia v spojitom priestore, ktoré sa následne premietali na najbližšiu prípustnú lokalitu v diskretnej doméne. V dôsledku toho mohlo viacero navrhnutých bodov viesť k tej istej kandidátnej lokalite. Počet unikátne navštívených lokalít preto poskytoval doplnkovú informáciu o tom, ako efektívne algoritmus pokrýval diskretný kandidátny priestor.

Okrem toho bol sledovaný aj počet volaní cieľovej funkcie. Táto hodnota umožnila skontrolovať, ako bol využitý stanovený rozpočet evaluácií a či algoritmus pracoval v rámci rovnakých experimentálnych obmedzení ako ostatné porovnávané metódy.

3.5.6 Význam metrik pre porovnanie výsledkov

Použitý súbor metrik poskytol základ pre porovnanie algoritmov vo výsledkovej časti práce. Metriky kvality riešenia umožnili porovnať finálne dosiahnuté hodnoty, konvergenčné metriky zachytili priebeh optimalizácie, metriky evaluácií vyjadrili efektivitu využitia dostupného rozpočtu a časové metriky doplnili informáciu o výpočtovej náročnosti.

Pri úlohe rozmiestnenia environmentálnych senzorov bolo hodnotenie rozšírené o úspešnosť nájdenia známeho optima, odchýlku od optima a počet unikátne navštívených kandidátskych lokalít. Pri úlohe rozmiestnenia veterných turbín bola pozornosť sústredená najmä na dosiahnutý energetický výkon, priebeh optimalizácie, efektivitu využitia evaluácií a časovú náročnosť pri rôznom počte turbín.

Takto definované metriky umožnili hodnotiť algoritmy z viacerých hľadísk. Výsledky preto neboli posudzované iba podľa toho, ktorý algoritmus dosiahol najvyššiu finálnu hodnotu, ale aj podľa toho, ako rýchlo sa k nej približoval, ako stabilne sa správal pri opakovaných behoch a akú výpočtovú réžiu jeho použitie vyžadovalo.

4 VÝSLEDKY EXPERIMENTOV

Táto kapitola prezentuje výsledky experimentálneho porovnania vybraných optimalizačných algoritmov na dvoch aplikačných úlohách. Hodnotenie nadväzuje na metriky definované v predchádzajúcej kapitole a je rozdelené podľa kvality dosiahnutého riešenia, konvergenčného priebehu, efektivity využitia evaluácií a časovej náročnosti. Výsledky sú prezentované vo forme tabuliek a konvergenčných grafov.

V tabuľkách sú algoritmy zoradené podľa hlavného porovnávacieho kritéria danej tabuľky. Pri tabuľkách kvality riešení sú riadky zoradené zostupne podľa priemernej najlepšej dosiahnutej hodnoty, keďže vyššia hodnota predstavuje kvalitnejšie riešenie. Ostatné stĺpce dopĺňajú hlavné kritérium o informáciu o odchýlke od optima, stabilite výsledkov alebo úspešnosti nájdenia optimálneho riešenia. Pri interpretácii výsledkov je preto potrebné rozlišovať medzi hlavným kritériom zoradenia a doplnkovými metrikami, ktoré spresňujú vlastnosti dosiahnutých výsledkov.

4.1 Výsledky úlohy rozmiestnenia environmentálnych senzorov

V tejto časti sú vyhodnotené výsledky úlohy rozmiestnenia environmentálnych senzorov. Keďže ide o diskrétnu úlohu so známym optimom pre jednotlivé problémové inštancie, bolo možné porovnávať nielen dosiahnutú hodnotu cieľovej funkcie, ale aj odchýlku od optimálnej hodnoty a úspešnosť nájdenia optima. Referenčná metóda Random Search je v tabuľkách ponechaná ako základné porovnanie, zatiaľ čo hlavná pozornosť je venovaná rozdielom medzi algoritmami založenými na náhradnom modeli.

4.1.1 Kvalita dosiahnutých riešení

Tabuľka 1 uvádza súhrnné porovnanie kvality dosiahnutých riešení pri úlohe rozmiestnenia environmentálnych senzorov. Hlavným porovnávacím kritériom je priemerná najlepšia hodnota cieľovej funkcie dosiahnutá počas opakovaných behov. Riadky tabuľky sú zoradené zostupne podľa tejto hodnoty. Stĺpce s priemernou, mediánovou a smerodajnou odchýlkou od optima dopĺňajú informáciu o tom, ako blízko sa algoritmus dostával k známej optimálnej hodnote a ako stabilné boli jeho výsledky. Stĺpec úspešnosti vyjadruje podiel behov, v ktorých bolo nájdené optimum.

Tab. 1: Súhrnné porovnanie kvality dosiahnutých riešení pri úlohe rozmiestnenia environmentálnych senzorov

Algoritmus	Priemerná najlepšia hodnota	Priemerná odchýlka od optima	Medián odchýlky od optima	Smerodajná odchýlka od optima	Úspešnosť nájdenia optima [%]
Random Search	1,710	0,000	0,000	0,000	100,000
PySOT-RBF-SRBF	1,679	0,032	0,005	0,055	85,094
PySOT-RBF-DYCORS	1,673	0,038	0,009	0,064	83,318
TuRBO	1,673	0,038	0,020	0,059	83,084
PyBADs	1,669	0,042	0,000	0,081	80,210
PySOT-POLY-DYCORS	1,628	0,082	0,048	0,104	67,196
PySOT-POLY-SRBF	1,626	0,085	0,051	0,104	65,701
PySOT-POLY-SOP	1,362	0,349	0,314	0,252	23,014
PySOT-RBF-SOP	1,232	0,478	0,459	0,319	10,070

Najlepšie hodnoty boli dosiahnuté referenčnou metódou Random Search. Tento výsledok bol spôsobený najmä charakterom riešenej úlohy. Úloha rozmiestnenia environmentálnych senzorov bola formulovaná ako diskretný problém, v ktorom boli kandidátne riešenia obmedzené na konečnú množinu lokalít odvodených z dostupných meracích staníc, vychádzajúcich z reálnych meraní vykonávaných počas 214 dní. Pri stanovenom experimentálnom rozpočete 500 evaluácií umožnil túto kandidátnu doménu pokryť veľmi účinne. Náhodná referenčná metóda preto v tomto prípade predstavovala príliš silné porovnanie pre algoritmy založené na náhradnom modeli. Pri modelovo riadených algoritmoch navyše dochádzalo k prevodu návrhov zo spojitého priestoru na najbližšie prípustné diskretné lokality, čo mohlo znížiť výhodu ich výberovej stratégie.

Spomedzi algoritmov založených na náhradnom modeli bola najlepšia hodnota dosiahnutá konfiguráciou knižnice PySOT s náhradným modelom RBF a výberovou stratégiou SRBF. Veľmi blízke výsledky boli dosiahnuté aj konfiguráciou knižnice PySOT s náhradným modelom RBF a výberovou stratégiou DYCORS, algoritmom TuRBO a algoritmom PyBADs.

Najslabšie výsledky boli dosiahnuté konfiguráciami knižnice PySOT využívajúcimi výberovú stratégiu SOP. Z pohľadu doplnkových metrík teda tieto konfigurácie vykazovali najväčšiu vzdialenosť od známeho optima a najnižšiu spoľahlivosť jeho nájdenia.

4.1.2 Konvergenčný priebeh algoritmov

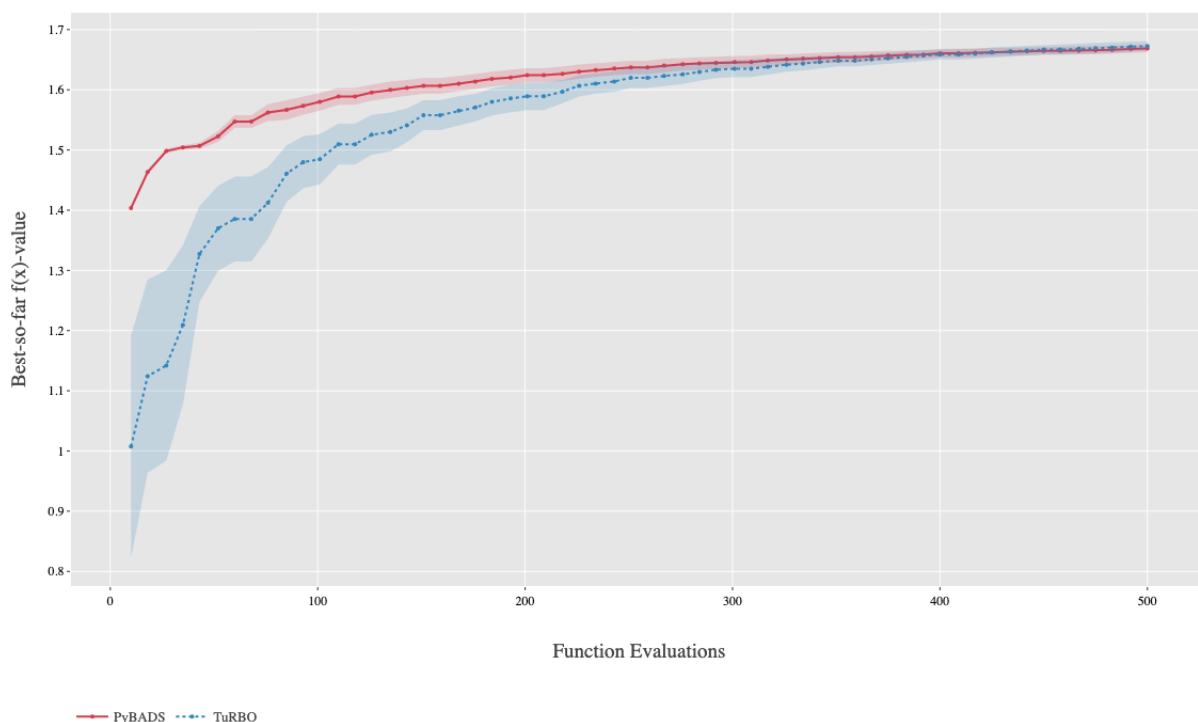
Konvergenčné priebehy algoritmov boli vyhodnotené pomocou nástroja IOHanalyzer. Keďže pri úlohe rozmiestnenia environmentálnych senzorov bolo spracovaných 214 problémových inštancií, 20 behov a 500 evaluácií pre každý algoritmus, výstupné dáta predstavovali veľmi rozsiahly súbor výsledkov. Pri zlúčení všetkých algoritmov vznikol súbor s viac než tromi

miliónmi riadkov, preto bolo potrebné dáta pred vizualizáciou agregovať a rozdeliť do menších tematických skupín.

Výsledné grafy zobrazujú priemerný priebeh dosahovania najlepšej hodnoty cieľovej funkcie v závislosti od počtu evaluácií. Krivky zobrazujú priemerný priebeh a tieňované pásma predstavuje smerodajnú odchýlku medzi opakovanými behmi. Vyšší priebeh znamená, že algoritmus v danej fáze optimalizácie dosahoval kvalitnejšie riešenia.

Grafy v tejto podkapitole boli zostavené tak, aby sa porovnávali najmä algoritmy založené na náhradnom modeli. Referenčná metóda Random Search bola z týchto čiastkových grafov vynechaná, pretože jej priebeh pri úlohe rozmiestnenia environmentálnych senzorov výrazne dominoval a znižoval čitateľnosť rozdielov medzi modelovo riadenými algoritmami. Do porovnania bola opätovne zaradená až v súhrnnom grafe, kde je zobrazený celkový kontext výsledkov všetkých porovnávaných algoritmov.

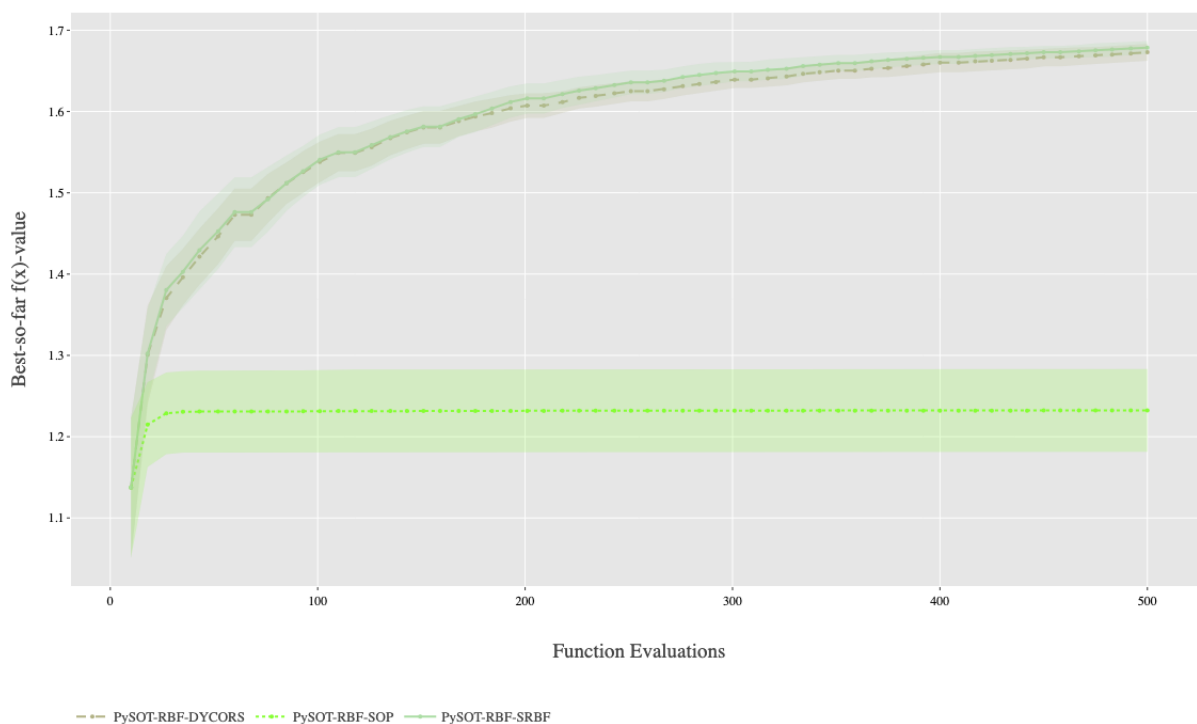
Na začiatok boli porovnané algoritmy PyBADS a TuRBO. Následne boli vyhodnotené konfigurácie knižnice PySOT s rovnakým typom náhradného modelu, konkrétne s modelom RBF a s polynomiálnym modelom. Potom boli porovnané rovnaké výberové stratégie pri dvoch typoch náhradného modelu, konkrétne stratégie DYCORs, SRBF a SOP. Posledný graf, zobrazovaný v logaritmickej mierke, slúži ako súhrnné porovnanie všetkých algoritmov.



Obr. 8: Konvergenčný priebeh algoritmov PyBADS a TuRBO pri úlohe rozmiestnenia environmentálnych senzorov.

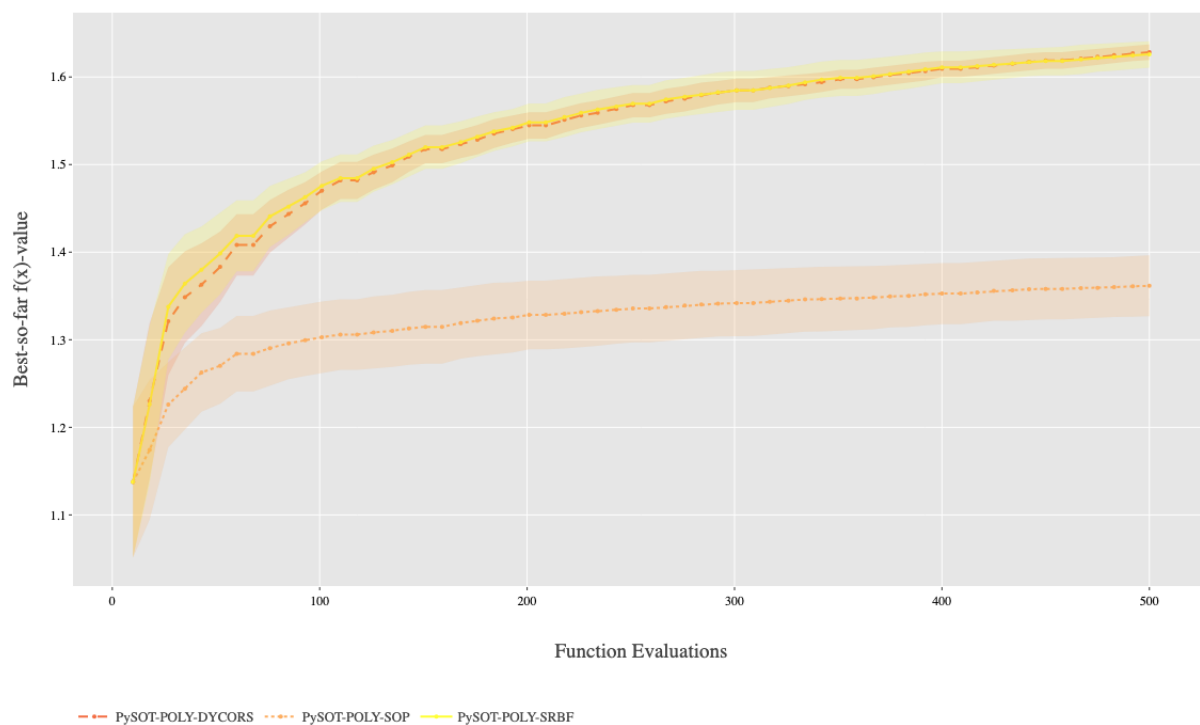
Na obrázku 1 je zobrazené porovnanie algoritmov PyBADS a TuRBO. Vyšší priebeh najlepšej doteraz dosiahnutej hodnoty bol počas väčšej časti optimalizácie dosiahnutý algoritmom PyBADS. Algoritmus TuRBO dosahoval v úvodnej fáze nižšie hodnoty, ale počas

ďalších evaluácií sa postupne zlepšoval. Ku koncu dostupného rozpočtu evaluácií sa rozdiel medzi oboma algoritmami zmenšoval, pričom ich finálne hodnoty boli veľmi blízke.



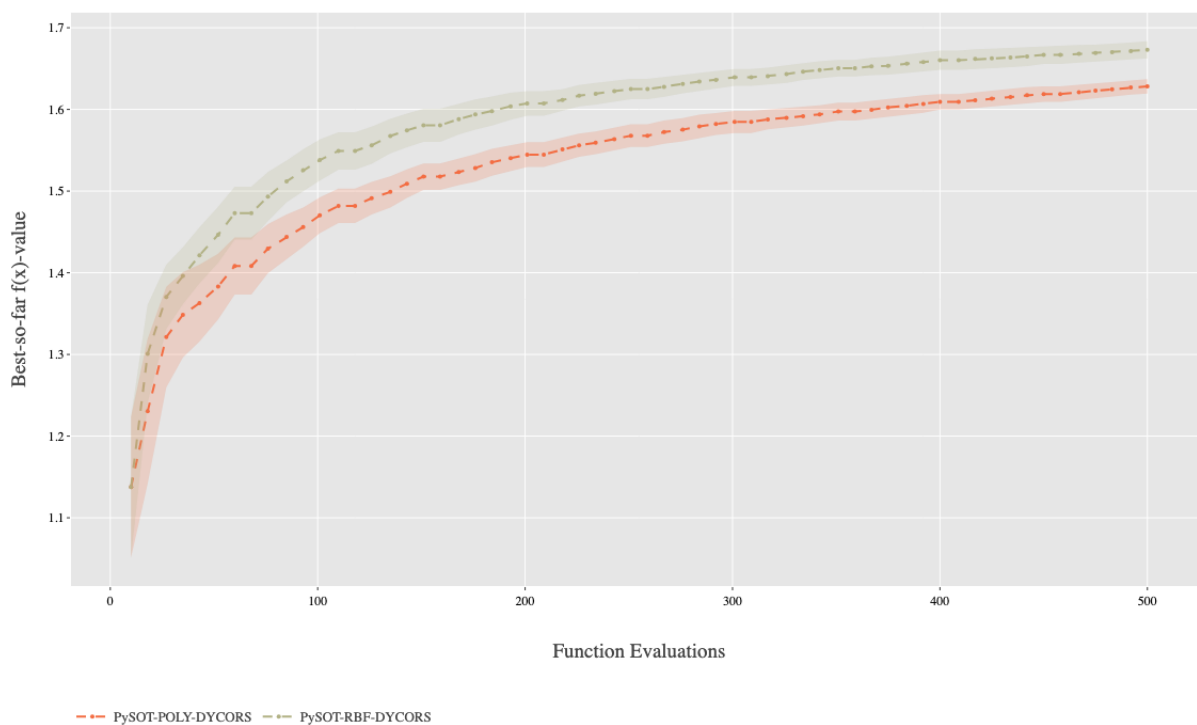
Obr. 9: Konvergenčný priebeh konfigurácií knižnice PySOT s náhradným modelom RBF pri úlohe rozmiestnenia environmentálnych senzorov.

Na obrázku 2 sú porovnané konfigurácie knižnice PySOT využívajúce náhradný model RBF. Najvyššie priebehy boli dosiahnuté konfiguráciami s výberovými stratégiami SRBF a DYCORS. Tieto dve konfigurácie sa počas väčšiny optimalizácie pohybovali veľmi blízko pri sebe a postupne sa zlepšovali až do konca dostupného rozpočtu evaluácií. Najnižší priebeh bol zaznamenaný pri konfigurácii so stratégiou SOP, ktorej hodnota sa stabilizovala už v úvodnej časti optimalizácie.



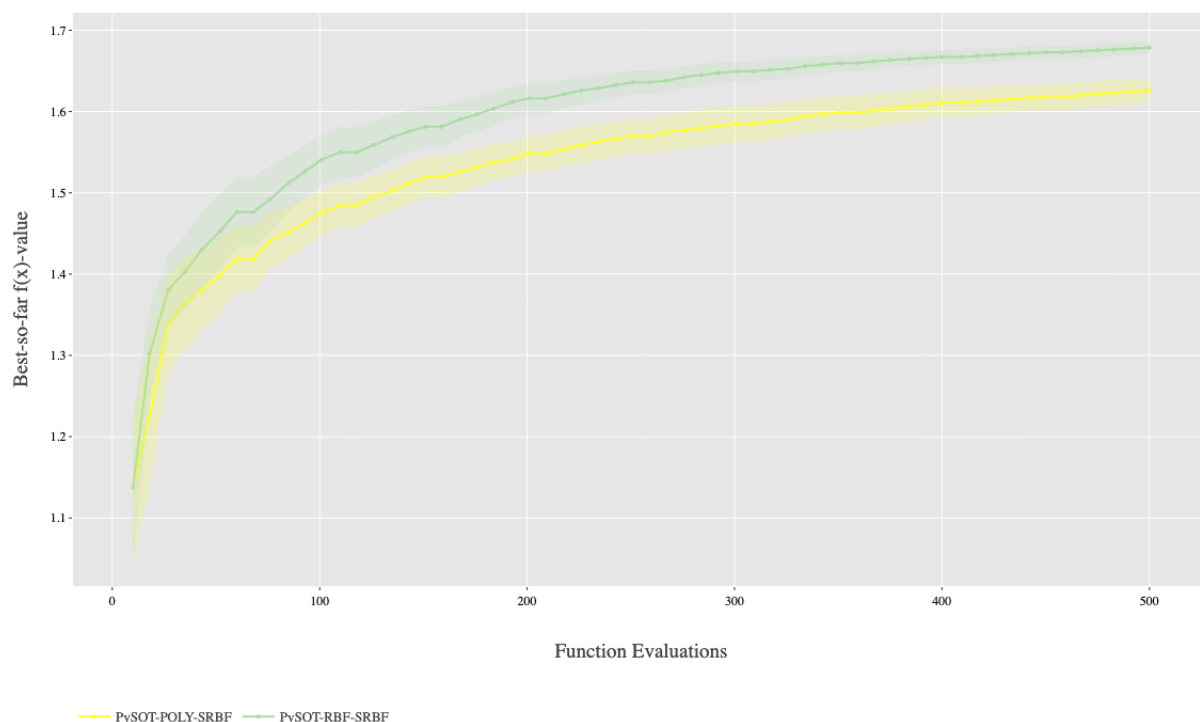
Obr. 10: Konvergenčný priebeh konfigurácií knižnice PySOT s polynomiálnym náhradným modelom pri úlohe rozmiestnenia environmentálnych senzorov.

Na obrázku 3 sú zobrazené konfigurácie knižnice PySOT využívajúce polynomiálny náhradný model. Najvyšší priebeh bol dosiahnutý konfiguráciou s výberovou stratégiou SRBF. Konfigurácia so stratégiou DYCORS dosahovala nižšie hodnoty, ale počas celého rozpočtu evaluácií sa postupne zlepšovala. Najnižšie hodnoty boli dosiahnuté konfiguráciou so stratégiou SOP, ktorá sa síce v úvodnej fáze zlepšila, ale následne zostala výrazne pod konfiguráciami so stratégiami SRBF a DYCORS.



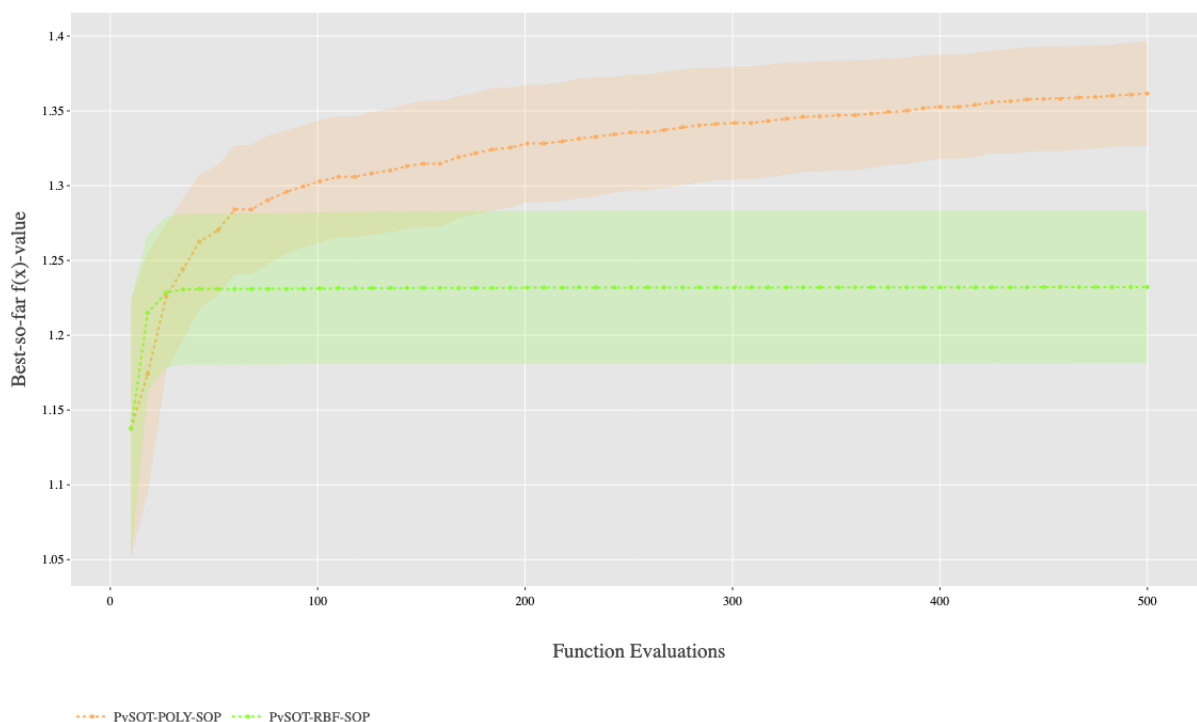
Obr. 11: Konvergenčný priebeh konfigurácií knižnice PySOT so stratégiou DYCORDS pri náhradnom modeli RBF a polynomiálnom náhradnom modeli.

Na obrázku 4 je porovnaná výberová stratégia DYCORDS pri dvoch typoch náhradného modelu. Vyšší priebeh bol dosiahnutý pri náhradnom modeli RBF. Polynomiálny náhradný model dosahoval nižšie hodnoty, ale počas optimalizácie sa priebežne zlepšoval. Rozdiel medzi oboma konfiguráciami zostal viditeľný aj vo finálnej časti rozpočtu evaluácií.



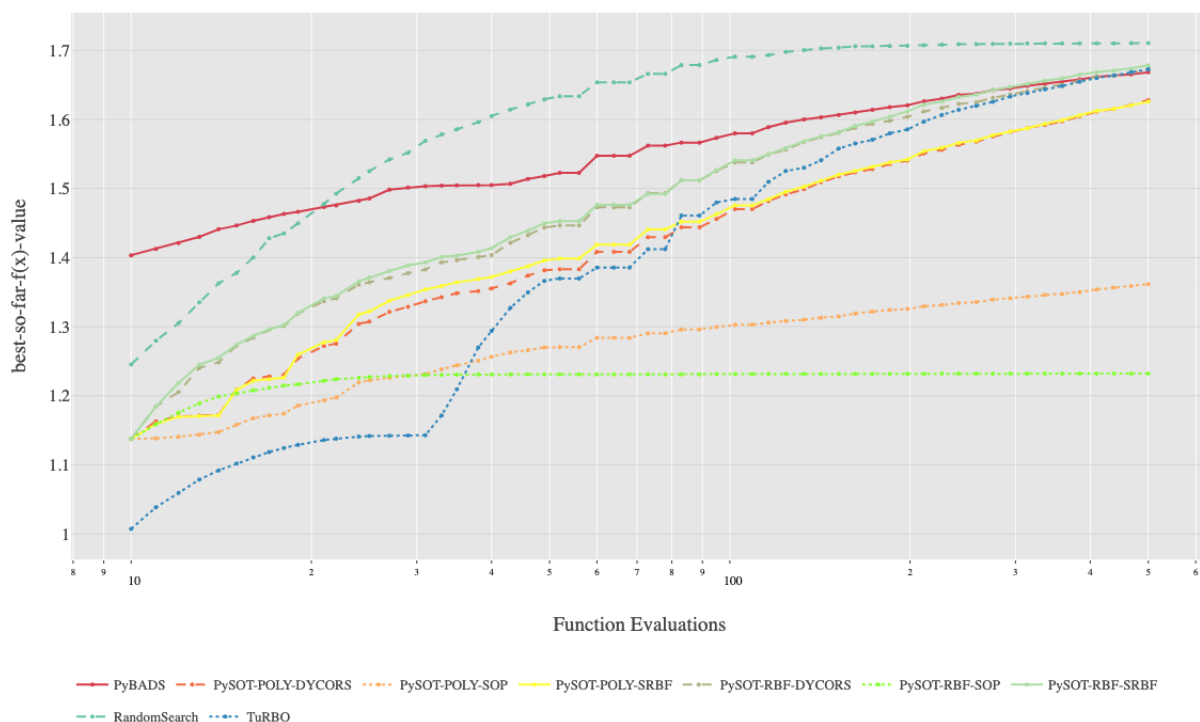
Obr. 12: Konvergenčný priebeh konfigurácií knižnice PySOT so stratégiou SRBF pri náhradnom modeli RBF a polynomiálnom náhradnom modeli.

Na obrázku 5 je porovnaná výberová stratégia SRBF pri dvoch typoch náhradného modelu. Vyšší priebeh bol dosiahnutý pri náhradnom modeli RBF. Polynomiálny náhradný model dosahoval mierne nižšie hodnoty, pričom rozdiel medzi oboma konfiguráciami bol zachovaný počas väčšej časti optimalizácie. Obe konfigurácie sa zlepšovali postupne a ich priebehy zostali rastúce až do záverečnej časti rozpočtu evaluácií.



Obr. 13: Konvergenčný priebeh konfigurácií knižnice PySOT so stratégiou SOP pri náhradnom modeli RBF a polynomiálnom náhradnom modeli.

Na obrázku 6 je porovnaná výberová stratégia SOP pri dvoch typoch náhradného modelu. Vyšší priebeh bol dosiahnutý pri polynomiálnom náhradnom modeli. Konfigurácia využívajúca náhradný model RBF dosahovala nižšie hodnoty a jej priebeh sa stabilizoval už po malej časti dostupného rozpočtu evaluácií. V porovnaní s ostatnými konfiguráciami knižnice PySOT boli pri stratégii SOP zaznamenané výrazne nižšie konvergenčné priebehy.



Obr. 14: Súhrnný konvergenčný priebeh všetkých porovnávaných algoritmov pri úlohe rozmiestnenia environmentálnych senzorov v logaritmickej mierke osi počtu evaluácií.

Na obrázku 7 je uvedené súhrnné porovnanie všetkých algoritmov vrátane referenčnej metódy Random Search. Táto metóda nebola zaradená do predchádzajúcich čiastkových grafov, aby bolo možné zreteľnejšie porovnať rozdiely medzi algoritmi založenými na náhradnom modeli. V súhrnnom grafe je však zobrazená z dôvodu celkového porovnania s referenčným prístupom.

Najvyšší priebeh bol dosiahnutý metódou Random Search, ktorá sa už pri nízkom počte evaluácií dostala na vysokú úroveň a následne sa stabilizovala. Spomedzi algoritmov založených na náhradnom modeli boli najvyššie priebehy dosiahnuté algoritmom PyBADS, algoritmom TuRBO a konfiguráciami knižnice PySOT využívajúcimi náhradný model RBF so stratégiami SRBF a DYCORS. Najnižšie priebehy boli zaznamenané pri konfiguráciách knižnice PySOT so stratégiou SOP, najmä pri použití náhradného modelu RBF.

Konvergenčné grafy dopĺňajú tabuľkové hodnotenie kvality riešení tým, že zobrazujú vývoj najlepšej doteraz dosiahnutej hodnoty počas dostupného rozpočtu evaluácií. Z čiastkových grafov vyplýva, že medzi algoritmi založenými na náhradnom modeli boli najpriaznivejšie priebehy dosiahnuté najmä pri algoritme PyBADS, algoritme TuRBO a konfiguráciách knižnice PySOT využívajúcich náhradný model RBF so stratégiami SRBF a DYCORS. V nasledujúcej časti je preto vyhodnotená efektivita využitia evaluácií, teda počet vyhodnotení cieľovej funkcie potrebný na dosiahnutie najlepšieho riešenia.

4.1.3 Efektivita využitia evaluácií

Tabuľka 2 uvádza efektivitu využitia evaluácií pri úlohe rozmiestnenia environmentálnych senzorov. Hlavným porovnávacím kritériom je priemerný počet evaluácií potrebných na dosiahnutie najlepšieho riešenia v danom behu experimentu. Stĺpec priemerného počtu unikátnych evaluácií dopĺňa informáciu o tom, koľko rozdielnych kandidátnych lokalít bolo v priemere skutočne navštívených.

Tab. 2: Efektivita využitia evaluácií pri úlohe rozmiestnenia environmentálnych senzorov

Algoritmus	Priemerný počet evaluácií do najlepšieho riešenia	Podiel rozpočtu do najlepšieho riešenia [%]	Priemerný počet unikátnych evaluácií
PySOT-RBF-SOP	6,195	1,239	11,708
Random Search	47,023	9,405	43,449
PySOT-POLY-SOP	48,832	9,766	16,556
PyBADs	85,597	17,119	30,826
PySOT-RBF-SRBF	121,122	24,224	37,750
PySOT-RBF-DYCORs	123,020	24,604	37,583
PySOT-POLY-SRBF	124,469	24,894	33,467
PySOT-POLY-DYCORs	135,285	27,057	33,737
TuRBO	138,881	27,776	37,908

Najnižší priemerný počet evaluácií do najlepšieho riešenia bol zaznamenaný pri konfigurácii knižnice PySOT s náhradným modelom RBF a výberovou stratégiou SOP. Táto konfigurácia dosiahla svoje najlepšie riešenie v priemere po 6,195 evaluáciách, čo predstavuje 1,239 % dostupného rozpočtu. Tento výsledok však treba hodnotiť spolu s tabuľkou kvality riešení, pretože rovnaká konfigurácia dosiahla najnižšiu úspešnosť nájdenia optima. Skoré dosiahnutie vlastného najlepšieho riešenia preto v tomto prípade znamenalo skôr skorú stabilizáciu na slabšej hodnote než vysokú efektivitu z pohľadu kvality riešenia.

Referenčnou metódou Random Search bolo najlepšie riešenie dosiahnuté v priemere po 47,023 evaluáciách, čo predstavuje 9,405 % dostupného rozpočtu. Zároveň však dosiahla najvyšší priemerný počet unikátnych evaluácií, konkrétne 43,449. Táto hodnota ukazuje, že náhodná metóda v diskkrétnej kandidátnej doméne pokrývala väčší počet rozdielnych kandidátnych lokalít než väčšina modelovo riadených algoritmov.

Spomedzi algoritmov založených na náhradnom modeli dosiahla okrem stratégie SOP skoré najlepšie riešenie aj konfigurácia knižnice PySOT s polynomiálnym náhradným modelom a výberovou stratégiou SOP. Jej najlepšia hodnota bola dosiahnutá v priemere po 48,832 evaluáciách, teda po 9,766 % dostupného rozpočtu. Vzhľadom na výsledky kvality riešení však aj v tomto prípade platí, že skoré dosiahnutie vlastného maxima nebolo spojené s najvyššou kvalitou dosiahnutého riešenia.

Stredné pásmo tabuľky tvorili algoritmus PyBADs a konfigurácie knižnice PySOT využívajúce stratégie SRBF a DYCORS. Pri algoritme PyBADs bolo najlepšie riešenie dosiahnuté v priemere po 85,597 evaluáciách, čo zodpovedá 17,119 % rozpočtu.

Najvyšší priemerný počet evaluácií do najlepšieho riešenia bol zaznamenaný pri algoritme TuRBO, ktorý dosiahol svoje najlepšie riešenie v priemere po 138,881 evaluáciách, teda po 27,776 % dostupného rozpočtu. Zároveň však dosiahol vysoký priemerný počet unikátnych evaluácií, konkrétne 37,908, čo bolo porovnateľné s konfiguráciami knižnice PySOT založenými na náhradnom modeli RBF.

4.1.4 Časová náročnosť

Tabuľka 3 uvádza časovú náročnosť algoritmov pri úlohe rozmiestnenia environmentálnych senzorov. Hlavným porovnávacím kritériom je priemerný čas behu, podľa ktorého sú riadky tabuľky zoradené vzostupne.

Tab. 3: Časová náročnosť algoritmov pri úlohe rozmiestnenia environmentálnych senzorov

Algoritmus	Priemerný čas behu [s]	Priemerný čas jednej evaluácie [ms]	Celkový čas experimentu [min]	Celkový čas experimentu [h]
Random Search	0,001	0,002	0,068	0,001
PySOT-RBF-DYCORS	0,399	0,798	28,456	0,474
PySOT-RBF-SRBF	0,404	0,808	28,819	0,480
PySOT-POLY-DYCORS	0,440	0,879	31,368	0,523
PySOT-POLY-SRBF	0,450	0,899	32,073	0,535
PyBADs	1,704	3,408	121,546	2,026
TuRBO	14,772	29,544	1 053,730	17,562
PySOT-POLY-SOP	30,234	60,469	2 156,727	35,945
PySOT-RBF-SOP	51,830	103,660	3 697,218	61,620

Najnižšia časová náročnosť bola zaznamenaná pri referenčnej metóde Random Search, ktorej priemerný čas jedného behu bol 0,001 s. Priemerný čas jednej evaluácie dosiahol 0,002 ms a celkový čas experimentu bol 0,068 min. Tento výsledok bol očakávaný, keďže pri tejto metóde nie je vytváraný ani aktualizovaný náhradný model a nevykonávajú sa dodatočné modelovo riadené výpočty.

Spomedzi algoritmov založených na náhradnom modeli boli časovo najmenej náročné konfigurácie knižnice PySOT využívajúce náhradný model RBF alebo polynomiálny náhradný model v kombinácii s výberovými stratégiami DYCORS a SRBF.

Algoritmus PyBADs dosiahol priemerný čas jedného behu 1,704 s a celkový čas experimentu 121,546 min, teda 2,026 h. V porovnaní s najrýchlejšími konfiguráciami knižnice

PySOT bol časovo náročnejší, ale stále výrazne menej náročný než algoritmus TuRBO a konfigurácie so stratégiou SOP.

Najvyššia časová náročnosť bola zaznamenaná pri konfiguráciách knižnice PySOT s výberovou stratégiou SOP. Experiment algoritmu s polynomiálnym náhradným modelom trval 35,945 h. Experiment algoritmu s náhradným modelom RBF trval zďaleka najdlhšie, a to 61,620 h.

4.2 Výsledky úlohy rozmiestnenia veterných turbín

Táto kapitola prezentuje výsledky experimentov pre úlohu rozmiestnenia veterných turbín. Výsledky sú vyhodnotené pre konfigurácie s 5, 10 a 20 turbínami, ktorým zodpovedajú dimenzie 10, 20 a 40. Porovnané boli algoritmy Random Search, PyBADs, TuRBO a jednotlivé konfigurácie knižnice PySOT.

Hodnotenie bolo založené na dosiahnutej hodnote výkonu, konvergenčnom priebehu, počte evaluácií potrebných na dosiahnutie najlepšieho riešenia a časovej náročnosti experimentov. Na rozdiel od úlohy rozmiestnenia environmentálnych senzorov išlo o spojitú optimalizačnú úlohu, pri ktorej bolo cieľom maximalizovať energetický výkon veternej farmy.

4.2.1 Kvalita dosiahnutých riešení

V prvej časti vyhodnotenia je porovnaná finálna kvalita riešení podľa priemernej najlepšej hodnoty výkonu dosiahnutej počas opakovaných behov. Porovnanie je uvedené samostatne pre konfigurácie s 5, 10 a 20 turbínami. Súčasťou vyhodnotenia je aj absolútny a relatívny rozdiel oproti najlepšiemu algoritmu v danej konfigurácii.

Tab. 4: Poradie algoritmov podľa priemerného najlepšieho výkonu pri 5 turbínach

Algoritmus	Priemerný výkon [GW]	Rozdiel oproti najlepšiemu [GW]	Relatívny rozdiel [%]
PySOT-POLY-SOP	74,492	0,000	0,000
PySOT-POLY-SRBF	74,096	0,396	0,532
PySOT-RBF-SOP	73,926	0,567	0,761
PySOT-POLY-DYCORS	73,852	0,64	0,859
PySOT-RBF-SRBF	73,737	0,756	1,014
PySOT-RBF-DYCORS	73,707	0,785	1,054
PyBADs	73,640	0,853	1,144
TuRBO	73,564	0,928	1,246
PySOT-GP-LCB	73,230	1,262	1,694
PySOT-GP-EI	73,013	1,479	1,985
Random Search	72,601	1,891	2,539

Pri konfigurácii s 5 turbínami bola najvyššia priemerná hodnota výkonu dosiahnutá konfiguráciou knižnice PySOT s polynomiálnym náhradným modelom a výberovou stratégiou SOP. Druhá najvyššia hodnota bola zaznamenaná pri konfigurácii s polynomiálnym náhradným modelom a stratégiou SRBF, ktorá za najlepšou konfiguráciou zaostala o 0,396 GW, teda o 0,532 %. V hornom pásme sa nachádzala aj konfigurácia s náhradným modelom RBF a stratégiou SOP, pri ktorej bol relatívny rozdiel 0,761 %.

Stredné pásmo výsledkov pri 5 turbínach tvorili algoritmus PyBADs, TuRBO, a konfigurácie knižnice PySOT využívajúce stratégie DYCORs a SRBF pri polynomiálnom modeli alebo modeli RBF. Ich relatívny rozdiel oproti najlepšiemu algoritmu sa pohyboval približne od 0,859 % do 1,246 %.

Najnižšie hodnoty pri konfigurácii s 5 turbínami bola zaznamenané pri referenčnej metóde Random Search a konfiguráciách knižnice PySOT založené na Gaussovskom procese a akvizíčných funkciách LCB a EI.

Tab. 5: Poradie algoritmov podľa priemerného najlepšieho výkonu pri 10 turbínach

Algoritmus	Priemerný výkon [GW]	Rozdiel oproti najlepšiemu [GW]	Relatívny rozdiel [%]
PySOT-POLY-SOP	147,634	0,000	0,000
PySOT-RBF-SOP	146,380	1,255	0,850
PyBADs	146,059	1,575	1,067
PySOT-POLY-SRBF	145,596	2,039	1,381
PySOT-POLY-DYCORs	145,563	2,071	1,403
PySOT-RBF-SRBF	145,550	2,084	1,412
PySOT-RBF-DYCORs	145,520	2,114	1,432
TuRBO	145,055	2,579	1,747
PySOT-GP-LCB	144,048	3,587	2,429
Random Search	143,534	4,101	2,778
PySOT-GP-EI	143,080	4,555	3,085

Pri konfigurácii s 10 turbínami bola opäť najvyššia priemerná hodnota výkonu dosiahnutá konfiguráciou knižnice PySOT s polynomiálnym náhradným modelom a výberovou stratégiou SOP. Na druhom mieste sa umiestnila konfigurácia s náhradným modelom radiálnych bázových funkcií a stratégiou SOP s hodnotou 146,380 GW, čo predstavovalo rozdiel 1,255 GW, teda 0,850 %. Tretia najvyššia hodnota bola dosiahnutá algoritmom PyBADs, pri ktorom bol relatívny rozdiel oproti najlepšej konfigurácii 1,067 %.

Stredné pásmo pri 10 turbínach tvorili konfigurácie knižnice PySOT s polynomiálnym a RBF náhradným modelom v kombinácii so stratégiami SRBF a DYCORs. Ich priemerné výkony sa pohybovali od 145,520 GW do 145,596 GW a relatívny rozdiel oproti najlepšiemu algoritmu bol približne od 1,381 % do 1,432 %. Algoritmus TuRBO dosiahol 145,055 GW, čo

predstavovalo relatívny rozdiel 1,747 %. V tejto konfigurácii sa teda najlepšie výsledky opäť sústredili najmä pri konfiguráciách knižnice PySOT využívajúcich stratégiu SOP a pri algoritme PyBADs.

Najnižšia hodnota pri konfigurácii s 10 turbínami bola zaznamenaná pri konfigurácii knižnice PySOT s Gaussovským procesom a akvizičnou funkciou EI. Dosiahnutá hodnota bola 143,080 GW, čo predstavovalo rozdiel 4,555 GW, teda 3,085 % oproti najlepšiemu algoritmu. Referenčná metóda Random Search dosiahla hodnotu 143,534 GW a jej relatívny rozdiel bol 2,778 %. Konfigurácia s Gaussovským procesom a akvizičnou funkciou LCB dosiahla hodnotu 144,048 GW, teda lepší výsledok než Random Search a konfigurácia s očakávaným zlepšením.

Tab. 6: Poradie algoritmov podľa priemerného najlepšieho výkonu pri 20 turbínach

Algoritmus	Priemerný výkon [GW]	Rozdiel oproti najlepšiemu [GW]	Relatívny rozdiel [%]
PySOT-POLY-SOP	292,801	0,000	0,000
PyBADs	292,017	0,784	0,268
PySOT-RBF-SOP	291,979	0,822	0,281
PySOT-RBF-DYCORDS	290,947	1,854	0,633
PySOT-POLY-DYCORDS	290,782	2,019	0,690
PySOT-POLY-SRBF	290,439	2,362	0,807
TuRBO	289,966	2,836	0,968
PySOT-RBF-SRBF	289,690	3,112	1,063
Random Search	289,066	3,735	1,276
PySOT-GP-LCB	287,277	5,525	1,887
PySOT-GP-EI	286,375	6,427	2,195

Pri konfigurácii s 20 turbínami bola najvyššia priemerná hodnota výkonu opäť dosiahnutá konfiguráciou knižnice PySOT s polynomiálnym náhradným modelom a výberovou stratégiou SOP. Druhá najvyššia hodnota bola zaznamenaná pri algoritme PyBADs, ktorý dosiahol 292,017 GW. Rozdiel oproti najlepšiemu algoritmu bol 0,784 GW, teda 0,268 %. Veľmi blízku hodnotu dosiahla aj konfigurácia knižnice PySOT s náhradným modelom RBF a stratégiou SOP, pri ktorej bol relatívny rozdiel 0,281 %.

Stredné pásmo pri 20 turbínach tvorili konfigurácie knižnice PySOT s výberovými stratégiami DYCORDS a SRBF pri oboch typoch náhradného modelu, spolu s algoritmom TuRBO. Ich relatívny rozdiel oproti najlepšej konfigurácii sa pohyboval približne od 0,633 % do 1,063 %. Algoritmus TuRBO dosiahol hodnotu 289,966 GW, čo predstavovalo rozdiel 2,836 GW, teda 0,968 %. Referenčná metóda Random Search dosiahla 289,066 GW a jej relatívny rozdiel bol 1,276 %.

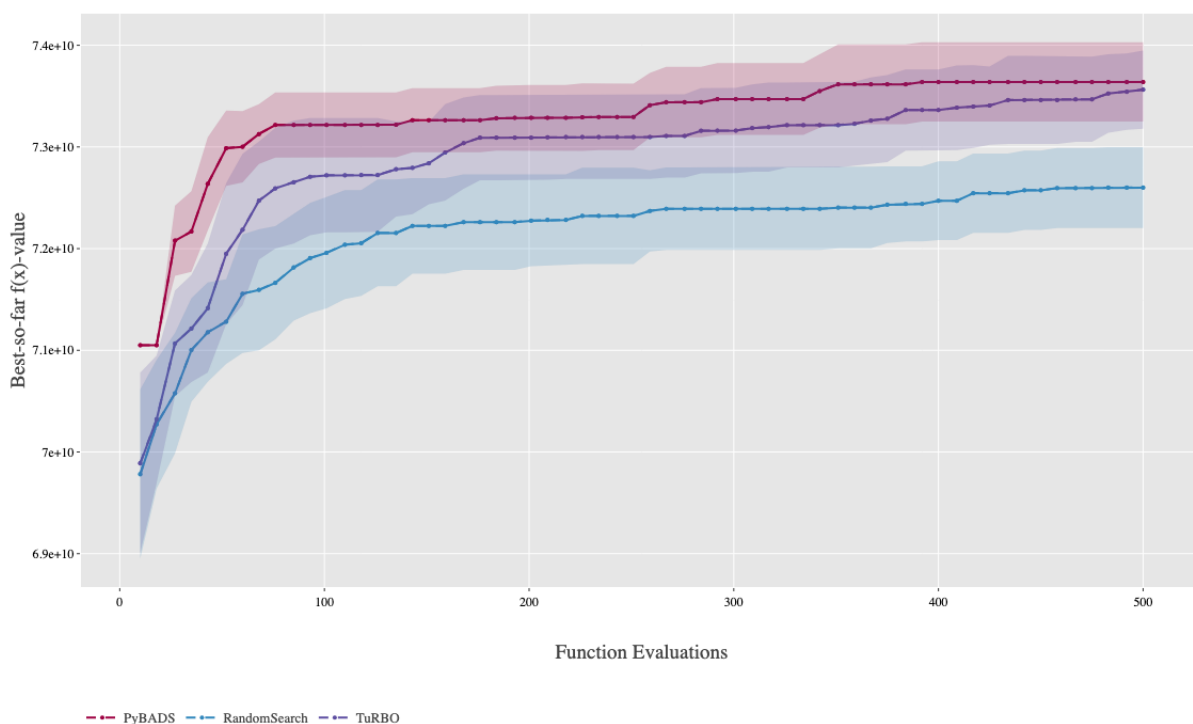
Najnižšie hodnoty pri konfigurácii s 20 turbínami boli zaznamenané pri konfiguráciách knižnice PySOT založených na Gaussovskom procese. Pri akvizičnej funkcii LCB bola dosiahnutá hodnota 287,277 GW, čo predstavovalo relatívny rozdiel 1,887 %. Pri akvizičnej funkcii EI bola dosiahnutá najnižšia hodnota celej tabuľky, konkrétne 286,375 GW, teda rozdiel 6,427 GW a relatívny rozdiel 2,195 % oproti najlepšej konfigurácii.

4.2.2 Konvergenčný priebeh algoritmov

Konvergenčné priebehy algoritmov boli vyhodnotené pomocou nástroja IOHanalyzer, do ktorého boli importované pripravené CSV výstupy obsahujúce hodnotu najlepšieho aktuálne dosiahnutého riešenia po jednotlivých evaluáciách. Tento spôsob spracovania bol zvolený z dôvodu veľkého objemu výstupných dát a potreby jednotného zobrazenia priebehov naprieč algoritmi, behmi a konfiguráciami počtu turbín.

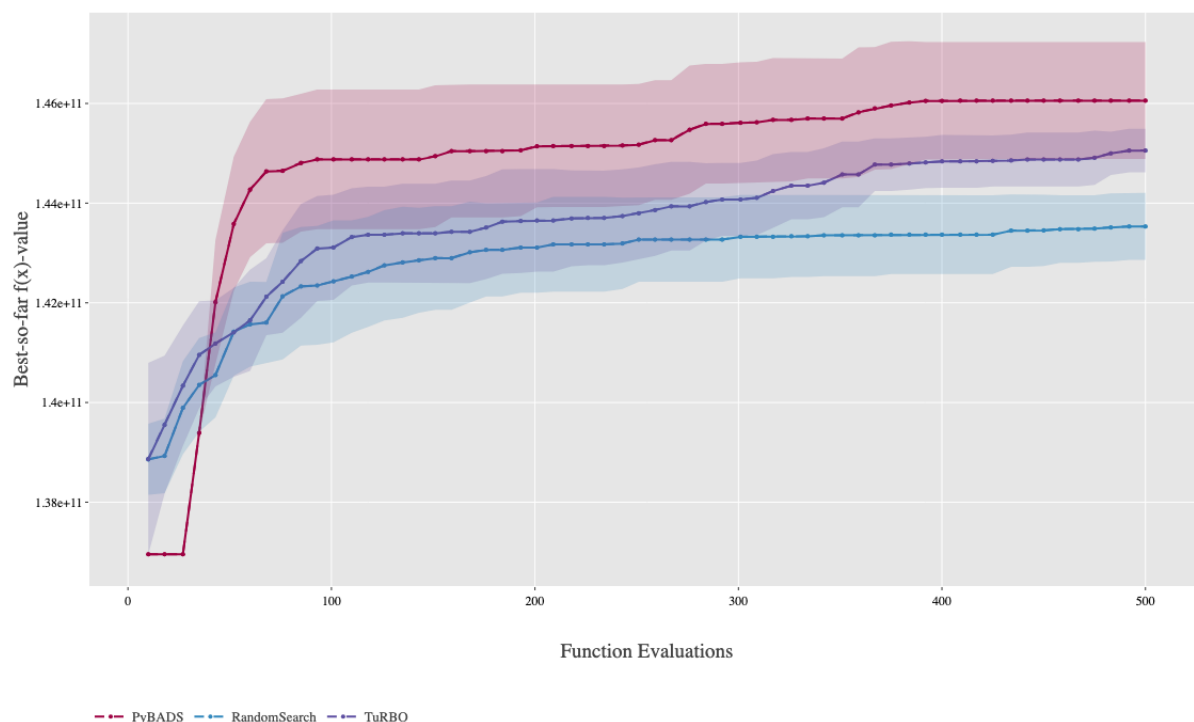
V grafoch je plnou čiarou zobrazený priemerný priebeh najlepšej aktuálne dosiahnutej hodnoty výkonu. Tieňované pásma predstavujú smerodajnú odchýlku medzi opakovanými behmi a slúži na orientačné posúdenie variability výsledkov.

Ako prvé boli porovnané konvergenčné priebehy algoritmov Random Search, PyBADs a TuRBO. Krivky zobrazujú priemerný priebeh najlepšej aktuálne dosiahnutej hodnoty výkonu počas 500 evaluácií.



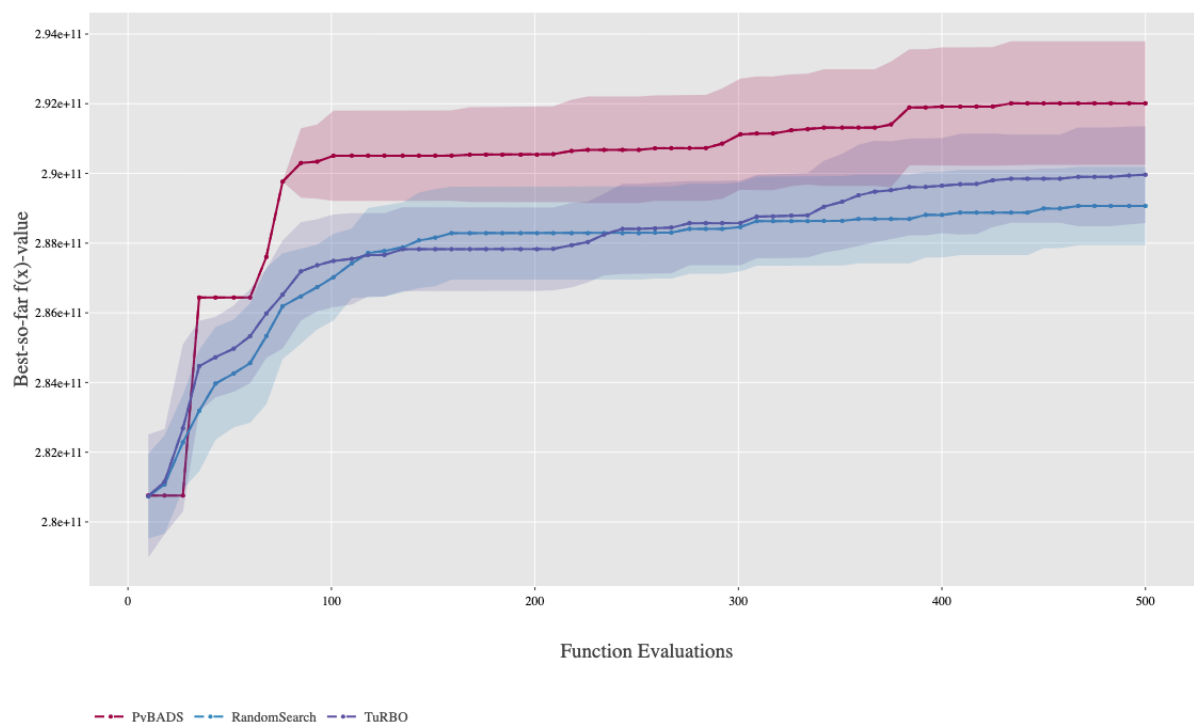
Obr. 15: Konvergenčný priebeh algoritmov Random Search, PyBADs a TuRBO pri konfigurácii s 5 turbínami.

Pri konfigurácii s 5 turbínami dosahoval PyBADs, počas väčšej časti optimalizácie, vyššie priemerné hodnoty najlepšieho aktuálne nájdeného výkonu než TuRBO a Random Search.



Obr. 16: Konvergenčný priebeh algoritmov Random Search, PyBADS a TuRBO pri konfigurácii s 10 turbínami.

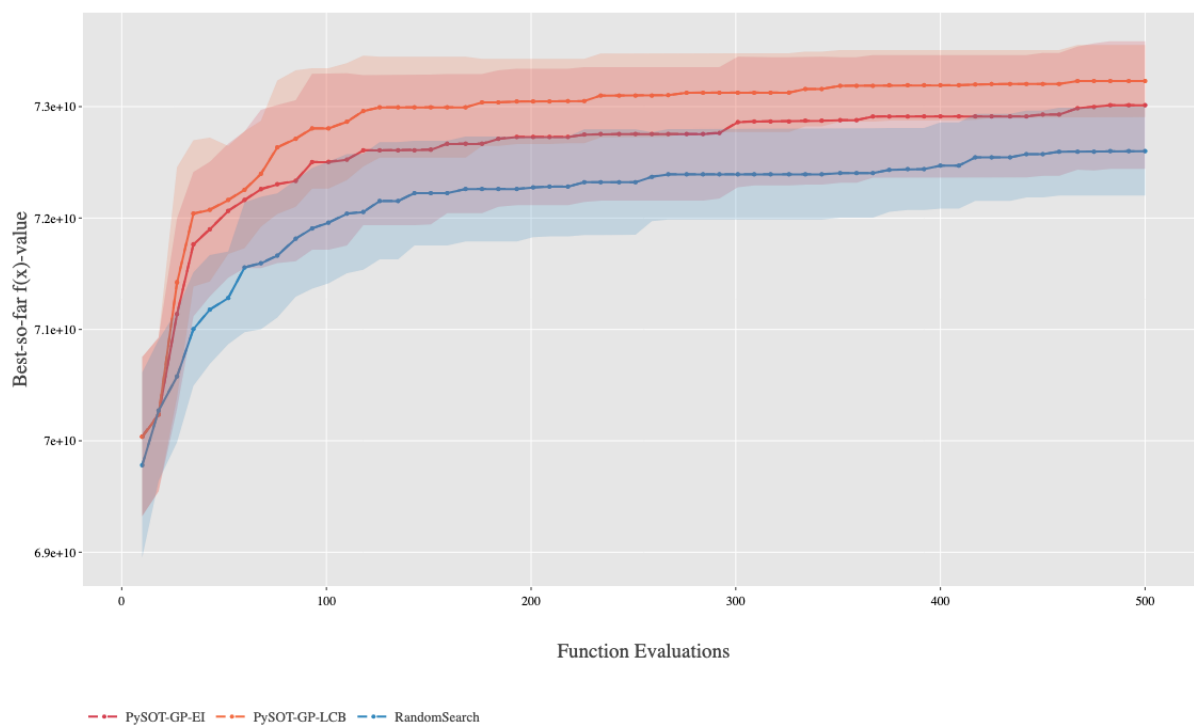
Pri konfigurácii s 10 turbínami dosahoval PyBADS algoritmus, počas väčšiny optimalizácie, najvyššie priemerné hodnoty najlepšieho aktuálne nájdeného výkonu. TuRBO dosahoval vyššie hodnoty než Random Search, ale počas priebehu optimalizácie zostával pod algoritmom PyBADS. Random Search dosahoval z porovnáwanej trojice najnižšie priemerné hodnoty.



Obr. 17: Konvergenčný priebeh algoritmov Random Search, PyBADS a TuRBO pri konfigurácii s 20 turbínami.

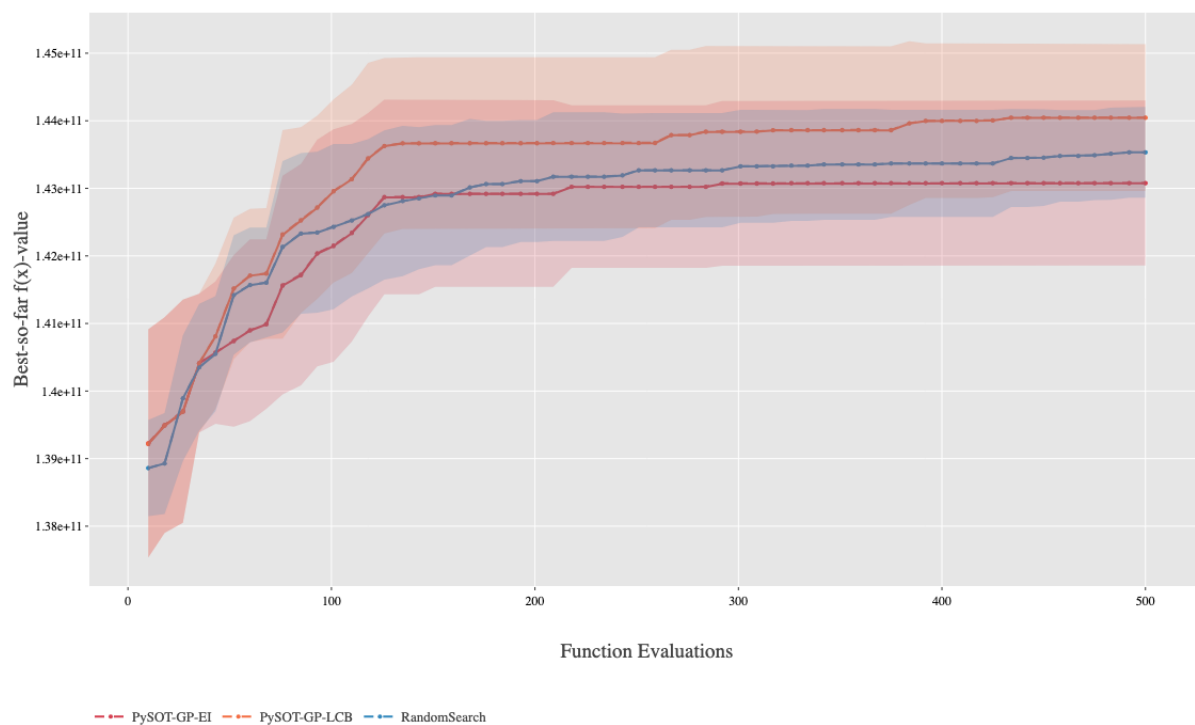
Pri konfigurácii s 20 turbínami dosahoval PyBADS opäť najvyššie priemerné hodnoty najlepšieho aktuálne nájdeného výkonu. TuRBO sa počas väčšiny priebehu nachádzal medzi algoritmami PyBADS a Random Search. Random Search dosahoval nižšie hodnoty než PyBADS a TuRBO, pričom rozdiel medzi TuRBO a Random Search bol v tejto konfigurácii menší než rozdiel medzi PyBADS a zvyšnými dvoma algoritmami.

Ďalšia skupina grafov porovnáva konfigurácie PySOT založené na Gaussovskom procese. Do porovnania boli zaradené akvizičné funkcie EI a LCB, pričom ako referenčná metóda je opäť uvedený Random Search. Cieľom porovnania je sledovať priebeh optimalizácie pri použití pravdepodobnostného náhradného modelu a akvizičných funkcií.



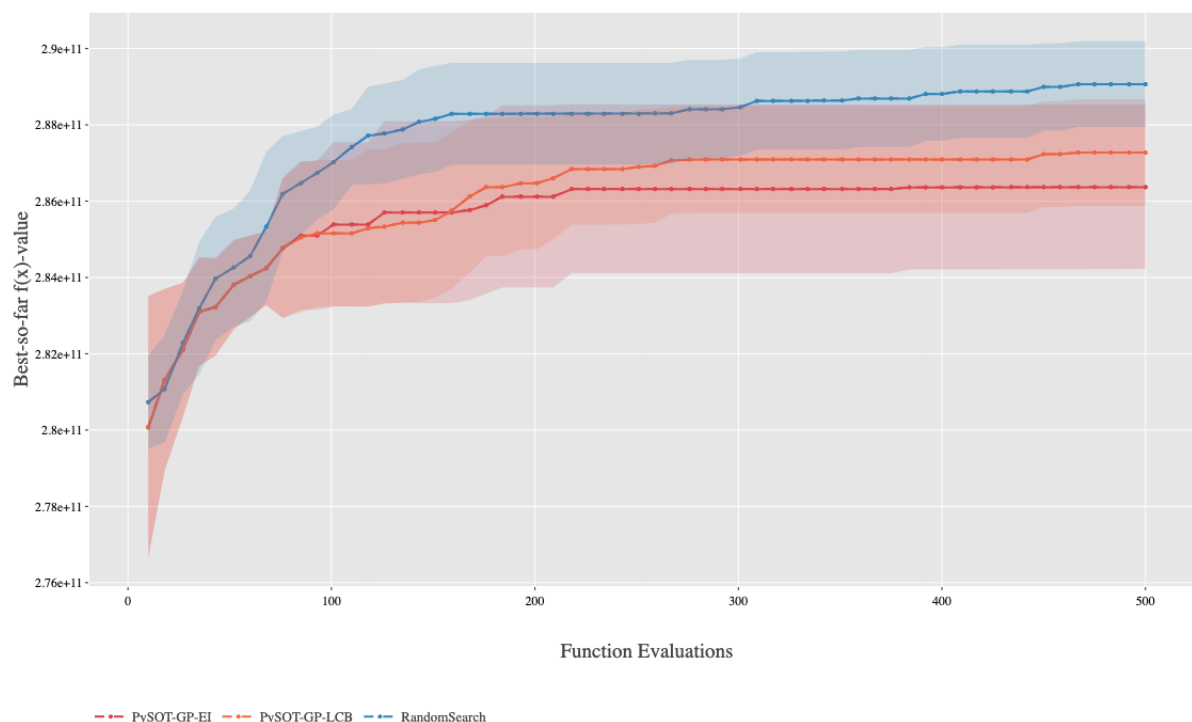
Obr. 18: Konvergenčný priebeh konfigurácií PySOT s Gaussovským procesom a akvizičnými funkciami EI a LCB pri konfigurácii s 5 turbínami.

Pri konfigurácii s 5 turbínami dosahovala konfigurácia PySOT, využívajúca Gaussovský proces a akvizičnú funkciu dolnej hranice spoľahlivosti, vyššie priemerné hodnoty než konfigurácia využívajúca očakávané zlepšenie aj referenčná metóda Random Search.



Obr. 19: Konvergenčný priebeh konfigurácií PySOT s Gaussovským procesom a akvizičnými funkciami EI a LCB pri konfigurácii s 10 turbínami.

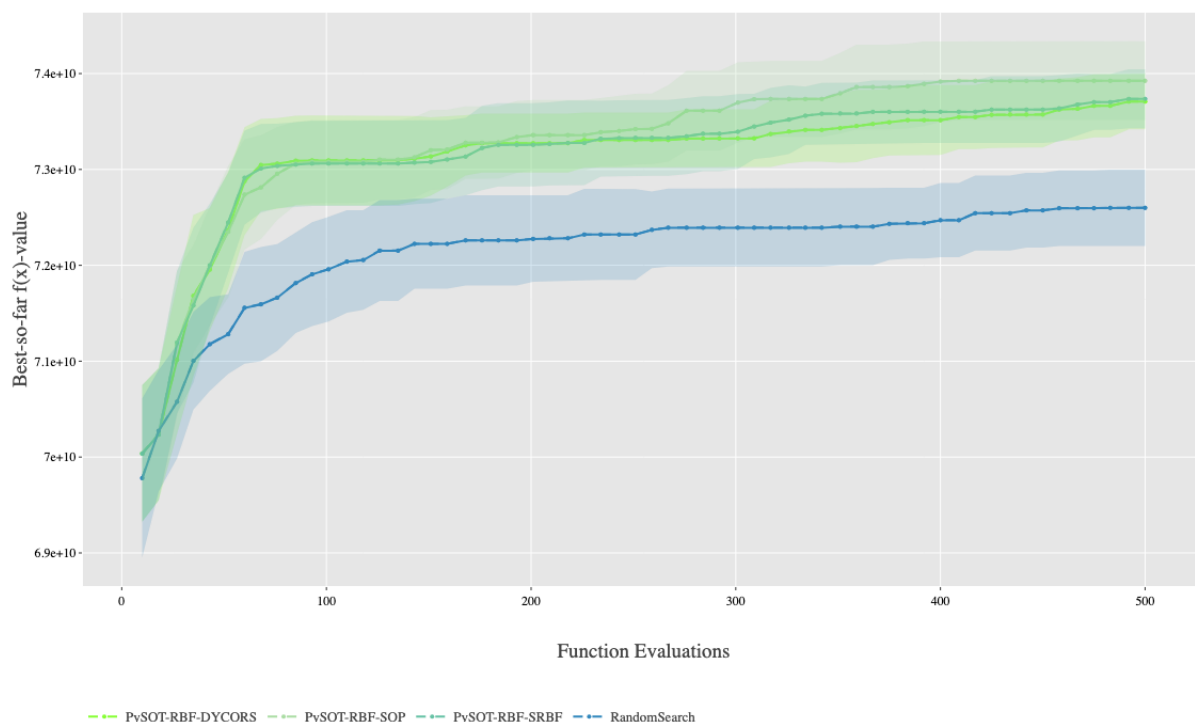
Pri konfigurácii s 10 turbínami dosahovala konfigurácia PySOT, s Gaussovským procesom a dolnou hranicou spoľahlivosti, vyššie priemerné hodnoty než konfigurácia s očakávaným zlepšením. Random Search sa v tomto porovnaní nachádzal medzi oboma konfiguráciami.



Obr. 20: Konvergenčný priebeh konfigurácií PySOT s Gaussovským procesom a akvizičnými funkciami EI a LCB pri konfigurácii s 20 turbínami.

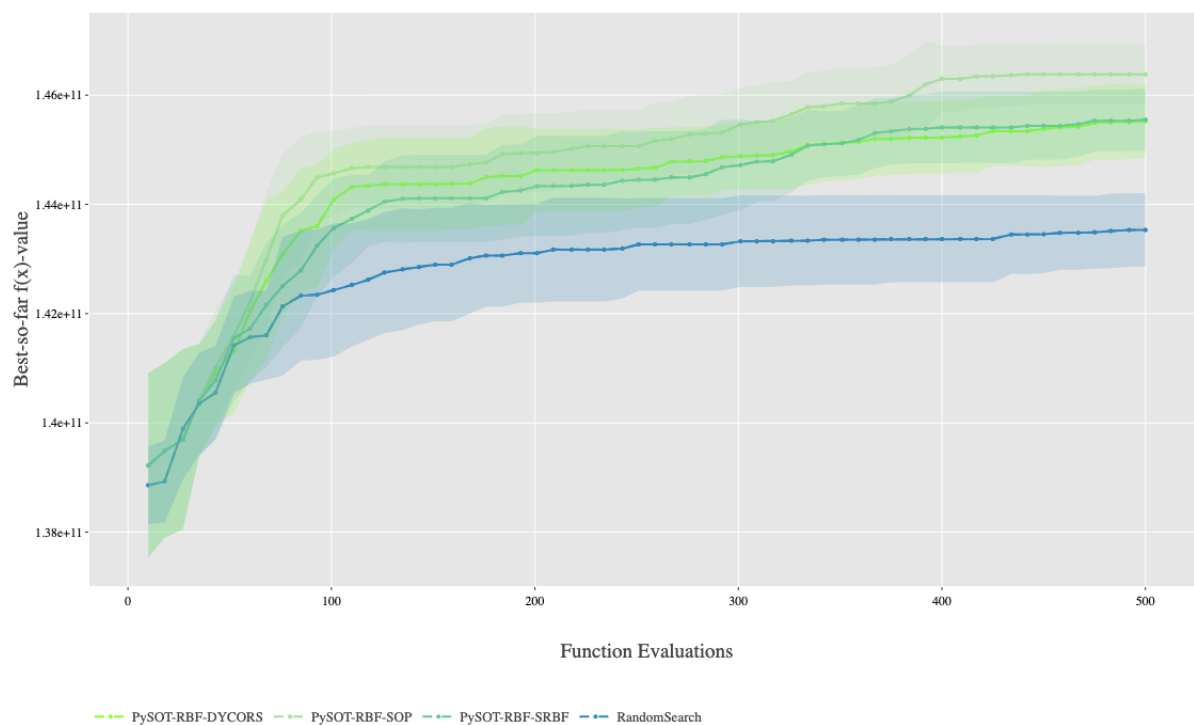
Pri konfigurácii s 20 turbínami dosahovala konfigurácia PySOT s Gaussovským procesom a dolnou hranicou spoľahlivosti vyššie hodnoty než konfigurácia s očakávaným zlepšením. Random Search však v tejto skupine dosiahol vyššie hodnoty než konfigurácia s očakávaným zlepšením a približoval sa alebo prekonával konfiguráciu s dolnou hranicou spoľahlivosti. Tento výsledok ukazuje, že samotné použitie Gaussovského procesu a akvizičnej funkcie nezaručilo v tejto úlohe najlepší konvergenčný priebeh.

Ďalšia skupina grafov porovnáva konfigurácie PySOT založené na náhradnom modeli RBF. Porovnané boli stratégie DYCORs, SRBF a SOP, pričom ako referenčná metóda bol opäť použitý Random Search. Cieľom je porovnať, ako sa pri rovnakom type náhradného modelu prejavili rozdielne stratégie výberu kandidátnych riešení.



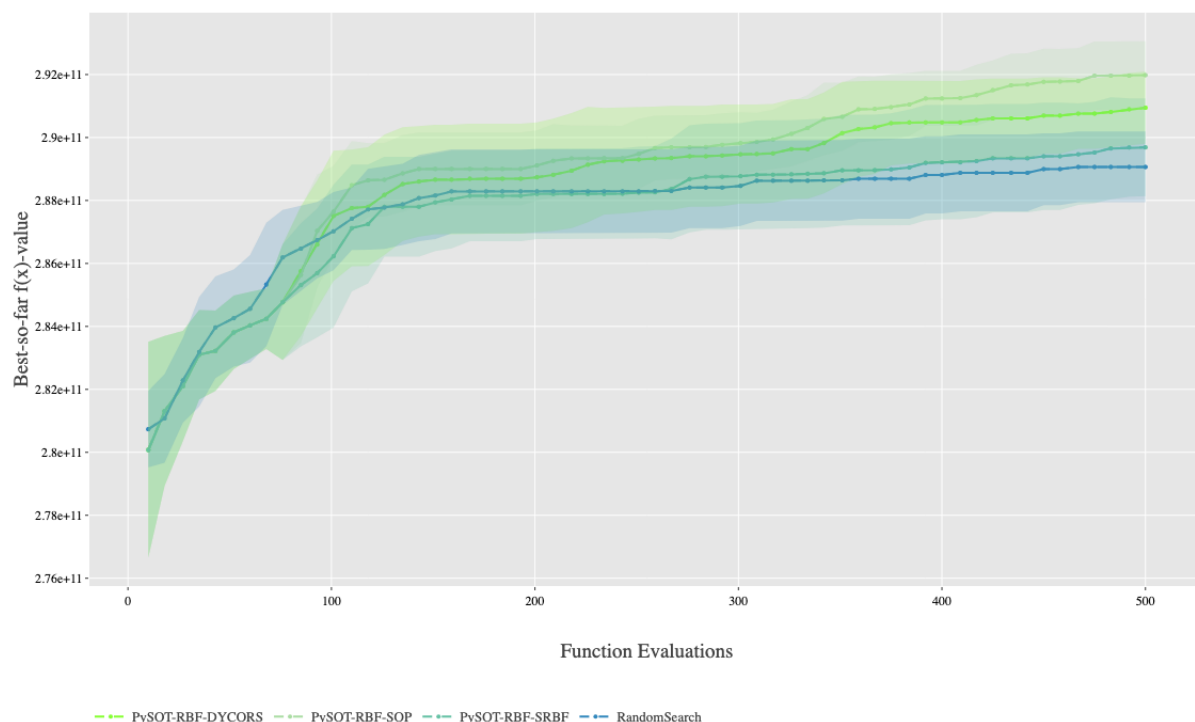
Obr. 21: Konvergenčný priebeh konfigurácií PySOT s RBF náhradným modelom a stratégiemi DYCORDS, SRBF a SOP pri konfigurácii s 5 turbínami.

Pri konfigurácii s 5 turbínami dosahovali všetky konfigurácie PySOT využívajúce RBF náhradný model vyššie hodnoty než Random Search. Najvyššie priemerné hodnoty z tejto skupiny dosahovala konfigurácia so stratégiou SOP. Konfigurácie so stratégiami DYCORDS a SRBF dosahovali navzájom podobné hodnoty, pričom ich priebeh bol počas optimalizácie stabilne nad referenčnou metódou Random Search.



Obr. 22: Konvergenčný priebeh konfigurácií PySOT s RBF náhradným modelom a stratégiami DYCORDS, SRBF a SOP pri konfigurácii s 10 turbínami.

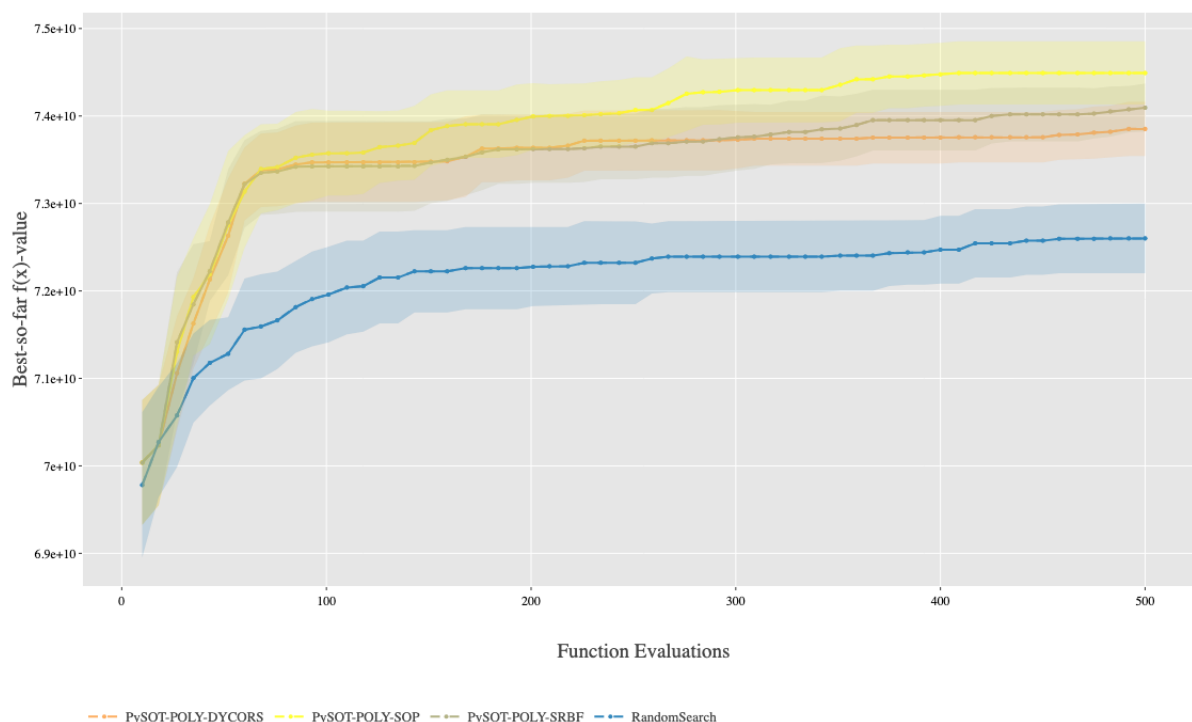
Pri konfigurácii s 10 turbínami dosahovala najvyššie priemerné hodnoty konfigurácia PySOT s RBF náhradným modelom a stratégiou SOP. Konfigurácie využívajúce stratégie DYCORDS a SRBF dosahovali nižšie, ale stále vyššie hodnoty než Random Search.



Obr. 23: Konvergenčný priebeh konfigurácií PySOT s RBF náhradným modelom a stratégiami DYCORDS, SRBF a SOP pri konfigurácii s 20 turbínami.

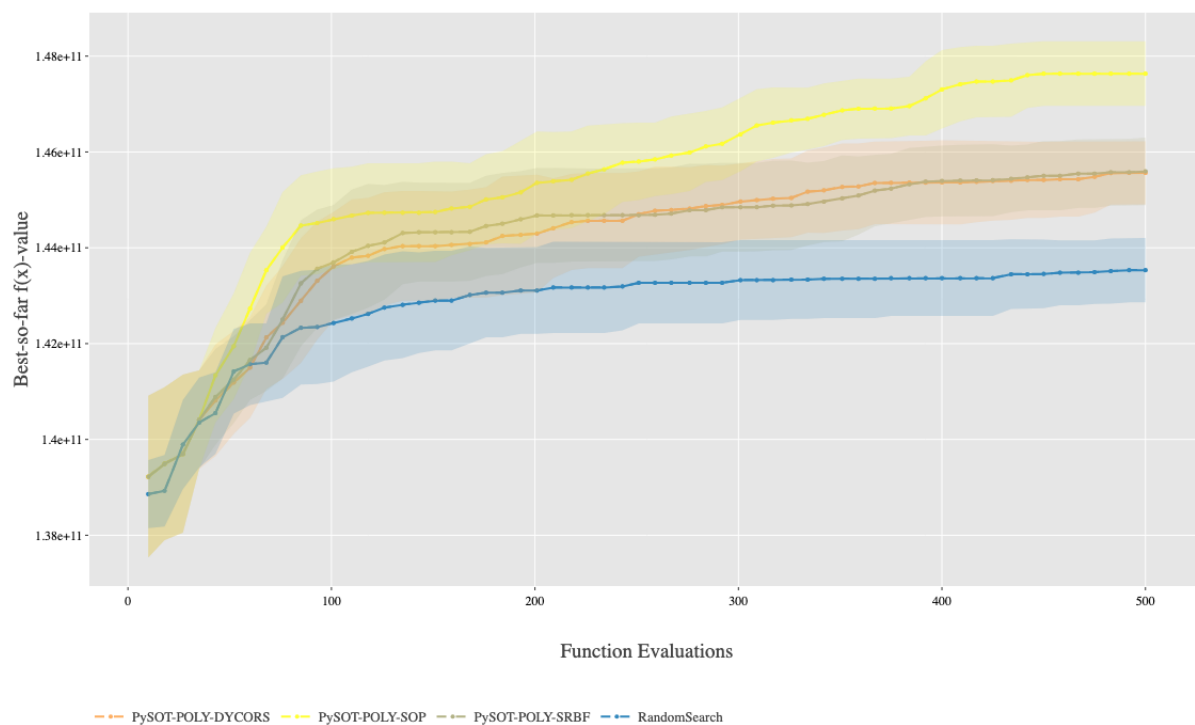
Pri konfigurácii s 20 turbínami dosahovala konfigurácia PySOT s RBF náhradným modelom a stratégiou SOP najvyššie hodnoty zo skupiny RBF stratégií. Konfigurácia so stratégiou DYCORDS dosahovala vyššie hodnoty než konfigurácia so stratégiou SRBF a referenčná metóda Random Search.

Ďalšia skupina grafov porovnáva konfigurácie PySOT založené na polynomiálnom náhradnom modeli. Porovnané boli stratégie DYCORDS, SRBF a SOP, pričom ako referenčná metóda bol opäť použitý Random Search. Porovnanie nadväzuje na predchádzajúcu skupinu RBF konfigurácií a umožňuje sledovať správanie rovnakých stratégií pri inom type náhradného modelu.



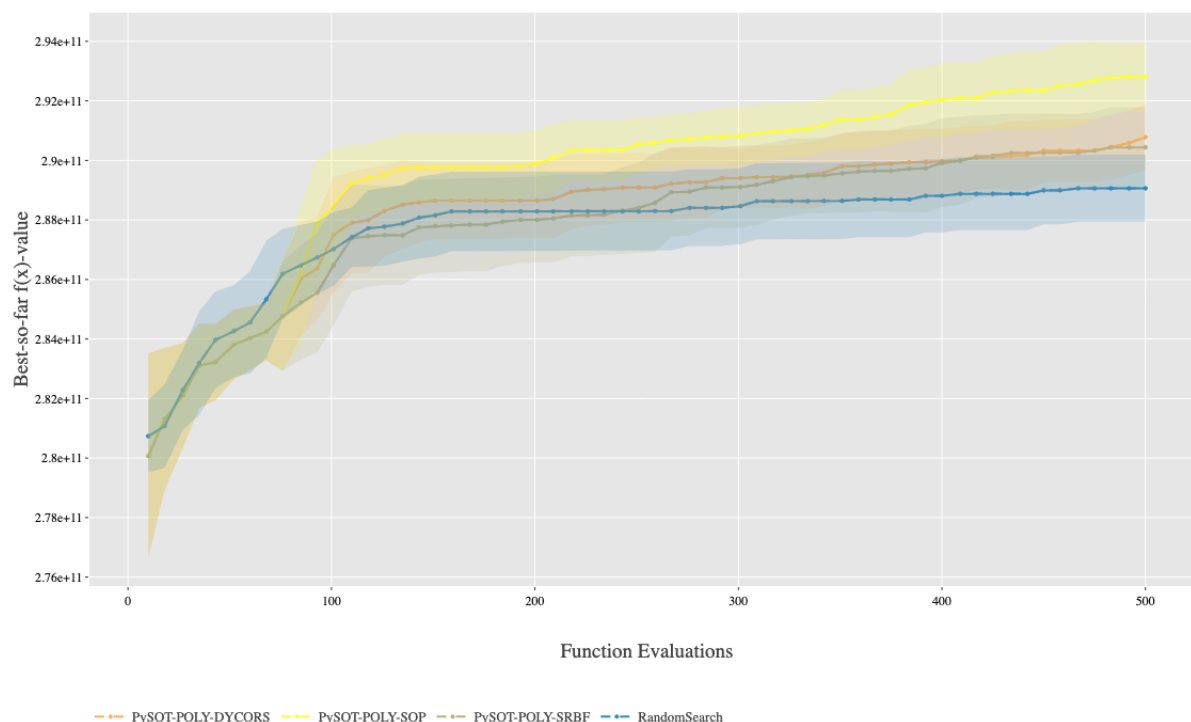
Obr. 24: Konvergenčný priebeh konfigurácií PySOT s polynomiálnym náhradným modelom a stratégiami DYCORDS, SRBF a SOP pri konfigurácii s 5 turbínami.

Pri konfigurácii s 5 turbínami dosahovali všetky konfigurácie PySOT využívajúce polynomiálny náhradný model vyššie priemerné hodnoty než Random Search. Najvyššie hodnoty dosahovala konfigurácia so stratégiou SOP, ktorá mala počas väčšiny priebehu optimalizácie najlepší konvergenčný priebeh z tejto skupiny. Konfigurácie so stratégiami DYCORDS a SRBF dosahovali nižšie, ale stále výrazne lepšie hodnoty než referenčná metóda Random Search.



Obr. 25: Konvergenčný priebeh konfigurácií PySOT s polynomiálnym náhradným modelom a stratégiami DYCORDS, SRBF a SOP pri konfigurácii s 10 turbínami.

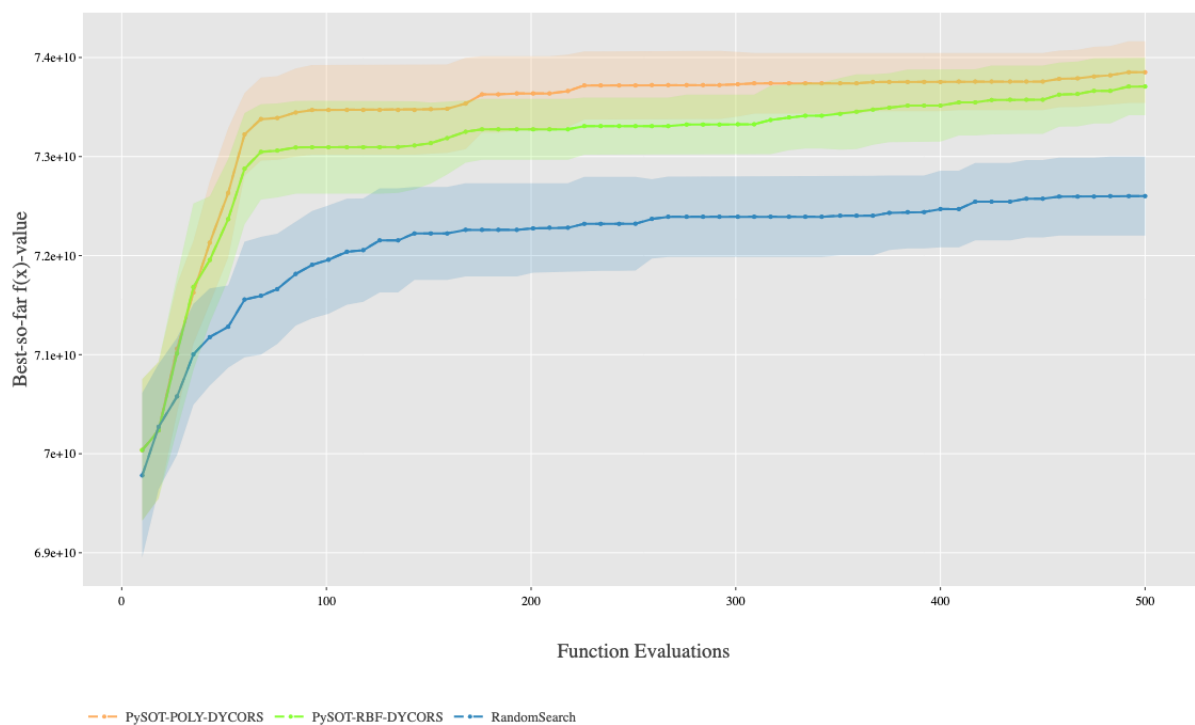
Pri konfigurácii s 10 turbínami dosahovala konfigurácia PySOT s polynomiálnym náhradným modelom a stratégiou SOP najvyššie priemerné hodnoty z porovnáwanej skupiny. Konfigurácie so stratégiami SRBF a DYCORDS dosahovali nižšie hodnoty, pričom obe zostávali nad referenčnou metódou Random Search. Rozdiel medzi stratégiou SOP a ostatnými polynomiálnymi konfiguráciami bol viditeľný najmä vo finálnej fáze optimalizácie.



Obr. 26: Konvergenčný priebeh konfigurácií PySOT s polynomiálnym náhradným modelom a stratégiami DYCORDS, SRBF a SOP pri konfigurácii s 20 turbínami.

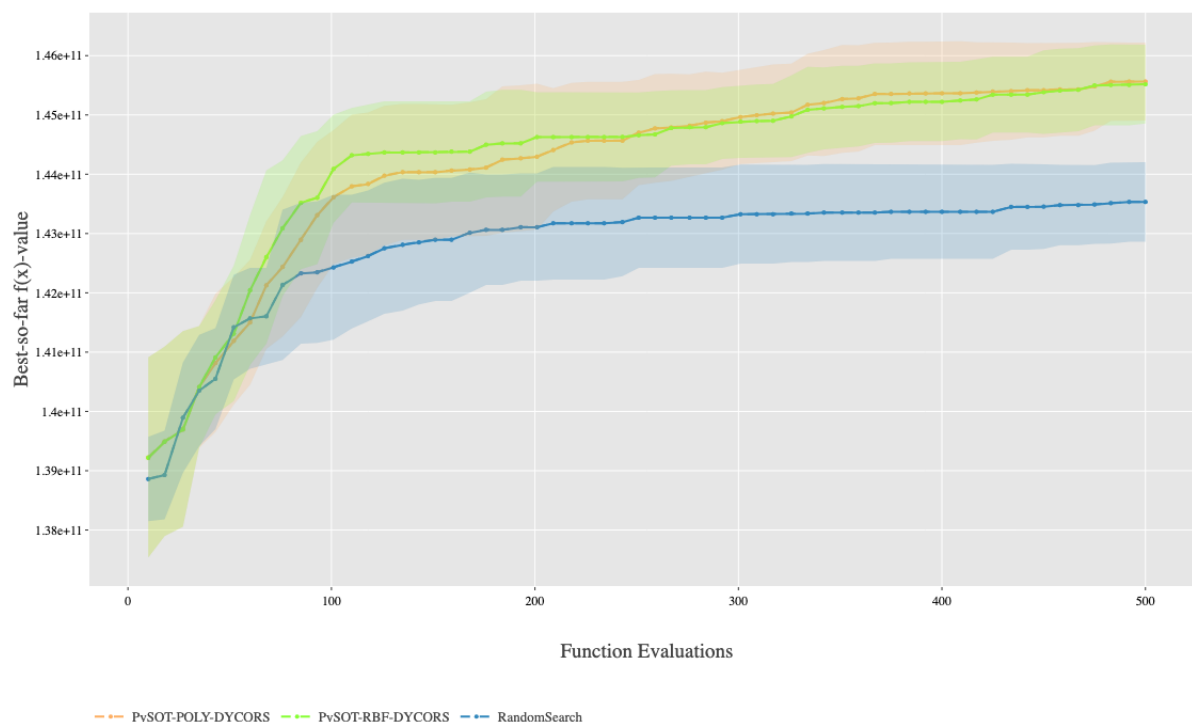
Pri konfigurácii s 20 turbínami dosiahla konfigurácia PySOT s polynomiálnym náhradným modelom a stratégiou SOP najvyšší priebeh spomedzi porovnávaných polynomiálnych konfigurácií. Konfigurácie so stratégiami DYCORDS a SRBF dosahovali nižšie hodnoty, ale stále sa nachádzali nad referenčnou metódou Random Search. Výsledok zodpovedá tabuľkovému porovnaniu kvality riešení, v ktorom rovnaká konfigurácia dosiahla najvyšší priemerný výkon aj pri tejto konfigurácii turbín.

Nasledujúca skupina grafov porovnáva stratégiu DYCORDS pri dvoch typoch náhradného modelu. Porovnané sú konfigurácie rámca PySOT využívajúce RBF a polynomiálny náhradný model, pričom ako referenčná metóda je opäť uvedený Random Search.



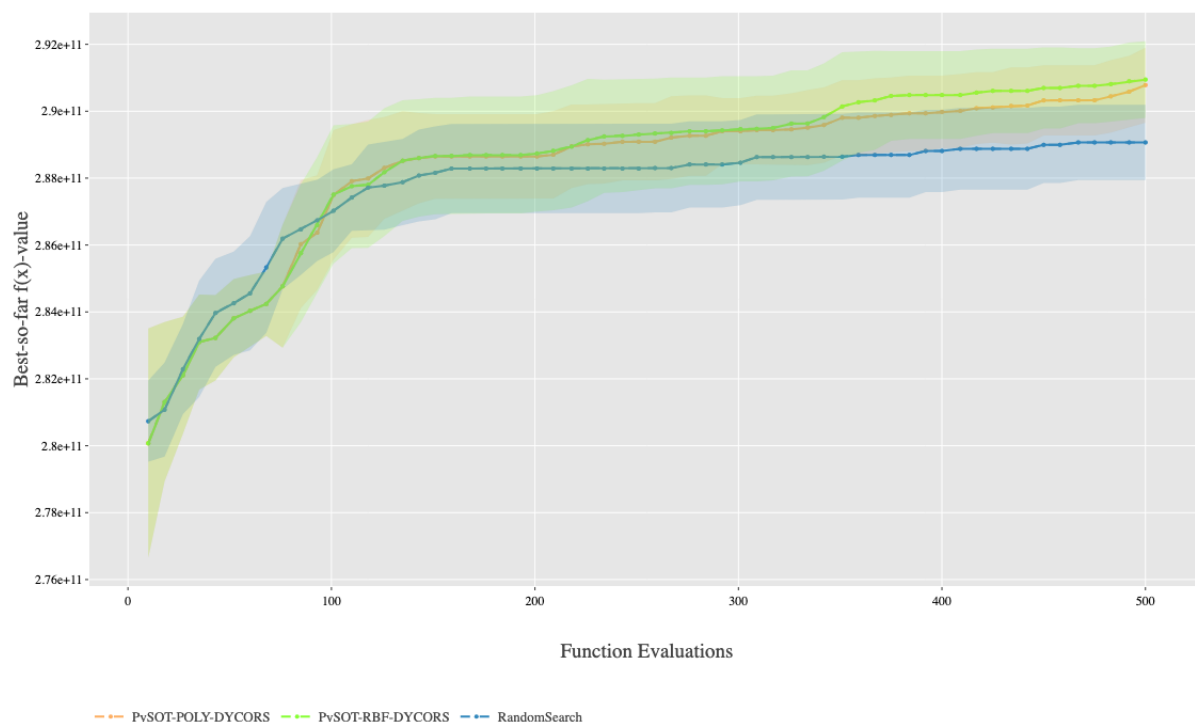
Obr. 27: Konvergenčný priebeh konfigurácií PySOT so stratégiou DYCORDS pri RBF a polynomiálnom náhradnom modeli pri konfigurácii s 5 turbínami.

Pri konfigurácii s 5 turbínami dosahovali obe konfigurácie so stratégiou DYCORDS vyššie priemerné hodnoty než Random Search. Konfigurácia využívajúca polynomiálny náhradný model dosahovala mierne vyššie hodnoty než konfigurácia využívajúca RBF model. Rozdiel medzi oboma náhradnými modelmi bol menší než rozdiel medzi modelovo riadenými konfiguráciami a referenčnou metódou Random Search.



Obr. 28: Konvergenčný priebeh konfigurácií PySOT so stratégiou DYCORS pri RBF a polynomiálnom náhradnom modeli pri konfigurácii s 10 turbínami.

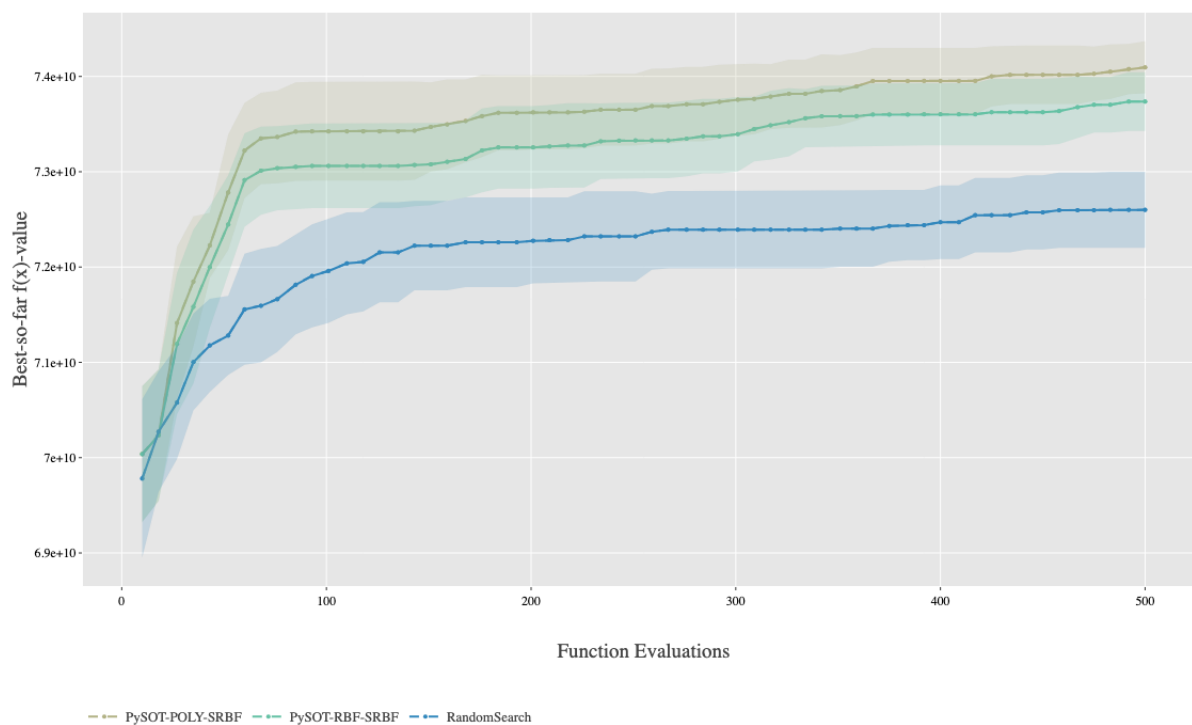
Pri konfigurácii s 10 turbínami dosahovali konfigurácie so stratégiou DYCORS a oboma typmi náhradného modelu podobné konvergenčné priebehy. Obe konfigurácie boli počas väčšiny optimalizácie nad hodnotami dosahovanými metódou Random Search. Rozdiel medzi RBF a polynomiálnym modelom bol v tejto skupine menej výrazný než rozdiel oproti referenčnej metóde.



Obr. 29: Konvergenčný priebeh konfigurácií PySOT so stratégiou DYCORDS pri RBF a polynomiálnom náhradnom modeli pri konfigurácii s 20 turbínami.

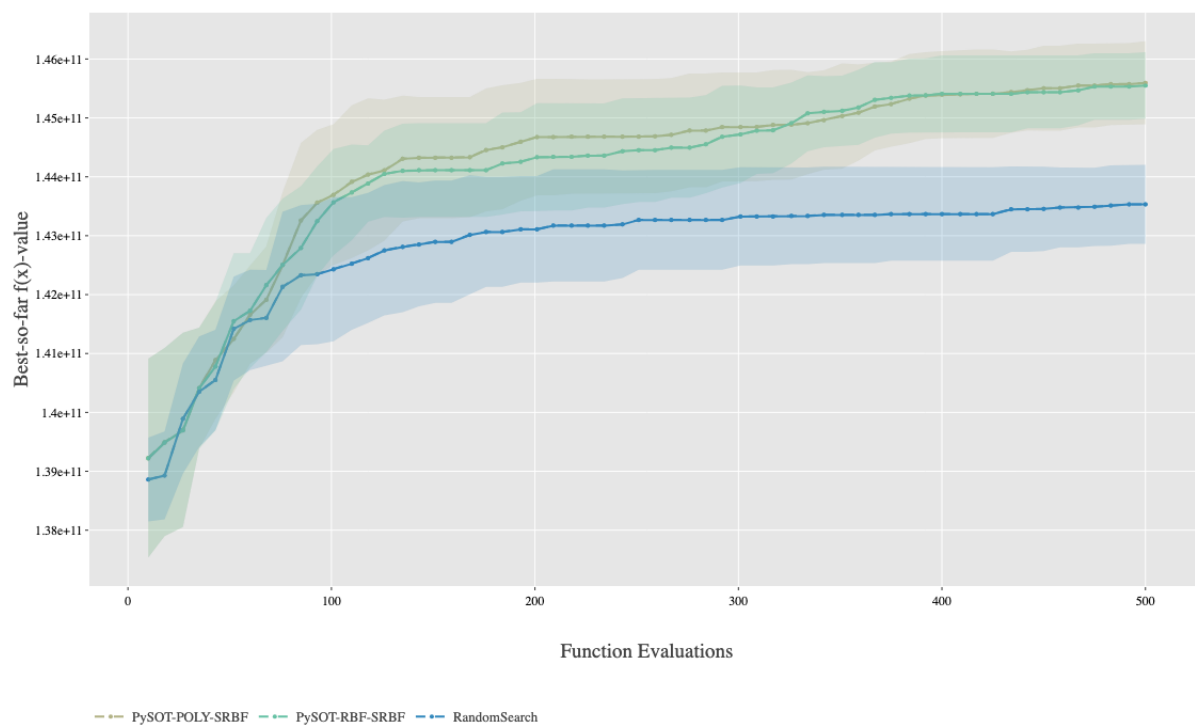
Pri konfigurácii s 20 turbínami dosahovala konfigurácia so stratégiou DYCORDS a RBF náhradným modelom mierne vyššie hodnoty než konfigurácia s polynomiálnym modelom a metóda Random Search.

Nasledujúca skupina grafov porovnáva stratégiu SRBF pri dvoch typoch náhradného modelu. Porovnané sú konfigurácie rámca PySOT využívajúce RBF a polynomiálny náhradný model, pričom ako referenčná metóda je uvedený Random Search. Porovnanie nadväzuje na predchádzajúcu skupinu s cieľom sledovať rozdiel medzi náhradnými modelmi pri rovnakej výberovej stratégii.



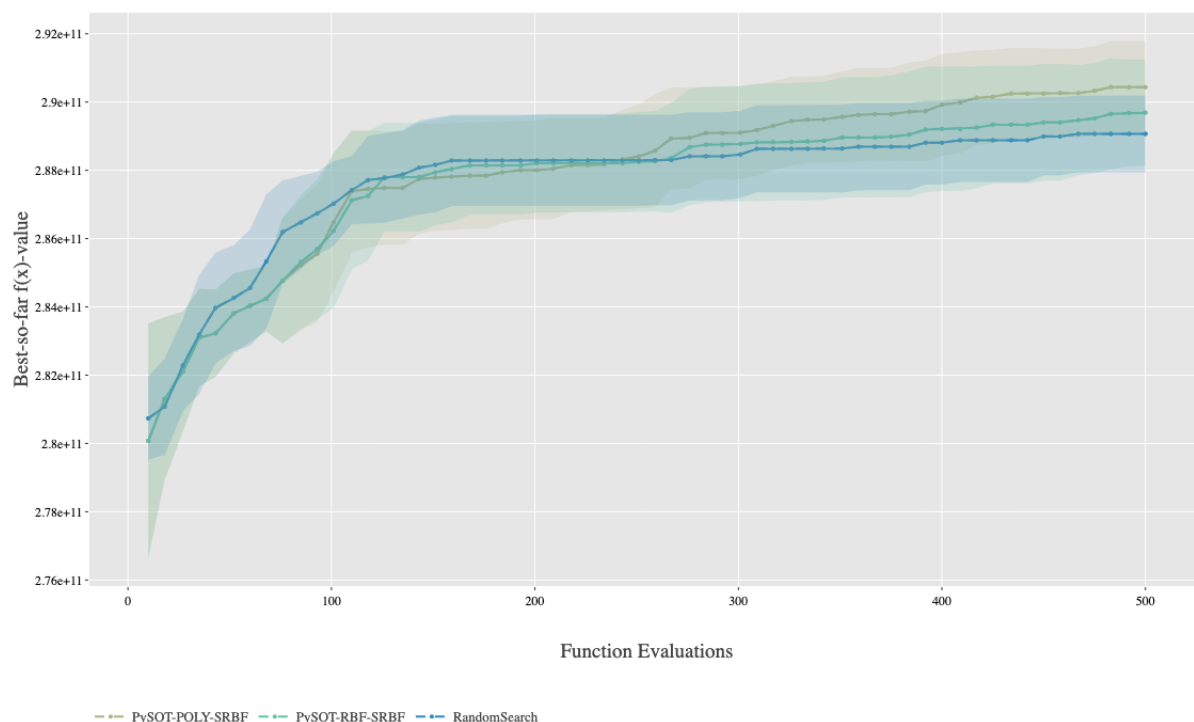
Obr. 30: Konvergenčný priebeh konfigurácií PySOT so stratégiou SRBF pri RBF a polynomiálnom náhradnom modeli pri konfigurácii s 5 turbínami.

Pri konfigurácii s 5 turbínami dosahovala konfigurácia so stratégiou SRBF a polynomiálnym náhradným modelom vyššie priemerné hodnoty než konfigurácia s RBF modelom. Obe konfigurácie boli počas optimalizácie nad referenčnou metódou Random Search.



Obr. 31: Konvergenčný priebeh konfigurácií PySOT so stratégiou SRBF pri RBF a polynomiálnom náhradnom modeli pri konfigurácii s 10 turbínami.

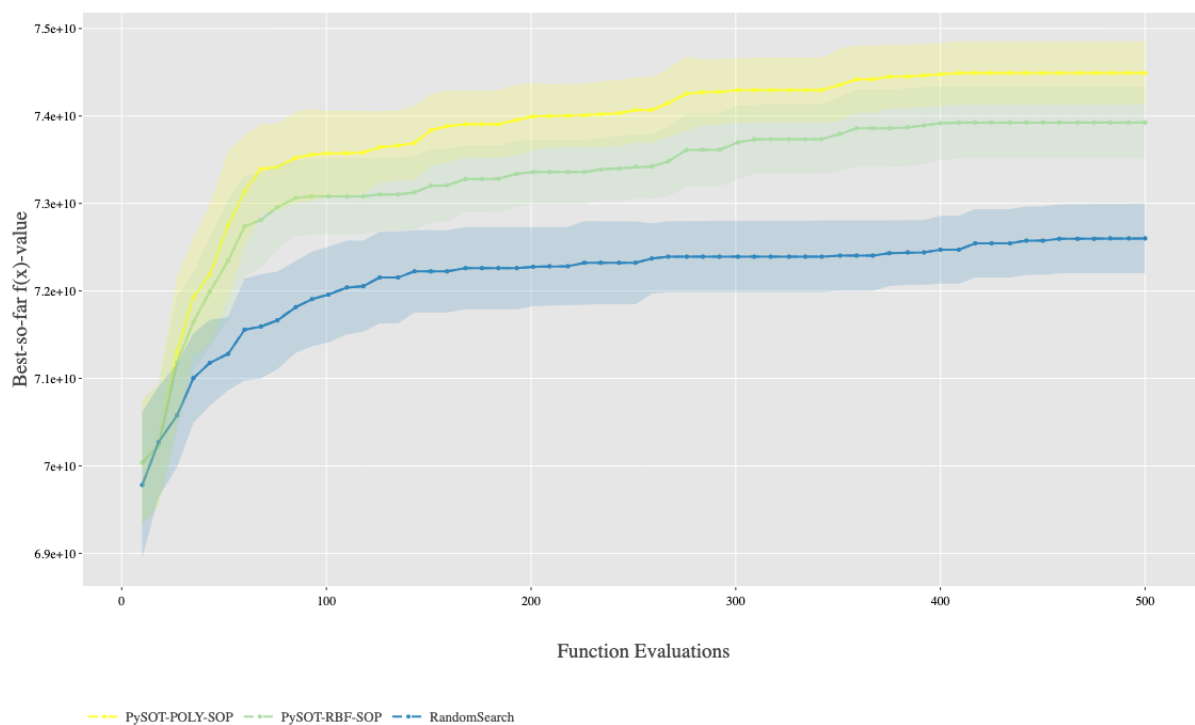
Pri konfigurácii s 10 turbínami dosahovali konfigurácie so stratégiou SRBF a oboma typmi náhradného modelu veľmi podobné hodnoty. Obe konfigurácie sa nachádzali nad priebehom metódy Random Search.



Obr. 32: Konvergenčný priebeh konfigurácií PySOT so stratégiou SRBF pri RBF a polynomiálnom náhradnom modeli pri konfigurácii s 20 turbínami.

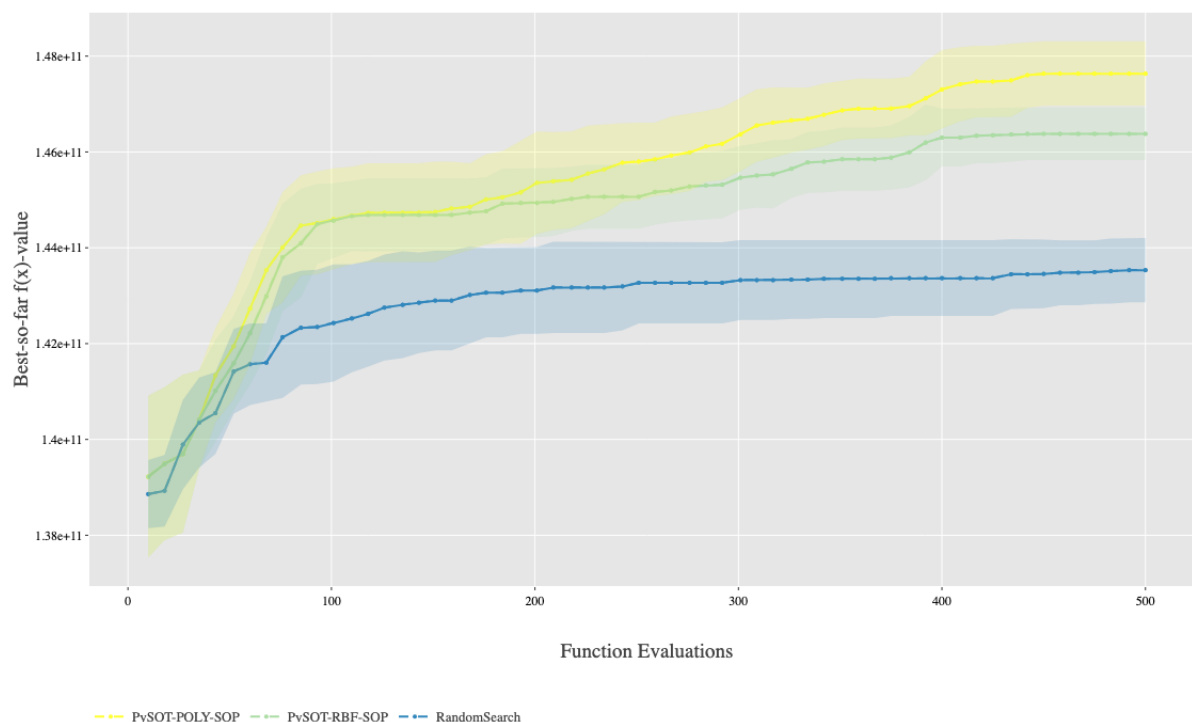
Pri konfigurácii s 20 turbínami dosahovala konfigurácia so stratégiou SRBF a polynomiálnym náhradným modelom vyššie hodnoty než konfigurácia s RBF modelom. Random Search sa nachádzal tesne za oboma konfiguráciami využívajúcimi stratégiu SRBF.

Posledná skupina v tejto časti porovnáva stratégiu SOP pri dvoch typoch náhradného modelu. Porovnané sú konfigurácie rámca PySOT využívajúce RBF a polynomiálny náhradný model, pričom ako referenčná metóda je uvedený Random Search. Táto skupina nadväzuje na predchádzajúce porovnania stratégií DYCORS a SRBF.



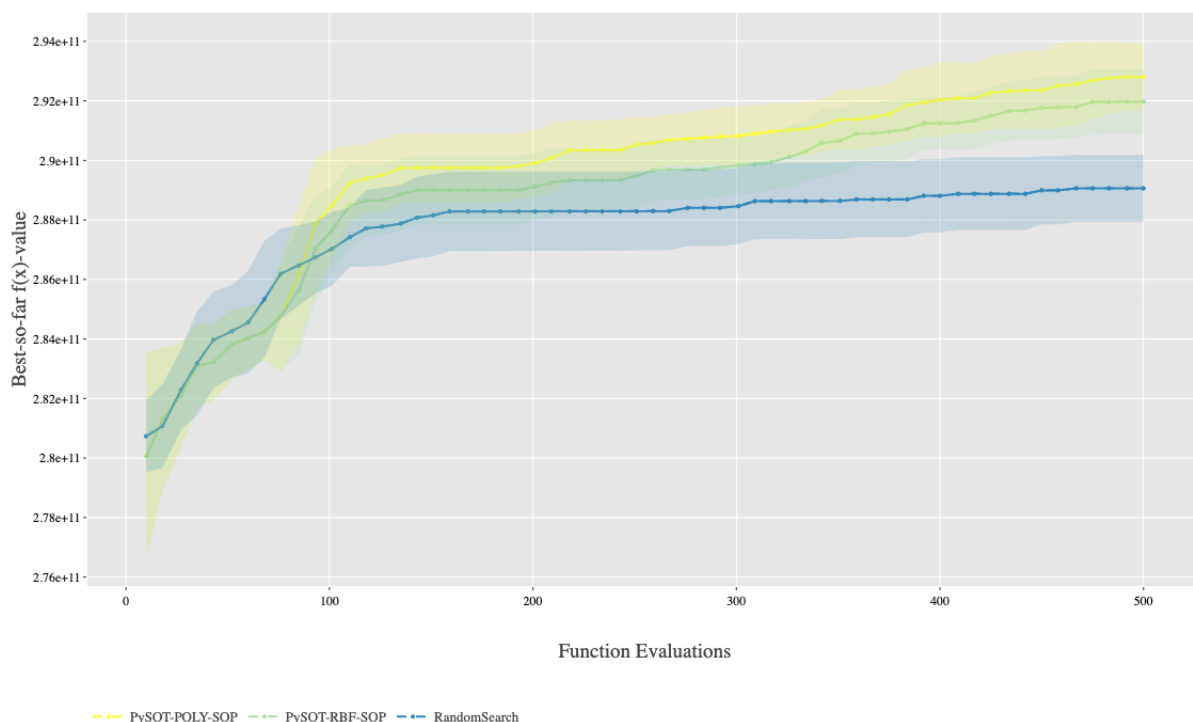
Obr. 33: Konvergenčný priebeh konfigurácií PySOT so stratégiou SOP pri RBF a polynomiálnom náhradnom modeli pri konfigurácii s 5 turbínami.

Pri konfigurácii s 5 turbínami dosahovala konfigurácia so stratégiou SOP a polynomiálnym náhradným modelom najvyššie priemerné hodnoty z porovnáwanej skupiny. Konfigurácia s RBF modelom dosahovala nižšie, ale stále vyššie hodnoty než Random Search.



Obr. 34: Konvergenčný priebeh konfigurácií PySOT so stratégiou SOP pri RBF a polynomiálnom náhradnom modeli pri konfigurácii s 10 turbínami.

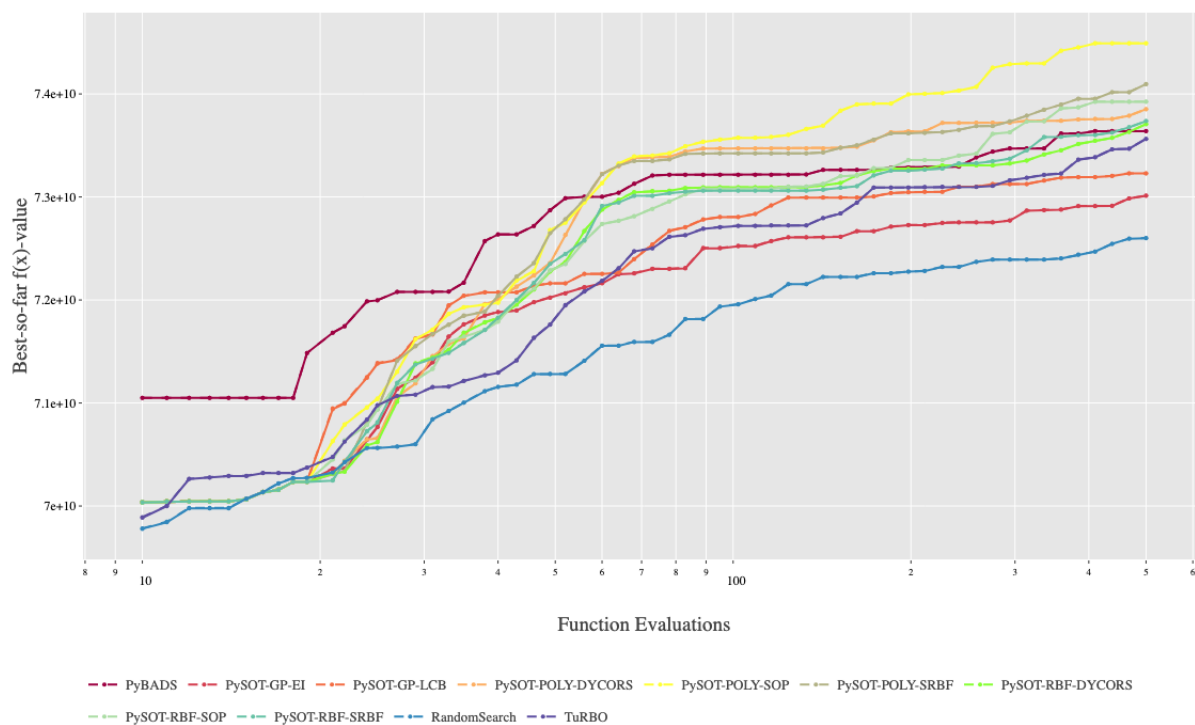
Pri konfigurácii s 10 turbínami dosahovala konfigurácia so stratégiou SOP a polynomiálnym náhradným modelom vyššie priemerné hodnoty než konfigurácia s RBF modelom. Obe konfigurácie SOP boli počas optimalizácie nad referenčnou metódou Random Search. Výsledok zodpovedá tabuľkovému poradiu, v ktorom konfigurácia s polynomiálnym modelom a stratégiou SOP dosiahla najvyšší priemerný výkon.



Obr. 35: Konvergenčný priebeh konfigurácií PySOT so stratégiou SOP pri RBF a polynomiálnom náhradnom modeli pri konfigurácii s 20 turbínami.

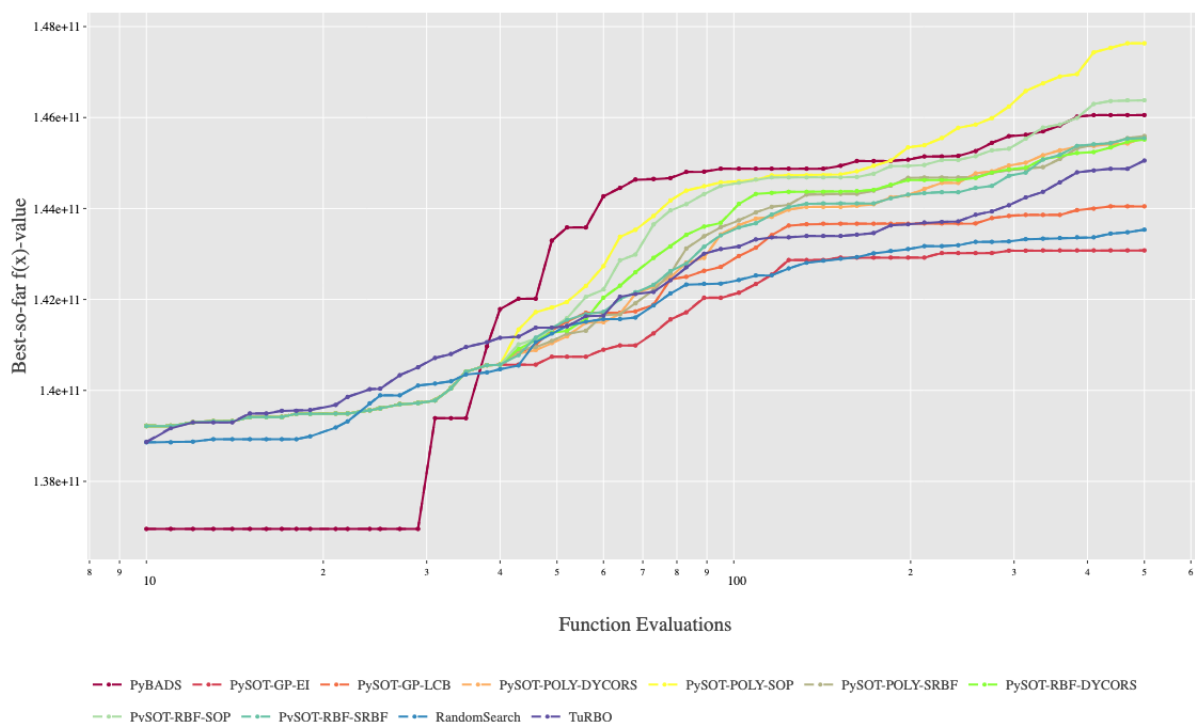
Pri konfigurácii s 20 turbínami dosahovala konfigurácia so stratégiou SOP a polynomiálnym náhradným modelom najvyššie hodnoty aj v tomto porovnaní. Konfigurácia s RBF modelom sa nachádzala pod polynomiálnou konfiguráciou, ale nad metódou Random Search.

Na záver konvergenčného vyhodnotenia boli doplnené aj súhrnné grafy obsahujúce všetky porovnávané algoritmy. Pre tieto grafy bola použitá logaritmická mierka osi evaluácií, pretože umožňuje lepšie rozlíšiť rozdiely v úvodnej fáze optimalizácie.



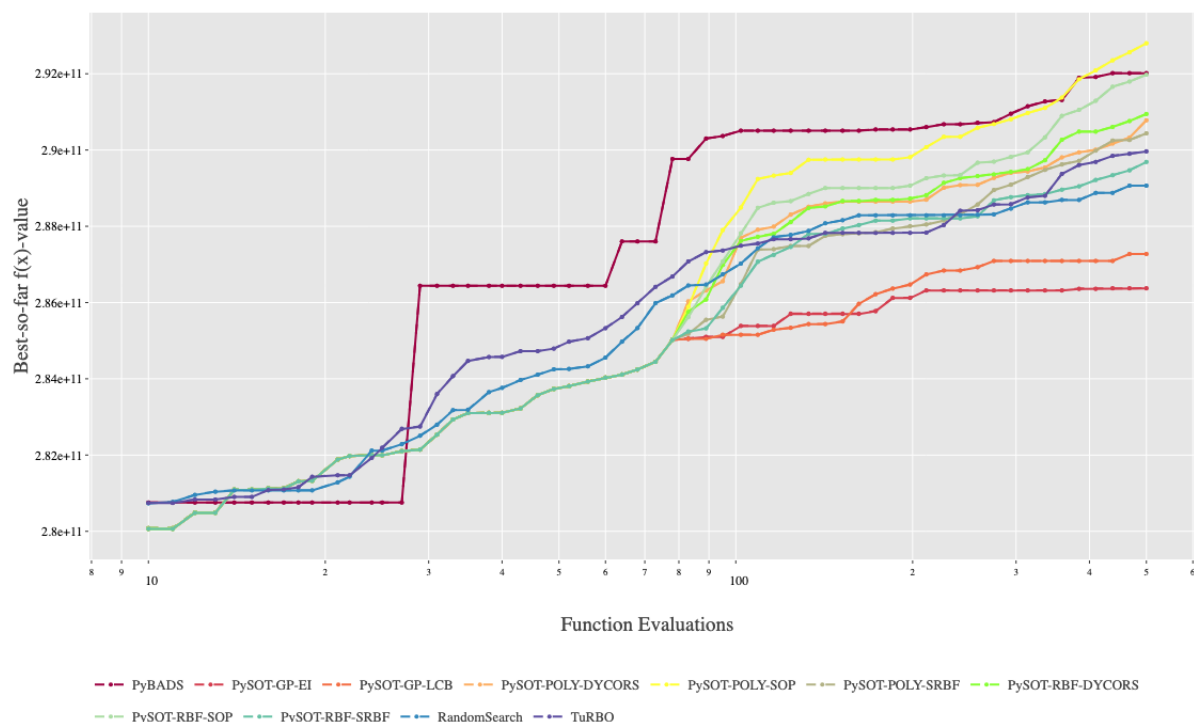
Obr. 36: Súhrnný konvergenčný priebeh všetkých porovnávaných algoritmov v logaritmickej mierke vodorovnej osi počtu evaluácií pri konfigurácii s 5 turbínami.

Pri konfigurácii s 5 turbínami súhrnný graf ukazuje, že väčšina algoritmov využívajúcich náhradný model dosahovala vyššie hodnoty než Random Search. Najvyššie priebehy dosahovali najmä konfigurácie rámca PySOT využívajúce polynomiálny náhradný model a stratégiu SOP alebo SRBF. Základné algoritmy PyBADs a TuRBO dosahovali lepší priebeh než Random Search, ale v celkovom porovnaní zostávali pod najlepšími konfiguráciami rámca PySOT.



Obr. 37: Súhrnný konvergenčný priebeh všetkých porovnávaných algoritmov v logaritmickej mierke vodorovnej osi počtu evaluácií pri konfigurácii s 10 turbínami.

Pri konfigurácii s 10 turbínami sa rozdiely medzi algoritmami prejavili výraznejšie. Najvyššie hodnoty dosahovala konfigurácia rámca PySOT využívajúca polynomiálny náhradný model a stratégiu SOP. Výrazne priaznivý priebeh dosiahli aj niektoré konfigurácie založené na RBF a polynomiálnych modeloch so stratégiami SOP, SRBF a DYCORS. Random Search sa nachádzal v spodnej časti porovnania, pričom konfigurácia s Gaussovským procesom a očakávaným zlepšením patrila medzi najslabšie priebehy.



Obr. 38: Súhrnný konvergenčný priebeh všetkých porovnávaných algoritmov v logaritmickej mierke vodorovnej osi počtu evaluácií pri konfigurácii s 20 turbínami.

Pri konfigurácii s 20 turbínami zostala najlepšou konfiguráciou z pohľadu konvergenčného priebehu kombinácia rámca PySOT s polynomiálnym náhradným modelom a stratégiou SOP. PyBADs dosahoval veľmi dobrý priebeh a v tejto konfigurácii sa zaradil medzi najlepšie metódy. Random Search dosahoval nižšie hodnoty než väčšina modelovo riadených prístupov, avšak prekonával niektoré konfigurácie založené na Gaussovskom procese. Súhrnný graf tak ukazuje, že kvalita výsledkov závisela nielen od použitia náhradného modelu, ale aj od konkrétnej kombinácie modelu a stratégie výberu kandidátnych riešení.

4.2.3 Efektivita využitia evaluácií

Efektivita využitia evaluácií bola hodnotená podľa počtu vyhodnotení cieľovej funkcie potrebných na dosiahnutie najlepšieho riešenia v danom behu. Táto metrika dopĺňa predchádzajúce hodnotenie kvality riešení, pretože algoritmus môže dosiahnuť vysokú finálnu hodnotu, ale až po väčšej časti dostupného rozpočtu. V tejto časti sú preto porovnané priemerné, mediánové a smerodajné hodnoty počtu evaluácií potrebných na dosiahnutie najlepšieho riešenia. Súčasne je uvedený aj podiel rozpočtu, pri ktorom bolo najlepšie riešenie v priemere dosiahnuté.

Tab. 7: Efektivita využitia evaluácií pri konfigurácii s 5 turbínami

Algoritmus	Priemerný počet evaluácií do najlepšieho riešenia	Medián evaluácií do najlepšieho riešenia	Smerodajná odchýlka evaluácií	Podiel rozpočtu do najlepšieho riešenia [%]
PySOT-GP-LCB	217,950	173,000	141,032	43,590
Random Search	241,900	194,500	147,691	48,380
PySOT-POLY-DYCORS	259,800	200,500	167,880	51,960
PySOT-RBF-SRBF	295,750	319,000	150,389	59,150
PySOT-POLY-SRBF	296,350	328,500	155,982	59,270
TuRBO	299,600	308,500	132,175	59,920
PySOT-GP-EI	302,600	337,000	160,694	60,520
PySOT-RBF-DYCORS	339,100	380,000	134,017	67,820
PyBADs	345,300	345,000	71,283	69,060
PySOT-POLY-SOP	364,300	354,000	30,911	72,860
PySOT-RBF-SOP	368,100	356,500	26,103	73,620

Pri konfigurácii s 5 turbínami bol najnižší priemerný počet evaluácií do najlepšieho riešenia zaznamenaný pri konfigurácii knižnice PySOT s Gaussovským procesom a akvizičnou funkciou dolnej hranice spoľahlivosti. Najlepšia hodnota bola dosiahnutá v priemere po 217,95 evaluáciách, čo predstavovalo 43,59 % dostupného rozpočtu. Referenčnou metódou Random Search bolo najlepšie riešenie dosiahnuté po 241,9 evaluáciách, teda po 48,380 % rozpočtu. V hornom pásme sa nachádzala aj konfigurácia knižnice PySOT s polynomiálnym náhradným modelom a výberovou stratégiou DYCORDS, pri ktorej bolo najlepšie riešenie dosiahnuté po 259,8 evaluáciách.

Stredné pásmo pri konfigurácii s 5 turbínami tvorili konfigurácie knižnice PySOT využívajúce stratégie SRBF pri oboch typoch náhradného modelu, algoritmus TuRBO a konfigurácia s Gaussovským procesom a akvizičnou funkciou očakávaného zlepšenia. Pri týchto metódach sa priemerný počet evaluácií do najlepšieho riešenia pohyboval približne od 295 do 303 evaluácií.

Najvyššie hodnoty tejto metriky boli zaznamenané pri konfiguráciách so stratégiou SOP. Pri polynomiálnom náhradnom modeli bolo najlepšie riešenie dosiahnuté po 364,3 evaluáciách a pri náhradnom modeli RBF po 368,1 evaluáciách.

Tab. 8: Efektivita využitia evaluácií pri konfigurácii s 10 turbínami

Algoritmus	Priemerný počet evaluácií do najlepšieho riešenia	Medián evaluácií do najlepšieho riešenia	Smerodajná odchýlka evaluácií	Podiel rozpočtu do najlepšieho riešenia [%]
PySOT-GP-EI	123,600	116,000	75,208	24,720
PySOT-GP-LCB	205,650	128,000	130,182	41,130
Random Search	256,400	226,000	153,122	51,280
PySOT-RBF-SRBF	297,000	322,500	114,473	59,400
TuRBO	334,000	358,500	122,300	66,800
PySOT-POLY-SRBF	334,300	372,500	137,768	66,860
PySOT-RBF-DYCORS	334,500	379,500	139,463	66,900
PyBADs	364,750	360,500	13,011	72,950
PySOT-POLY-DYCORS	371,450	362,500	76,790	74,290
PySOT-RBF-SOP	396,600	389,500	27,956	79,320
PySOT-POLY-SOP	416,050	427,000	28,172	83,210

Pri konfigurácii s 10 turbínami bola najnižšia hodnota dosiahnutá konfiguráciou knižnice PySOT s Gaussovským procesom a akvizičnou funkciou očakávaného zlepšenia. Najlepšie riešenie bolo dosiahnuté v priemere po 123,6 evaluáciách, čo predstavovalo 24,72 % dostupného rozpočtu. Druhá najnižšia hodnota bola zaznamenaná pri konfigurácii s Gaussovským procesom a akvizičnou funkciou dolnej hranice spoľahlivosti, pri ktorej bolo najlepšie riešenie dosiahnuté po 205,65 evaluáciách. Referenčná metóda Random Search dosiahla svoje najlepšie riešenie po 256,4 evaluáciách, teda po 51,28 % dostupného rozpočtu.

Stredné pásmo pri konfigurácii s 10 turbínami tvorili algoritmus TuRBO, algoritmus PyBADs a konfigurácie knižnice PySOT so stratégiami SRBF a DYCORDS. Pri týchto algoritmoch sa najlepšie riešenie objavovalo približne medzi 297 a 371 evaluáciami.

Najvyššie hodnoty boli opäť zaznamenané pri konfiguráciách so stratégiou SOP. Pri náhradnom modeli radiálnych bázových funkcií bolo najlepšie riešenie dosiahnuté po 396,6 evaluáciách a pri polynomiálnom náhradnom modeli po 416,05 evaluáciách, čo zodpovedalo 79,32 % a 83,21 % dostupného rozpočtu.

Tab. 9: Efektivita využitia evaluácií pri konfigurácii s 20 turbínami

Algoritmus	Priemerný počet evaluácií do najlepšieho riešenia	Medián evaluácií do najlepšieho riešenia	Smerodajná odchýlka evaluácií	Podiel rozpočtu do najlepšieho riešenia [%]
PySOT-GP-EI	125,600	81,000	89,100	25,120
PySOT-GP-LCB	192,800	180,000	111,218	38,560
Random Search	234,800	200,000	128,729	46,960
PySOT-RBF-SRBF	362,300	384,500	124,910	72,460
PySOT-POLY-SRBF	366,600	397,500	88,282	73,320
TuRBO	373,400	372,500	84,471	74,680
PySOT-RBF-DYCORS	380,400	373,000	107,361	76,080
PyBADS	389,000	378,000	20,905	77,800
PySOT-POLY-DYCORS	398,400	433,500	121,580	79,680
PySOT-RBF-SOP	441,900	443,000	28,948	88,380
PySOT-POLY-SOP	445,050	459,000	39,905	89,010

Pri konfigurácii s 20 turbínami bola najnižšia hodnota opäť zaznamenaná pri konfigurácii knižnice PySOT s Gaussovským procesom a akvizičnou funkciou očakávaného zlepšenia. Najlepšie riešenie bolo dosiahnuté po 125,6 evaluáciách, teda po 25,12 % dostupného rozpočtu. Konfigurácia s Gaussovským procesom a akvizičnou funkciou dolnej hranice spoľahlivosti dosiahla najlepšie riešenie po 192,8 evaluáciách. Referenčná metóda Random Search dosiahla vlastné najlepšie riešenie po 234,8 evaluáciách, čo predstavovalo 46,960 % rozpočtu.

Stredné pásmo pri konfigurácii s 20 turbínami tvorili konfigurácie knižnice PySOT so stratégiami SRBF a DYCORS, algoritmus TuRBO a algoritmus PyBADS. Pri týchto metódach sa najlepšie riešenie objavovalo približne medzi 362 a 398 evaluáciami.

Najvyššie hodnoty boli zaznamenané pri konfiguráciách so stratégiou SOP. Pri náhradnom modeli radiálnych bazových funkcií bolo najlepšie riešenie dosiahnuté po 441,9 evaluáciách a pri polynomiálnom náhradnom modeli po 445,05 evaluáciách, čo predstavovalo 88,38 % a 89,01 % dostupného rozpočtu.

4.2.4 Časová náročnosť

Časová náročnosť algoritmov bola hodnotená na základe priemerného času jedného behu, smerodajnej odchýlky času behu, priemerného času na jednu evaluáciu a celkového času experimentu hromadného spustenia behov. Táto metrika dopĺňa predchádzajúce hodnotenie, pretože algoritmus s vysokou kvalitou riešenia nemusí byť zároveň výpočtovo najefektívnejší. Pri interpretácii časových výsledkov je potrebné zohľadniť, že všetky experimenty boli vykonané v rovnakom výpočtovom prostredí.

Tab. 10: Časová náročnosť algoritmov pri konfigurácii s 5 turbínami

Algoritmus	Priemerný čas behu [min]	Priemerný čas jednej evaluácie [s]	Celkový čas experimentu [min]	Celkový čas experimentu [h]
Random Search	5,295	0,636	105,913	1,765
PySOT-POLY-SRBF	8,404	1,009	168,083	2,801
PySOT-RBF-SRBF	9,076	1,089	181,523	3,025
PySOT-POLY-DYCORS	9,432	1,132	188,636	3,144
TuRBO	10,351	1,242	207,030	3,451
PySOT-POLY-SOP	10,667	1,280	213,340	3,556
PyBADs	11,346	1,362	226,942	3,782
PySOT-RBF-DYCORS	14,078	1,689	281,572	4,693
PySOT-RBF-SOP	16,768	2,012	335,368	5,589
PySOT-GP-LCB	32,340	3,881	646,806	10,780
PySOT-GP-EI	35,285	4,234	705,732	11,762

Pri konfigurácii s 5 turbínami bol najnižší priemerný čas jedného behu zaznamenaný pri referenčnej metóde Random Search. Priemerný čas behu bol 5,295 min a celkový čas experimentu dosiahol 105,913 min, teda 1,765 h. Tento výsledok bol očakávaný, keďže pri referenčnej metóde nie je vytváraný ani aktualizovaný náhradný model a nevykonávajú sa dodatočné modelovo riadené výpočty.

Spomedzi algoritmov založených na náhradnom modeli boli pri konfigurácii s 5 turbínami časovo najmenej náročné konfigurácie knižnice PySOT s polynomiálnym náhradným modelom a stratégiou SRBF, s náhradným modelom radiálnych bázových funkcií a stratégiou SRBF a s polynomiálnym náhradným modelom a stratégiou DYCORDS. Ich priemerný čas behu sa pohyboval od 8,404 min do 9,432 min. Do stredného pásma časovej náročnosti patrili algoritmy TuRBO a PyBADs spolu s konfiguráciami využívajúcimi stratégiu SOP alebo DYCORDS pri náhradnom modeli radiálnych bázových funkcií.

Najvyššia časová náročnosť bola pri tejto konfigurácii zaznamenaná pri konfiguráciách s Gaussovským procesom, najmä pri akvizičnej funkcii očakávaného zlepšenia, kde celkový čas experimentu dosiahol 705,732 min, teda 11,762 h.

Tab. 11: Časová náročnosť algoritmov pri konfigurácii s 10 turbínami

Algoritmus	Priemerný čas behu [min]	Priemerný čas jednej evaluácie [s]	Celkový čas experimentu [min]	Celkový čas experimentu [h]
Random Search	10,380	1,247	207,612	3,460
PySOT-POLY-DYCORS	18,261	2,191	365,223	6,087
PySOT-POLY-SRBF	18,941	2,273	378,832	6,314
PyBADs	22,402	2,688	448,053	7,468
TuRBO	25,175	3,021	503,516	8,392
PySOT-RBF-SRBF	31,334	3,760	626,699	10,445
PySOT-RBF-DYCORS	33,013	3,962	660,264	11,004
PySOT-POLY-SOP	34,285	4,114	685,721	11,429
PySOT-RBF-SOP	37,061	4,447	741,227	12,354
PySOT-GP-LCB	125,490	15,059	2 509,810	41,830
PySOT-GP-EI	131,765	15,812	2 635,304	43,922

Pri konfigurácii s 10 turbínami sa časová náročnosť zvýšila pri všetkých algoritmoch. Najnižší priemerný čas behu bol opäť zaznamenaný pri metóde Random Search, konkrétne 10,380 min, pričom celkový čas experimentu bol 207,612 min, teda 3,460 h. Medzi najrýchlejšie modelovo riadené prístupy patrili konfigurácie knižnice PySOT s polynomiálnym náhradným modelom a stratégiami DYCORDS a SRBF. Ich priemerný čas behu bol 18,261 min a 18,941 min.

Stredné pásmo pri konfigurácii s 10 turbínami tvorili algoritmy PyBADs a TuRBO spolu s konfiguráciami knižnice PySOT využívajúcimi náhradný model radiálnych bázových funkcií alebo stratégiu SOP. Priemerný čas behu sa v tejto skupine pohyboval približne od 22,402 min do 37,061 min.

Najvyššie hodnoty boli zaznamenané pri konfiguráciách založených na Gaussovskom procese. Pri akvizičnej funkcii dolnej hranice spoľahlivosti bol celkový čas experimentu 2 509,810 min, teda 41,830 h, a pri akvizičnej funkcii očakávaného zlepšenia 2 635,304 min, teda 43,922 h.

Tab. 12: Časová náročnosť algoritmov pri konfigurácii s 20 turbínami

Algoritmus	Priemerný čas behu [min]	Priemerný čas jednej evaluácie [s]	Celkový čas experimentu [min]	Celkový čas experimentu [h]
PySOT-RBF-SRBF	28,733	3,448	574,670	9,578
Random Search	29,358	3,523	587,170	9,786
PySOT-RBF-DYCORS	37,821	4,539	756,424	12,607
TuRBO	46,897	5,628	937,941	15,632
PySOT-RBF-SOP	51,421	6,171	1 028,428	17,140
PySOT-POLY-SRBF	53,547	6,426	1 070,966	17,849
PySOT-POLY-SOP	55,496	6,660	1 109,923	18,499
PySOT-POLY-DYCORS	59,851	7,182	1 197,038	19,951
PyBADS	63,234	7,588	1 264,698	21,078
PySOT-GP-LCB	206,441	24,773	4 128,830	68,814
PySOT-GP-EI	297,566	35,708	5 951,334	99,189

Pri konfigurácii s 20 turbínami bol najnižší priemerný čas behu zaznamenaný pri konfigurácii knižnice PySOT s náhradným modelom radiálnych bázových funkcií a stratégiou SRBF. Priemerný čas behu bol 28,733 min a celkový čas experimentu dosiahol 574,670 min, teda 9,578 h. Referenčná metóda Random Search dosiahla veľmi podobný čas, konkrétne 29,358 min na jeden beh a 587,170 min celkového času experimentu. Oproti konfiguráciám s 5 a 10 turbínami teda pri 20 turbínach nebola referenčná metóda najrýchlejšia, ale zostala v najnižšom časovom pásme.

Stredné pásmo časovej náročnosti pri 20 turbínach tvorili konfigurácie knižnice PySOT s náhradným modelom radiálnych bázových funkcií alebo polynomiálnym náhradným modelom, algoritmus TuRBO a algoritmus PyBADS. Priemerný čas behu sa v tejto skupine pohyboval približne od 37,821 min do 63,234 min.

Najvyššie časy boli opäť zaznamenané pri konfiguráciách založených na Gaussovskom procese. Pri akvizičnej funkcii dolnej hranice spoľahlivosti bol celkový čas experimentu 4 128,830 min, teda 68,814 h, zatiaľ čo pri akvizičnej funkcii očakávaného zlepšenia dosiahol 5 951,334 min, teda 99,189 h.

5 ZHODNOTENIE A DISKUSIA

Kapitola výsledkov prezentovala namerané hodnoty získané pri experimentoch s úlohou rozmiestnenia environmentálnych senzorov a úlohou rozmiestnenia veterných turbín. Zameriavala sa na dosiahnutú kvalitu riešení, konvergenčný priebeh algoritmov, efektivitu využitia evaluácií a časovú náročnosť jednotlivých metód. Táto kapitola nadväzuje na uvedené výsledky a sústreďuje sa na ich interpretáciu.

Cieľom diskusie je vysvetliť, prečo sa porovnávané algoritmy správali rozdielne v diskkrétnej a spojitaj optimalizačnej úlohe. Osobitná pozornosť je venovaná tomu, prečo referenčná metóda Random Search dosiahla veľmi dobré výsledky pri úlohe rozmiestnenia environmentálnych senzorov, zatiaľ čo pri úlohe rozmiestnenia veterných turbín boli úspešnejšie viaceré metódy založené na náhradných modeloch. Diskusia sa zároveň venuje limitom implementácie, výpočtovým obmedzeniam a praktickému významu optimalizácie s využitím náhradných modelov v klimaticky orientovaných aplikáciách.

5.1 Súhrnné porovnanie diskkrétnej a spojitaj úlohy

V diplomovej práci boli riešené dve aplikačné úlohy s odlišným charakterom optimalizačného priestoru. Úloha rozmiestnenia environmentálnych senzorov bola diskrétna, pretože kandidátne riešenia boli obmedzené na konečnú množinu dostupných lokalít. Úloha rozmiestnenia veterných turbín bola naopak spojitá, keďže riešenie bolo reprezentované súradnicami turbín v priestore prípustných rozmiestnení.

Pri diskkrétnej úlohe environmentálnych senzorov bol dostupný rozpočet 500 evaluácií dostatočne veľký na to, aby mohla byť kandidátna doména pokrytá veľmi účinne aj jednoduchou náhodnou metódou. Zároveň bolo pri tejto úlohe známe optimum jednotlivých problémových inštancií, čo umožnilo priamo hodnotiť úspešnosť jeho nájdenia.

Pri spojitaj úlohe veterných turbín bola situácia odlišná. Optimalizačný priestor bol rozsiahlejší a jeho dimenzia rástla s počtom turbín. Náhodné vyhľadávanie v takomto priestore nedokázalo systematicky využívať informácie z predchádzajúcich evaluácií. Algoritmy založené na náhradných modeloch preto mali lepšie podmienky na uplatnenie, pretože mohli postupne vytvárať aproximáciu cieľovej funkcie a využívať ju pri návrhu ďalších kandidátnych riešení.

Z porovnania oboch úloh vyplýva, že prínos náhradných modelov nemožno posudzovať všeobecne bez ohľadu na charakter problému. Ich význam rastie najmä v spojitých alebo výpočtovo náročných úlohách, kde je potrebné efektívne využívať obmedzený počet evaluácií. Pri diskrétnych úlohách s relatívne dostupnou kandidátnou množinou môže byť naopak jednoduchá referenčná metóda veľmi konkurencieschopná a skresľovať výkon algoritmov založených na náhradných modeloch.

5.2 Interpretácia výsledkov úlohy rozmiestnenia veterných turbín

Najlepšie výsledky boli pri tejto úlohe dosiahnuté konfiguráciami knižnice PySOT využívajúcimi polynomiálny náhradný model alebo náhradný model radiálnych bázových funkcií v kombinácii so stratégiou SOP. Tento výsledok naznačuje, že pri danej formulácii úlohy bola vhodná práve kombinácia jednoduchšej modelovej aproximácie a stratégie. Výsledky potvrdzujú, že najzložitejší model nemusí byť automaticky najúspešnejší.

Dôležité bolo aj správanie algoritmu PyBADs. Pri konfigurácii s vyšším počtom turbín sa zaradil medzi najúspešnejšie metódy. Jeho výsledky ukazujú, že lokálne modelovo podporené vyhľadávanie môže byť pri spojitých úlohách účinné.

Algoritmus TuRBO dosahoval použiteľné výsledky, ale vo väčšine porovnaní nepredstavoval najvýkonnejší prístup. Jeho správanie možno chápať ako stabilnejšie než čisté náhodné vyhľadávanie, no v tomto experimentálnom nastavení neprekonal najlepšie konfigurácie knižnice PySOT ani algoritmus PyBADs.

Konfigurácie založené na Gaussovskom procese nedosiahli pri úlohe veterných turbín najlepšie výsledky, hoci z teoretického hľadiska poskytujú výhodu vo forme odhadu neistoty. Zároveň boli spojené s výrazne vyššou časovou náročnosťou, najmä pri vyššom počte turbín.

Z pohľadu praktického použitia boli pri úlohe rozmiestnenia veterných turbín najvýhodnejšie také metódy, ktoré dosahovali dobrý kompromis medzi kvalitou riešenia a časovou náročnosťou. Výsledky ukázali, že túto úlohu nemožno hodnotiť iba podľa finálneho výkonu, ale aj podľa toho, koľko výpočtových zdrojov bolo potrebných na jeho dosiahnutie.

5.3 Interpretácia výsledkov úlohy rozmiestnenia environmentálnych senzorov

Pri úlohe rozmiestnenia environmentálnych senzorov dosiahla najlepšie výsledky referenčná metóda Random Search. Tento výsledok je možné vysvetliť diskretným charakterom úlohy a konečnou množinou kandidátnych lokalít. Pri rozpočte 500 evaluácií dokázala náhodná metóda pokryť kandidátnu doménu dostatočne účinne na to, aby vo všetkých riešených prípadoch našla známe optimum.

Spomedzi modelovo riadených prístupov dosiahli najlepšie výsledky najmä konfigurácie knižnice PySOT využívajúce náhradný model RBF so stratégiami SRBF a DYCORS. Podobne priaznivé výsledky boli zaznamenané aj pri algoritmoch TuRBO a PyBADs. Tieto výsledky naznačujú, že náhradné modely dokázali v úlohe environmentálnych senzorov poskytovať výhodu vo využití náhradného modelu pre usmernenia vyhľadávania najlepšieho riešenia.

Najslabšie výsledky boli zaznamenané pri konfiguráciách so stratégiou SOP. Tieto konfigurácie dosahovali nízku úspešnosť nájdenia optima. Zároveň sa pri nich prejavila skorá stabilizácia na slabšej hodnote. Tento výsledok ukazuje, že rýchle dosiahnutie vlastného najlepšieho riešenia nemusí znamenať vysokú kvalitu riešenia, ak sa algoritmus prestane zlepšovať príliš skoro.

5.4 Porovnanie algoritmov z pohľadu kvality, konvergenzie a času

Porovnanie algoritmov ukázalo, že ich hodnotenie závisí od zvolenej metriky. Algoritmus s najvyššou finálnou kvalitou riešenia nemusel byť zároveň najrýchlejší a algoritmus, ktorý dosiahol svoje vlastné najlepšie riešenie najskôr, nemusel dosiahnuť najlepšiu absolútnu hodnotu. Preto bolo potrebné hodnotiť kvalitu riešenia, konvergenciu, efektivitu využitia evaluácií a časovú náročnosť súčasne.

Random Search predstavoval veľmi silnú referenčnú metódu pri diskretnej úlohe enviromentálnych senzorov, ale pri spojitkej úlohe veterných turbín vo väčšine porovnaní zaostával za modelovo riadenými metódami. Jeho hlavnou výhodou bola nízka výpočtová réžia a jednoduchosť. Jeho hlavnou slabinou bolo, že nedokázal systematicky využívať informácie získané počas predchádzajúcich evaluácií.

Algoritmus PyBADs dosahoval stabilné výsledky najmä pri spojitkej úlohe rozmiestnenia veterných turbín. Jeho výhodou bolo lokálne modelovo podporené vyhľadávanie, ktoré sa ukázalo ako vhodné pri postupnom zlepšovaní kandidátnych riešení.

Algoritmus TuRBO dosahoval použiteľné výsledky, ale v tejto práci nebol dominantnou metódou. Pri úlohe enviromentálnych senzorov patril medzi lepšie modelovo riadené prístupy. Pri úlohe veterných turbín dosiahol lepšie výsledky než Random Search v základnom porovnaní, ale zvyčajne zaostával za najlepšími konfiguráciami PySOT a za algoritmom PyBADs.

Najväčšie rozdiely boli pozorované pri konfiguráciách knižnice PySOT. Jej výkonnosť výrazne závisela od kombinácie náhradného modelu a výberovej stratégie. Pri úlohe rozmiestnenia enviromentálnych senzorov sa ako vhodnejšie ukázali modely RBF so stratégiami SRBF a DYCORs. Pri úlohe rozmiestnenia veterných turbín bola najúspešnejšia stratégia SOP, najmä v kombinácii s polynomiálnym modelom alebo modelom RBF. Tento rozdiel potvrdzuje, že univerzálne najlepšia konfigurácia nebola identifikovaná.

Z pohľadu času boli najnáročnejšie najmä konfigurácie založené na Gaussovskom procese. Napriek tomu, že tento model poskytuje informáciu o neistote predikcie, v experimentoch nedosiahol najlepšiu finálnu kvalitu riešenia. Výsledky preto ukazujú, že pri praktickom použití nestačí hodnotiť iba teoretické vlastnosti náhradného modelu.

5.5 Limity implementácie a experimentálneho nastavenia

Pri interpretácii výsledkov je potrebné zohľadniť viacero limitov implementácie a experimentálneho nastavenia. Tieto limity neznameniajú neplatnosť výsledkov, ale určujú rozsah, v ktorom je možné ich interpretovať. Porovnávané algoritmy boli hodnotené pri konkrétnom rozpočte evaluácií, počte behov, použitých dátach a dostupnom výpočtovom prostredí.

Prvým limitom bol rozpočet 500 evaluácií cieľovej funkcie. Tento rozpočet umožnil porovnateľné experimenty naprieč algoritmi, ale jeho význam bol v oboch úlohách odlišný. Pri diskretnej úlohe LAQN bol dostatočný na veľmi účinné pokrytie kandidátnej domény. Pri

spojitej úlohe WindWake však predstavoval výraznejšie obmedzenie, pretože optimalizačný priestor bol rozsiahlejší a jeho dimenzia rástla s počtom turbín.

Druhým limitom bol počet opakovaných behov. V hlavných porovnaniach bolo použitých 20 nezávislých behov, čo umožnilo sledovať priemerné hodnoty a variabilitu výsledkov. Pri vyššom počte behov by však bolo možné získať presnejší odhad stability stochastických algoritmov, najmä pri metódach citlivých na náhodnú inicializáciu alebo náhodný výber kandidátnych bodov.

Významným praktickým limitom bola časová náročnosť experimentov. Niektoré konfigurácie, najmä pri vyššom počte turbín alebo pri použití Gaussovského procesu, vyžadovali veľmi dlhý výpočtový čas. Z tohto dôvodu nebolo možné rozšíriť experimenty na všetky možné kombinácie nastavení, vyšší počet behov alebo výrazne vyššie rozpočty evaluácií. Plánované experimenty optimalizačných algoritmov knižnice PySOT s Gaussovským procesom neboli zaradené do finálneho vyhodnotenia, pretože ich dokončenie nebolo pri dostupnom výpočtovom prostredí realistické.

Výsledky boli zároveň získané v konkrétnom výpočtovom prostredí. Časové metriky preto treba chápať najmä ako relatívne porovnanie algoritmov v rámci rovnakej experimentálnej zostavy. Nemožno ich priamo považovať za univerzálne hodnoty platné pre iné hardvérové alebo softvérové prostredie.

6 ZÁVER

Cieľom tejto diplomovej práce bolo preskúmať možnosti optimalizácie s využitím náhradných modelov v aplikáciách súvisiacich s klimatickou zmenou. Práca sa zamerala na dve odlišné aplikačné úlohy: rozmiestnenie environmentálnych senzorov a rozmiestnenie veterných turbín. Tieto úlohy boli zvolené tak, aby reprezentovali dva rozdielne typy optimalizačných problémov. Prvá úloha mala diskretný charakter a vychádzala z dát siete London Air Quality Network, zatiaľ čo druhá úloha bola formulovaná ako spojitý optimalizačný problém rozmiestnenia veterných turbín s cieľom maximalizovať výkon veternej farmy. Takto bolo možné porovnať správanie algoritmov s náhradnými modelmi v dvoch rozdielnych experimentálnych podmienkach.

V teoretickej časti boli spracované východiská optimalizácie nákladných black-box funkcií, princíp náhradného modelovania, základné typy náhradných modelov a postupy výberu ďalších kandidátnych bodov. Pozornosť bola venovaná najmä polynomiálnym modelom, modelom radiálnych bázových funkcií, Gaussovským procesom, akvizíčným funkciám a vybraným algoritmom optimalizácie s využitím náhradných modelov. Tieto poznatky vytvorili základ pre následnú implementáciu a experimentálne porovnanie algoritmov PyBADS, TuRBO a konfigurácií knižnice PySOT. Súčasťou porovnania bola aj referenčná metóda Random Search, ktorá slúžila ako jednoduchý základný prístup bez využitia náhradného modelu.

V praktickej časti bol vytvorený experimentálny rámec, ktorý umožnil jednotné spúšťanie, vyhodnocovanie a porovnávanie algoritmov na oboch aplikačných úlohách. Pre úlohu rozmiestnenia environmentálnych senzorov bolo potrebné prispôbiť spojitým orientovaným optimalizačným algoritmom diskretnú kandidátnu doménu. Tento problém bol riešený pomocou premietania bodov navrhnutých v spojitom priestore na najbližšie prípustné diskretné lokality. Pre úlohu rozmiestnenia veterných turbín boli algoritmy napojené na spojitý optimalizačný problém, v ktorom boli kandidátne riešenia reprezentované súradnicami turbín. Implementácia zahŕňala hromadné spúšťanie experimentov, záznam priebehov optimalizácie, výpočet hodnotiacich metrík a prípravu dát pre tabuľkové aj grafické vyhodnotenie.

Na základe výsledkov boli ciele práce splnené. Boli spracované teoretické východiská optimalizácie s využitím náhradných modelov, boli zvolené dve klimaticky orientované aplikačné úlohy s rozdielnym charakterom optimalizačného priestoru, bol vytvorený experimentálny rámec na porovnanie vybraných algoritmov a boli vyhodnotené výsledky z hľadiska kvality riešenia, konvergenčného priebehu, efektivity využitia evaluácií a časovej náročnosti.

Inžiniersky prínos práce spočíva v praktickom prepojení existujúcich optimalizačných algoritmov, testovacích prostredí a vlastného experimentálneho vyhodnocovacieho rámca. Oproti všeobecnej literatúre, bola v tejto práci vykonaná ich praktická aplikácia a porovnanie na dvoch konkrétnych klimaticky orientovaných úlohách. Prínosom je systematické experimentálne overenie toho, ako sa vybrané algoritmy správajú pri rozdielnom type úlohy, a

rovnakom rozpočte evaluácií. Takéto porovnanie poskytuje praktickejší pohľad na použiteľnosť náhradných modelov než samotné teoretické vlastnosti jednotlivých metód.

Diplomová práca tak potvrdila, že optimalizácia s využitím náhradných modelov predstavuje použiteľný nástroj pre vybrané aplikácie súvisiace s klimatickou zmenou, najmä tam, kde je cieľová funkcia výpočtovo náročná a počet evaluácií je obmedzený. Zároveň však ukázala, že jej prínos je podmienený vhodnou formuláciou úlohy, voľbou modelu, stratégiou výberu kandidátnych riešení a výpočtovými obmedzeniami. Výsledky preto poskytujú základ pre ďalšie experimenty, v ktorých by bolo možné rozšíriť počet testovaných úloh, zvýšiť rozpočet evaluácií, podrobnejšie ladit' interné parametre algoritmov alebo skúmať ďalšie typy náhradných modelov a výberových stratégií.

ZOZNAM POUŽITEJ LITERATÚRY

- [1] D. R. Jones, M. Schonlau, a W. J. Welch, „Efficient Global Optimization of Expensive Black-Box Functions“, *J. Glob. Optim.*, roč. 13, č. 4, s. 455–492, pro. 1998, doi: 10.1023/A:1008306431147.
- [2] P. I. Frazier, „A Tutorial on Bayesian Optimization“, 8. červenec 2018, *arXiv*: arXiv:1807.02811. doi: 10.48550/arXiv.1807.02811.
- [3] N. V. Queipo, R. T. Haftka, W. Shyy, T. Goel, R. Vaidyanathan, a P. Kevin Tucker, „Surrogate-based analysis and optimization“, *Prog. Aerosp. Sci.*, roč. 41, č. 1, s. 1–28, led. 2005, doi: 10.1016/j.paerosci.2005.02.001.
- [4] A. I. J. Forrester, A. Sóbester, a A. J. Keane, *Engineering Design via Surrogate Modelling: A Practical Guide*, 1. vyd. Wiley, 2008. doi: 10.1002/9780470770801.
- [5] S. P. Hellan, C. G. Lucas, a N. H. Goddard, „Bayesian Optimisation Against Climate Change: Applications and Benchmarks“, in *Learning and Intelligent Optimization*, roč. 15744, Y. Zhang, M. Hladik, a H. Moosaei, Ed., in Lecture Notes in Computer Science, vol. 15744. , Cham: Springer Nature Switzerland, 2026, s. 107–118. doi: 10.1007/978-3-032-09156-7_8.
- [6] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, a N. De Freitas, „Taking the Human Out of the Loop: A Review of Bayesian Optimization“, *Proc. IEEE*, roč. 104, č. 1, s. 148–175, led. 2016, doi: 10.1109/JPROC.2015.2494218.
- [7] Y. Jin, „Surrogate-assisted evolutionary computation: Recent advances and future challenges“, *Swarm Evol. Comput.*, roč. 1, č. 2, s. 61–70, čer. 2011, doi: 10.1016/j.swevo.2011.05.001.
- [8] L. Blik, A. Guijt, R. Karlsson, S. Verwer, a M. De Weerd, „Benchmarking surrogate-based optimisation algorithms on expensive black-box functions“, *Appl. Soft Comput.*, roč. 147, s. 110744, lis. 2023, doi: 10.1016/j.asoc.2023.110744.
- [9] A. Bhosekar a M. Ierapetritou, „Advances in surrogate based modeling, feasibility analysis, and optimization: A review“, *Comput. Chem. Eng.*, roč. 108, s. 250–267, led. 2018, doi: 10.1016/j.compchemeng.2017.09.017.
- [10] A. I. J. Forrester a A. J. Keane, „Recent advances in surrogate-based optimization“, *Prog. Aerosp. Sci.*, roč. 45, č. 1–3, s. 50–79, led. 2009, doi: 10.1016/j.paerosci.2008.11.001.
- [11] E. Brochu, V. M. Cora, a N. de Freitas, „A Tutorial on Bayesian Optimization of Expensive Cost Functions, with Application to Active User Modeling and Hierarchical Reinforcement Learning“, 12. prosinec 2010, *arXiv*: arXiv:1012.2599. doi: 10.48550/arXiv.1012.2599.
- [12] S. P. Hellan, C. G. Lucas, a N. H. Goddard, „Bayesian Optimisation for Active Monitoring of Air Pollution“, *Proc. AAAI Conf. Artif. Intell.*, roč. 36, č. 11, s. 11908–11916, čer. 2022, doi: 10.1609/aaai.v36i11.21448.
- [13] J. Herbert-Acero, O. Probst, P.-E. Réthoré, G. Larsen, a K. Castillo-Villar, „A Review of Methodological Approaches for the Design and Optimization of Wind Farms“, *Energies*, roč. 7, č. 11, s. 6930–7016, říj. 2014, doi: 10.3390/en7116930.

- [14] J. Kudela a R. Matousek, „Recent advances and applications of surrogate models for finite element method computations: a review“, *Soft Comput.*, roč. 26, č. 24, s. 13709–13733, pro. 2022, doi: 10.1007/s00500-022-07362-8.
- [15] C. E. Rasmussen a C. K. I. Williams, *Gaussian processes for machine learning*, 3. print. in Adaptive computation and machine learning. Cambridge, Mass.: MIT Press, 2008.
- [16] D. Eriksson, M. Pearce, J. Gardner, R. D. Turner, a M. Poloczek, „Scalable Global Optimization via Local Bayesian Optimization“.
- [17] R. G. Regis a C. A. Shoemaker, „A Stochastic Radial Basis Function Method for the Global Optimization of Expensive Functions“.
- [18] R. G. Regis a C. A. Shoemaker, „Parallel Stochastic Global Optimization Using Radial Basis Functions“.
- [19] R. G. Regis a C. A. Shoemaker, „Combining radial basis function surrogates and dynamic coordinate search in high-dimensional expensive black-box optimization“, *Eng. Optim.*, roč. 45, č. 5, s. 529–555, kvě. 2013, doi: 10.1080/0305215X.2012.687731.
- [20] T. Krityakierne, T. Akhtar, a C. A. Shoemaker, „SOP: parallel surrogate global optimization with Pareto center selection for computationally expensive single objective problems“, *J. Glob. Optim.*, roč. 66, č. 3, s. 417–437, lis. 2016, doi: 10.1007/s10898-016-0407-7.
- [21] „Welcome to the pySOT documentation! — pySOT documentation“. Viděno: 9. květen 2026. [Online]. Dostupné z: <https://pysot.readthedocs.io/en/latest/>
- [22] L. Acerbi a W. J. Ma, „Practical Bayesian Optimization for Model Fitting with Bayesian Adaptive Direct Search“, 2. listopad 2017, *arXiv*: arXiv:1705.04405. doi: 10.48550/arXiv.1705.04405.
- [23] C. Audet a J. E. Dennis, „Mesh Adaptive Direct Search Algorithms for Constrained Optimization“, *SIAM J. Optim.*, roč. 17, č. 1, s. 188–217, led. 2006, doi: 10.1137/040603371.
- [24] G. S. Singh a L. Acerbi, „PyBADs: Fast and robust black-box optimization inPython“, *J. Open Source Softw.*, roč. 9, č. 94, s. 5694, úno. 2024, doi: 10.21105/joss.05694.
- [25] D. Eriksson, M. Pearce, J. Gardner, R. D. Turner, a M. Poloczek, „Scalable Global Optimization via Local Bayesian Optimization“.
- [26] „Surrogate optimization — pySOT documentation“. Viděno: 9. květen 2026. [Online]. Dostupné z: https://pysot.readthedocs.io/en/latest/surrogate_optimization.html

ZOZNAM SKRATIEK, SYMBOLOV, OBRÁZKOV A TABULIEK

Zoznam skratiek

BO – Bayesovská optimalizácia
 BADS – Bayesian Adaptive Direct Search
 CFD – Computational Fluid Dynamics
 CSV – Comma-Separated Values
 DYCORS – Dynamic Coordinate Search
 EI – Expected Improvement, očakávané zlepšenie
 FLORIS – FLOW Redirection and Induction in Steady State
 GP – Gaussovský proces
 IOH – Iterative Optimization Heuristics
 JSON – JavaScript Object Notation
 LAQN – London Air Quality Network
 LCB – Lower Confidence Bound, dolná hranica spoľahlivosti
 MADS – Mesh Adaptive Direct Search
 NO₂ – oxid dusičitý
 PI – Probability of Improvement, pravdepodobnosť zlepšenia
 PyBADS – Python implementácia algoritmu Bayesian Adaptive Direct Search
 PySOT – Python Surrogate Optimization Toolbox
 RBF – Radial Basis Function, radiálna bazová funkcia
 SOP – Surrogate Optimization with Pareto center selection
 SRBF – Stochastic Radial Basis Function
 TuRBO – Trust Region Bayesian Optimization
 UCB – Upper Confidence Bound, horná hranica spoľahlivosti

Zoznam symbolov

x – vektor rozhodovacích premenných
 x_i – konkrétny bod priestoru riešení alebo poloha i -tej turbíny
 x_* – nový bod, v ktorom sa vykonáva predikcia
 $\mathcal{X}$ – priestor prípustných riešení
 X – množina trénovacích vstupných bodov
 $f(x)$ – pôvodná cieľová alebo kritériálna funkcia
 $\hat{f}(x)$ – náhradný model aproximujúci cieľovú funkciu
 y_i – pozorovaná hodnota cieľovej funkcie v bode x_i
 ε_i – náhodná chyba merania alebo simulácie
 $\mathcal{D}$ – množina trénovacích dát
 B – rozpočet evaluácií
 N_{eval} – počet vykonaných evaluácií cieľovej funkcie
 w_i – váha v modeli radiálnych bazových funkcií

λ_i – koeficient radiálnej bázeovej funkcie
 $\phi(r)$ – radiálna bázeová funkcia
 r – vzdialenosť medzi bodmi
 $p(x)$ – polynomiálny člen v RBF modeli
 a_i – koeficient polynomiálneho modelu
 $m(x)$ – stredná funkcia Gaussovského procesu
 $k(x, x')$ – kovariančná funkcia Gaussovského procesu
 $K(X, X)$ – kovariančná matica trénovacích bodov
 σ_n^2 – rozptyl šumu pozorovania
 I – jednotková matica
 $\mu(x)$ – posteriorná stredná hodnota predikcie
 $\sigma(x)$ – smerodajná odchýlka predikcie
 $\sigma^2(x)$ – rozptyl predikcie
 $I(x)$ – zlepšenie oproti aktuálnej najlepšej hodnote
 $EI(x)$ – očakávané zlepšenie
 $PI(x)$ – pravdepodobnosť zlepšenia
 $LCB(x)$ – dolná hranica spoľahlivosti
 f_{best} – najlepšia doteraz nájdená hodnota cieľovej funkcie
 z – normalizovaná hodnota používaná pri výpočte EI
 $\Phi(z)$ – distribučná funkcia štandardného normálneho rozdelenia
 $\varphi(z)$ – hustota štandardného normálneho rozdelenia
 ξ – tolerancia zlepšenia pri PI
 κ – parameter určujúci mieru preskúmania pri LCB
 S – množina vybraných senzorových pozícií
 $|S|$ – počet prvkov množiny S
 k – počet vybraných senzorových pozícií
 Ω – kandidátna oblasť alebo množina prípustných lokalít
 $J(S)$ – kritérium kvality zvoleného rozmiestnenia senzorov

Zoznam obrázkov

Obr. 1.: Postup optimalizácie s využitím náhradného modelu
 Obr. 2: Porovnanie akvizičných funkcií PI, EI a UCB nad rovnakým posteriorným modelom.
 Obr. 3: Iteračný princíp Bayesovskej optimalizácie.
 Obr. 4: Dve fázy algoritmu PyBADS.
 Obr. 5: Princíp algoritmu TuRBO.
 Obr. 6: Ilustrácia vplyvu wind wake efektu na rozmiestnenie veterných turbín.
 Obr. 7: Príklady problémových inštancií LAQN-BO pre rôzne dni.
 Obr. 8: Konvergenčný priebeh algoritmov PyBADS a TuRBO pri úlohe rozmiestnenia environmentálnych senzorov.
 Obr. 9: Konvergenčný priebeh konfigurácií knižnice PySOT s náhradným modelom RBF pri úlohe rozmiestnenia environmentálnych senzorov.

- Obr. 10: Konvergenčný priebeh konfigurácií knižnice PySOT s polynomiálnym náhradným modelom pri úlohe rozmiestnenia environmentálnych senzorov.
- Obr. 11: Konvergenčný priebeh konfigurácií knižnice PySOT so stratégiou DYCORDS pri náhradnom modeli RBF a polynomiálnom náhradnom modeli.
- Obr. 12: Konvergenčný priebeh konfigurácií knižnice PySOT so stratégiou SRBF pri náhradnom modeli RBF a polynomiálnom náhradnom modeli.
- Obr. 13: Konvergenčný priebeh konfigurácií knižnice PySOT so stratégiou SOP pri náhradnom modeli RBF a polynomiálnom náhradnom modeli.
- Obr. 14: Súhrnný konvergenčný priebeh všetkých porovnávaných algoritmov pri úlohe rozmiestnenia environmentálnych senzorov v logaritmickej mierke osi počtu evaluácií.
- Obr. 15: Konvergenčný priebeh algoritmov Random Search, PyBADs a TuRBO pri konfigurácii s 5 turbínami.
- Obr. 16: Konvergenčný priebeh algoritmov Random Search, PyBADs a TuRBO pri konfigurácii s 10 turbínami.
- Obr. 17: Konvergenčný priebeh algoritmov Random Search, PyBADs a TuRBO pri konfigurácii s 20 turbínami.
- Obr. 18: Konvergenčný priebeh konfigurácií PySOT s Gaussovským procesom a akvizičnými funkciami EI a LCB pri konfigurácii s 5 turbínami.
- Obr. 19: Konvergenčný priebeh konfigurácií PySOT s Gaussovským procesom a akvizičnými funkciami EI a LCB pri konfigurácii s 10 turbínami.
- Obr. 20: Konvergenčný priebeh konfigurácií PySOT s Gaussovským procesom a akvizičnými funkciami EI a LCB pri konfigurácii s 20 turbínami.
- Obr. 21: Konvergenčný priebeh konfigurácií PySOT s RBF náhradným modelom a stratégiami DYCORDS, SRBF a SOP pri konfigurácii s 5 turbínami.
- Obr. 22: Konvergenčný priebeh konfigurácií PySOT s RBF náhradným modelom a stratégiami DYCORDS, SRBF a SOP pri konfigurácii s 10 turbínami.
- Obr. 23: Konvergenčný priebeh konfigurácií PySOT s RBF náhradným modelom a stratégiami DYCORDS, SRBF a SOP pri konfigurácii s 20 turbínami.
- Obr. 24: Konvergenčný priebeh konfigurácií PySOT s polynomiálnym náhradným modelom a stratégiami DYCORDS, SRBF a SOP pri konfigurácii s 5 turbínami.
- Obr. 25: Konvergenčný priebeh konfigurácií PySOT s polynomiálnym náhradným modelom a stratégiami DYCORDS, SRBF a SOP pri konfigurácii s 10 turbínami.
- Obr. 26: Konvergenčný priebeh konfigurácií PySOT s polynomiálnym náhradným modelom a stratégiami DYCORDS, SRBF a SOP pri konfigurácii s 20 turbínami.
- Obr. 27: Konvergenčný priebeh konfigurácií PySOT so stratégiou DYCORDS pri RBF a polynomiálnom náhradnom modeli pri konfigurácii s 5 turbínami.
- Obr. 28: Konvergenčný priebeh konfigurácií PySOT so stratégiou DYCORDS pri RBF a polynomiálnom náhradnom modeli pri konfigurácii s 10 turbínami.
- Obr. 29: Konvergenčný priebeh konfigurácií PySOT so stratégiou DYCORDS pri RBF a polynomiálnom náhradnom modeli pri konfigurácii s 20 turbínami.
- Obr. 30: Konvergenčný priebeh konfigurácií PySOT so stratégiou SRBF pri RBF a polynomiálnom náhradnom modeli pri konfigurácii s 5 turbínami.

Obr. 31: Konvergenčný priebeh konfigurácií PySOT so stratégiou SRBF pri RBF a polynomiálnom náhradnom modeli pri konfigurácii s 10 turbínami.

Obr. 32: Konvergenčný priebeh konfigurácií PySOT so stratégiou SRBF pri RBF a polynomiálnom náhradnom modeli pri konfigurácii s 20 turbínami.

Obr. 33: Konvergenčný priebeh konfigurácií PySOT so stratégiou SOP pri RBF a polynomiálnom náhradnom modeli pri konfigurácii s 5 turbínami.

Obr. 34: Konvergenčný priebeh konfigurácií PySOT so stratégiou SOP pri RBF a polynomiálnom náhradnom modeli pri konfigurácii s 10 turbínami.

Obr. 35: Konvergenčný priebeh konfigurácií PySOT so stratégiou SOP pri RBF a polynomiálnom náhradnom modeli pri konfigurácii s 20 turbínami.

Obr. 36: Súhrnný konvergenčný priebeh všetkých porovnávaných algoritmov v logaritmickej mierke vodorovnej osi počtu evaluácií pri konfigurácii s 5 turbínami.

Obr. 37: Súhrnný konvergenčný priebeh všetkých porovnávaných algoritmov v logaritmickej mierke vodorovnej osi počtu evaluácií pri konfigurácii s 10 turbínami.

Obr. 38: Súhrnný konvergenčný priebeh všetkých porovnávaných algoritmov v logaritmickej mierke vodorovnej osi počtu evaluácií pri konfigurácii s 20 turbínami.

Zoznam tabuliek

Tab. 1: Súhrnné porovnanie kvality dosiahnutých riešení pri úlohe rozmiestnenia environmentálnych senzorov.

Tab. 2: Efektivita využitia evaluácií pri úlohe rozmiestnenia environmentálnych senzorov.

Tab. 3: Časová náročnosť algoritmov pri úlohe rozmiestnenia environmentálnych senzorov.

Tab. 4: Poradie algoritmov podľa priemerného najlepšieho výkonu pri 5 turbínach.

Tab. 5: Poradie algoritmov podľa priemerného najlepšieho výkonu pri 10 turbínach.

Tab. 6: Poradie algoritmov podľa priemerného najlepšieho výkonu pri 20 turbínach.

Tab. 7: Efektivita využitia evaluácií pri konfigurácii s 5 turbínami.

Tab. 8: Efektivita využitia evaluácií pri konfigurácii s 10 turbínami.

Tab. 9: Efektivita využitia evaluácií pri konfigurácii s 20 turbínami.

Tab. 10: Časová náročnosť algoritmov pri konfigurácii s 5 turbínami.

Tab. 11: Časová náročnosť algoritmov pri konfigurácii s 10 turbínami.

Tab. 12: Časová náročnosť algoritmov pri konfigurácii s 20 turbínami.

ZOZNAM PRÍLOH

Projektový repozitár na webovom portále Github

PŘÍLOHY

https://github.com/HighPool/thesis_bosm_climate/